

Quantum Fault Tolerance Meets Toom's Contours

David Ponarovsky

July 26, 2026

TODO:

1. Move from 'assignments' to 'chains'.
2. Enter the computational model to the Notations (and preliminaries).
3. Enter that we care only about pauli errors somewhere.
4. Rewrite the all outline.
5. Advance the sweeprule definition prior to the moving into the infinite grid.
6. Define what is mean a finite chain.
7. Logical operators - change the decoding procedure.
8. Mention that in 'applying the sweep rule on a chain', we refer to the application of the first stage of the sweep cycle. Namely, all the vertices work, yet we don't care about the dual base.
9. In all the claims about chains, set x to note the default chain (instead z which we keep to stabilizers)
10. Change all the 'Pauli' to capital letter.
11. Add after the definition of the computation DAG, that the order J has to respect time order.

Abstract

We suggest a fault tolerance scheme in the circuit model that both uses the 4D Toric code and has a constant-depth correction cycle. Until now, in the circuit model, the use of a topological code as the primary encoding and a constant-depth correction cycle were only achieved separately; this is the first work to achieve them simultaneously. In addition, to date, the 4D-Toric code has been proposed as a memory element in the thermal noise model. Our work shows that it is also suitable as a memory in the circuit model. To achieve that, we adopt the sweep rule defined in [KP19], as the main component of the correction cycle, thereby bypassing the need for a hierarchical correction cycle. The proof is based on alternative proofs of Gacs's theorems [BS88], [Gac21], which are distinguished from the standard approaches in that they trace the past that leads to error clusters.

Contents

1	Introduction	2
1.1	Background	2
1.2	Stating Our Results	3
1.3	Discussions, Applications, and Further Directions	4
2	Outline of the Proof	4
3	Notations and Preliminaries	14
4	The (d, d')-Dimensional Toric Code	18
4.1	Infinite Sides Torics	26
4.2	Sweep Rule	32
4.3	Reduced Error	35
4.4	Shrinking	39
4.4.1	The Sweep-Rule Phase in the Dual Code	41
4.4.2	Ideal decoder	44
4.5	Intermediate	44
4.6	Gottesman argument	46
4.7	Toric Encoded Circuits and Toom's Contours	46
4.8	Space-Time Graph	49
4.9	Explantation of a Spanned Slice	55
4.10	Proof of the Theorem	58
5	Appendix	61
5.1	Base case for Claim 4.33	62
5.2	Time Coarse-Graining	63
5.3	Checks Commutation	65
5.4	Counting Subtrees.	65
5.5	Tree Separator Lemma	66
6	Garbage	67

1 Introduction

1.1 Background

Fault-tolerant constructions take ideal quantum circuits and establish their logical equivalence such that they endure fault occurrences. Their efficiency is typically measured by the amount of additional resources – qubits, computation time, range connectivity, etc. – required by the resulting circuit, beyond the scope of theoretical discussion; those parameters translate into the machine complexity and energy cost required to run the circuits. Minimizing overheads brings quantum computing applications closer to the near future.

Since the introduction of the quantum threshold theorem [AB99], which showed that quantum fault tolerance is in principle feasible, the main effort in subsequent work has focused on reducing the additional qubit overhead. A line of work has led to a constant-factor overhead in qubits, first in the instant-classical computation model [Got14], then in the standard circuit model [Gro19], [FGL18], [NP25]. Yet, although the best qubit overhead was reached, the depth overhead was only slightly improved from poly-logarithmic to logarithmic [NP25]. In fact, it seems that the same means that are used to reduce the qubit overhead are the ones that challenge reducing the depth overhead.

A key concept for reducing qubit overhead is to encode qubits with a high-rate quantum LDPC code. For those codes, except for the control-not, there are no known gates that can be realized in constant depth; furthermore, since the structure of the codes is relatively complicated – based on the skeletons of expander graphs – it seems hard to look for realizations of their logical operators. Without a constant-depth realization of logical operators, any construction that doesn’t interfere with the order of the logical operations would admit a non constant-depth overhead.

Contrary to high-rate LDPC codes, topological error codes have a geometrical image that is well understood and, as a result, a wide range of works suggest efficient realization of their logical operators. For example, consider the 3D-color code [Bom15], fold-transversal gates on the surface code [BB24], and realization of the Clifford group on the 2D Toric [GV24].

Another branch of work, whose primary goal was to show that the embedding dimension of the computation can be reduced to 2D [BDL26], and more recently [DST26], uses topological codes as the main error correction code. Those constructions simplify the hardware realization; they suggest that quantum computers can be implemented by nearest-neighbor interactions automata – what is believed to be easier to develop. Yet, as in the threshold theorem [AB99], and in contrast to the works that use high-rate LDPC codes, they rely on a hierarchical correction cycle whose execution time grows with system size. Hence, even though those directions have the potential to overcome the non-constant depth overhead per gate, it seems that the correction cycle imposes a barrier for them to come up with a constant factor in depth-overhead construction.

1.2 Stating Our Results

In this work, we present a fault-tolerance construction that uses the 4D Toric code and has a constant-depth ‘correction’ cycle. Or in other words, a construction uses a relatively simple code, yet doesn’t admit the cost of hierarchical correction cycle.

The computation model we consider is quantum circuits subject to *local-stochastic noise*, where only qubits are subjected to the noise, while classical bits are non-noisy. Classical computation is allowed, and the classical depth matters. Namely, at each time unit, only a constant depth of classical computation can be done. Local stochastic channel is defined to act such that the probability of any subset of locations S (space-time coordinates), which is picked prior to the channel’s action, being affected decays exponentially with the subset size.

Definition 1.1 (Local stochastic noise channel). *Let $p \in (0, 1)$. We say that the noise channel $\mathcal{N}_{\mathcal{E}}$ is local stochastic with rate p if for every finite set of locations $S \subset \mathbb{N} \times \mathbb{N}$,*

$$\Pr_{\mathcal{E} \sim \mathcal{N}_{\mathcal{E}}} [S \subset \text{Loc}(\mathcal{E})] \leq p^{|S|}.$$

For circuits $\tilde{C}$ and C , we say that $\tilde{C}$ simulates C with probability p if, when one stops a noisy run of $\tilde{C}$ at a midpoint, one can, with probability p , recover by non-noisy operation an 'image' of a midpoint in the running of C , and if stopping at the last time point of $\tilde{C}$ enables recovery of the 'image' of C at the end of its run. We proved the following theorem:

Theorem (Informal version of Theorem 4.67). *There exists a constant $p_0 \in (0, 1)$ such that for any local stochastic noise channel $\mathcal{N}_{\mathcal{E}}$ with rate $p < p_0$, a quantum circuit C with depth μ and width w , and a big enough integer n , there exists a 'logical equivalence' circuit $\tilde{C}$ with depth $\Theta(\mu n)$ and width $\Theta((w + \mu)n)$ that simulates C with probability:*

$$1 - \Theta(\text{poly}(w, \mu, n)) p^{\Theta(n^{\frac{1}{4}})}$$

Furthermore, $\tilde{C}$ has the following two properties:

1. *After a short initialization stage (at depth $\Theta(\log n)$), each of the qubits is encoded in the n -length 4D Toric code.*
2. *An error correction cycle takes $O(1)$ depth, and the gates in each correction cycle don't depend on time. In particular, if C is the identity circuit, then after the initialization stage, the layers in $\tilde{C}$ are the same.*

As mentioned earlier, both code simplicity and constant depth have already been achieved: the former as part of the quantum automation line of work, and the latter through the use of high-rate error correction codes. However, this is the first construction to achieve both simultaneously. Table 1 summarizes and compares the different works:

WORKS LINE	ERROR CORRECTION CODE	CONSTANT-DEPTH CORRECTION CYCLE	DEPTH/QUBIT OVERHEAD	REF
CONCATENATION	Concatenation of fixed length code	✗	$f(x)$	[AB99]
HIGH-RATE	Quantum expanders/ good QLDPC	✓	$f(x)$	[Gro19], [FGL18], [NP25]
QUANTUM AUTOMATA	2D-Toric code	✗	$f(x)$	[BDL26], [DST26]
OURS	4D-Toric code	✓	$f(x)$	

Table 1: Comparing different types of works on fault-tolerance.

1.3 Discussions, Applications, and Further Directions

Our result can be understood as follow: when the qubit overhead is not a consideration, it means that the 4D Toric code is as good as the good quantum LDPC / Hyperproduct code. That translates to the two following applications:

1. New approach for quantum fault tolerance: With the assistance of resource states and gate teleportation, we obtain a fault-tolerant construction with logarithmic depth overhead. Our proof is inherently different from prior works, describes in higher resolution the expected error, and might be used as a springboard for a constant depth overhead construction.
2. Memory in the circuit model: Protecting a 4D Toric encoded state for an exponentially long time is achieved by performing the same operation at each time unit, suggesting that the 4D Toric code is a promising candidate for memory in the circuit model. Combined with evidence that the 4D Toric code also functions as memory in thermal noise models [Ali+08], and given its relative simplicity, it appears to be a natural choice for real-world memory applications.

2 Outline of the Proof

In studying fault tolerance, the typical method is to first distinguish between configurations that can be tolerated and those that cannot. For example, in von Neumann-style proofs [Neu56], it is common to track the number of flipped bits; if fewer than half are flipped, the logical states can be recovered. In the work of Aharonov and Ben-Or [AB99], a logical bit at the n level admits a logical error if at least a constant fraction of the $n - 1$ level logical bits that compose it admit an error. In the work of [Gro19], a tolerated configuration is one in which all encoded registers can be considered as distributed according to sufficiently weak local stochastic noise. The next step is to show that from a tolerated configuration, we move with high probability to another tolerated configuration. For our goals, each approach presents a challenge: The concatenation method, by definition, requires non-trivial depth computation at each step. Tracking the number of flipped bits fails almost immediately, as topological error correction codes have sublinear distance when a linear number of bits is expected to be flipped. And preserving the local stochastic noise distribution seems to heavily rely on the expansion properties [Gro19] of the underlying structure, where for topological codes, we expect to forge a 'spatial' dependence.

In contrast, Berman and Simon provided a proof that, instead of tracking the dynamics, considers the entire run as a possible outcome, ruling out improbable runs only at the end [BS88]. We outline their idea informally as follows: a logical bit is represented by $n \times n$ bits set on the vertices of the grid. In each correction cycle, the bits determine their value by taking the majority of their current value, and their north and east neighbors values, a rule known as Toom's rule. For simplicity, let's ignore the logical operations. Now, given a run, and in particular the set of space-time locations of faults, consider the following directed graph $\mathcal{H} := (V_{\mathcal{H}}, E_{\mathcal{H}})$ induced by it:

1. If at time t , the bit at vertex v diverges relative to the ideal run, then add (v, t) to the vertices.
2. If at time t , vertex v 'leads' one of its neighbors u to diverge, then connect (v, t) and $(u, t + 1)$ by an 'arrow'.
3. Connect any two vertices in $V_{\mathcal{H}}$, which are connected to a third by 'arrows', by a 'fork'.

Notice that the vertices of $\mathcal{H}$ with no incoming arrows correspond to faults; let's call them leaves. Thus, the probability of each running can be bounded by the number of leaves in the induced graph. Their key lemma states that a cluster of flipped bits with a 'perimeter' s , connected by arrows, induces a subgraph with at least $\Theta(s)$ leaves¹. In other words, the probability of a connected component (by arrows) being flipped decays exponentially with its perimeter.

The graph $\mathcal{H}$ was later named Toom's contours, a term that, to the best of our knowledge, was coined in [SST26].

We chose to follow their approach. The rest of the section reviews the challenges that we encountered on the way to generalize it and the ideas to overcome them.

The Toric Code, the Sweep-Rule, and the Infinite Plane. The first issue was finding an analog for Toom's rule. We decided to adopt the sweep rule invented by Preskill and Kubica in [KP19], which they introduced as a local rule for any CSS code induced by a simplicial Toric, equipped with a notion of 'positive' and 'negative' directions (in their paper 'future' and 'past'). They also provided a rigorous proof that the iterative non-noisy applications of the sweep rule succeed in decoding faults that were sampled independently. For our purposes, we had to extend their proof for cubical Toric². The main reason for this is that the alignment of the positive/negative directions with the axes direction of $\mathbb{R}^4$ space allows quantifying the 'perimeter' of error clusters. Yet, we are sure that our proof can be adjusted to support simplicial Toric complexes as well, and we leave it for further work. In the formal section, we define and prove the claims for Torices in any dimension greater than two, except for Claim 4.33, which we proved only to dimension four. (Yet a numerical study hints that the claim might also be generalized; see ??.)

In complete contrast to typical works in topology, where global properties are studied and local characterization is left as general as possible, here we need notations to describe the local view of each vertex of the complex. We needed it for defining the sweep-rule action and stating claims about how it shrinks errors. That led us to the $\theta_{\mathbf{v}, S'}$ notation defined in detail in Definition 4.1. In general, we denote by S the generator set of the Toric, and for a subset of generators $S' \subset S$, we define $\theta_{\mathbf{v}, S'}$ to be the face whose 'bottom-left' corner is the vertex v , and sides of the face correspond to edges on paths obtained by stepping through subsets of S' (for example, if $S' = \{g_1, g_2, g_3\}$ then $\{g_2 g_1 v, g_3 g_2 g_1 v\}$ is an edge of $\theta_{\mathbf{v}, S'}$). Figure 1 shows how the 2D-Toric complex is characterized via $\theta_{\mathbf{v}, S'}$ notation.

¹And number of edges $\leq \Theta(\text{leaves})$.

²The standard Toric codes.



Figure 1: Here we consider a 2D Toric complex. We name the set of generators $S = \{g_1, g_2\}$, where g_1 is the generator that sends us one step in the north direction, and g_2 sends us in the east direction. For vertices v, u, w , we mark in blue three faces that are rooted at them.

The 4D-Toric code is defined as the binary assignment over the 2D-faces (plaquettes), where the Z -stabilizers (parity checks) are set on the edges, namely restrict the Z product of the faces adjacent to each edge, and the X -stabilizers (phase checks) are set on the 4D-faces (hypercubes). To present the sweep-rule, we need to have a definition of directions. For this section, we will be satisfied with the following hand-waving: For a vertex v , the bits and checks on the first 'quarter' of it (high-dimensional quarter) will be named the positive bits and checks of v .

Definition 2.1 (Sweep Rule (Inforaml)). *Let z be a binary assignment on the Toric faces representing 'error'. The sweep rule decoding procedure at a vertex v is defined as follows:*

Measure the positive checks of v and look for an assignment $\tilde{z}$ on the positive bits of v such that the syndromes of the positive checks of v assign the same local syndrome for both z and $\tilde{z}$.

See Definition 4.26 for the formal version. An error correction cycle, or a sweep-cycle, consists of applying the sweep-rule to each vertex and then doing the same in

the dual space. Figure 2 provides an example of the result of the sweep rule given a syndrome.



Figure 2: An example of applying the sweep rule within the $(3, 2)$ -Toric code. In the $(3, 2)$ -Toric code, bits are placed on the squares and Z -stabilizers (parity checks) on the edges. The view is restricted to two vertices, u and v . On the left, the unsatisfied positive checks of u and v are highlighted in blue. On the right, the bits to be flipped are marked in green. Given the syndrome, vertex v flips one of its positive faces, whereas for u , there is no subset of positive faces that, when flipped, decreases the number of local unsatisfied checks. Therefore, u flips no bit.

The way we quantify the ‘perimeter’ of an error cluster is by considering the (hyper)triangle that bounds the vertices supporting the cluster. The poles of an error cluster are the vertices that lie on the sides of the triangle. We followed [BS88], and by embedding the Toric in the infinite plane, we were able to identify the poles as the max/min points of linear functions which we call *potential walls*. Then, we show that the sweep rule advances only one pole, and hence one side, forward the others. As a consequence, a sweep-cycle shrinks the bounding triangle. The embedding restricts periodicity conditions on the assignments. We encounter this twice: first, an assignment on the Toric’s faces that closes a non-trivial cycle translates to an infinite assignment in the infinite plane. In that case, no triangle bounds the error, which agrees with the fact that a logical error—in that context, an error that changes the logical states—should be ‘decoded’ to a different codeword. From a technical perspective, it comes into effect as the claims about shrinking assume that a given assignment is a connected finite assignment. The second time we have to deal with the mapping onto the infinite plane is in the calculation of the probability of a given Toom’s contour. Since different leaves (faults) of Toom’s contour might be associated with the same bit and hence dependent, to bypass this, we took from [Gac21] the claim that states any such graph has a subgraph whose leaves are independent, and their number is at least a constant factor of the original leaves.

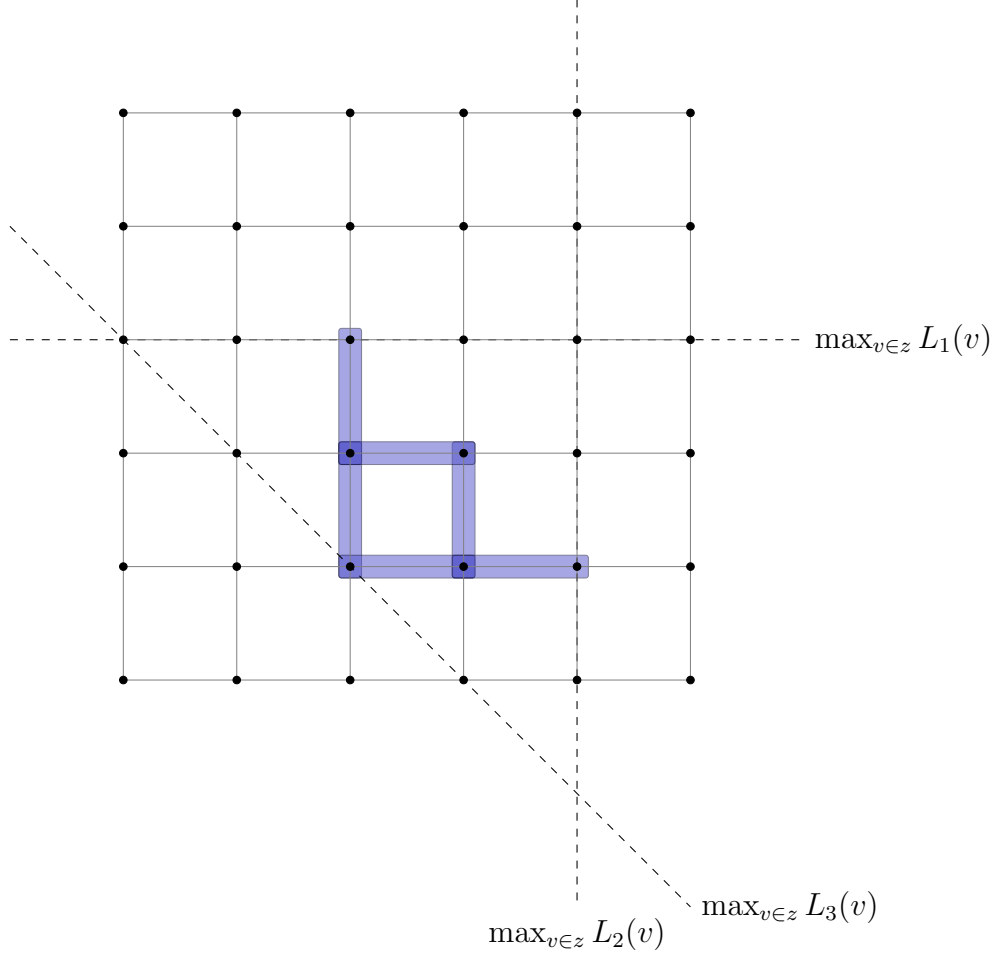


Figure 3: A graphical example of the maximal values of the potential walls. Here we consider the $(2, 1)$ -Toric code, namely the bits are set on the edges where the Z -stabilizers (parity checks) are set on the vertices. The grid is a finite 6×6 patch of the infinite 2D plane. The edges marked in blue correspond to a bits assignment z . Each of the dashed lines marks the line of vertices whose L_i value converges with the maximal value reached by a vertex in z . Moving into the infinite plane enables us to determine those lines and draw the triangle that bounds z .

See Section 4.1 for embedding into the infinite plane, Section 4.4 for the proof of shrinking, and Section 5.5 for the claim about the existence of a sufficient subgraph.

The Intermediate Theorem. Now that we have the code and decoder, the next difference we have to address is how to classify an error. Unlike the classical regime, flipping a stabilizer's bits doesn't count as a fault. Moreover, the sweep-rule discussed above doesn't shrink stabilizers (as it acts trivially on them). Therefore, we need to redefine what is considered a divergence from the ideal computation. As a result, instead of defining the vertices of Toom's contour graph to be the flipped vertices as

in [BS88], we define them to be bits³ that encounter unsatisfied checks in their local view.

We then face another challenge: if we analyze Toom’s contour, which is induced by the single time unit running consisting of flipping all the bits at once, then in the layer following the fault cycle, no bit sees an unsatisfied check. In the language of Toom’s contour, as will be defined later, the Toom’s contour is empty, and no bounds on the event’s probability to occur can be inferred using it. We address this by introducing an analysis framework where all computation, including faults, is imagined to occur sequentially — in steps. Within this framework, we can prove an *intermediate theorem* stating that on the path to a logical error configuration, one must pass through a step with a large cluster of unsatisfied checks. Thus, a sufficient condition for an error to occur in some time unit is the existence of a step with a sufficiently large observed syndrome. In the proof of the theorem, we show that this condition happens with negligible probability. To elaborate on the issue, we first need to present the computation model.

We model a noisy running of a given circuit C as follows: first, we sample space-time locations for the faults and denote the outcome by $\mathcal{E}$. Then, we impose layers of computation alternately, such that the even and odd layers are taken from $\mathcal{E}$ and C according to their order. The combination of the circuit C with a subset of faults $\mathcal{E}$ defines a possible running, and we call it a subjection of C to $\mathcal{E}$ and denote it by $C[\mathcal{E}]$. Furthermore, we showed in Claim 3.8 that for a local-stochastic noise model, unbalancing partition to noisy and logical layers, for example placing a noisy layer after every three logical layers of C , is equivalent to odd-even alternations up to a rescaling of the noise rate (by a constant). We use that since the error correction cycle of the sweep rule takes more than a single layer.

A Tanner graph of an error correction code is a bipartite graph where the left vertices correspond to bits and the right vertices correspond to checks, and there is an edge between a left vertex and a right vertex if they match a bit and a check supported by it. A logical error is an error where an attempt to decode it might lead to a different codeword (different from the original prior to the noise). As stated in [Got14], a sufficient condition for an error to be correctly decodable is that, relative to the Tanner graph of the code, the error is a decomposition of connected components (where two bits are connected if they share a check) such that each of them is comparatively small (at least by a factor of half) to the code distance.

As mentioned, we are going to analyze the likelihood of getting logical errors by looking back in time and asking what a run that has led to them is supposed to look like. The sweep rule ensures that an error cluster below a certain size shrinks. The problem is that a single fault layer (i.e., a layer whose source is in $\mathcal{E}$) might increase the error cluster above that size. At first glance, one might be tempted to show that the expansion of error clusters beyond a fraction of the code distance happens with negligible probability, but that calculation soon becomes complicated when considering the merging of disjoint clusters. Instead, we split the computation into steps. In each step, only one gate or one fault is applied, and steps are ordered such that they agree

³In fact: vertices on the bits’ support, see Section 4.7.



Figure 4: Example of a subjected quantum circuit: the even layers (0, 2, 4) contain the original gates, while the faults are imposed on the odd layers and marked in red.

with the computation. Figure 4 illustrates a quantum circuit and a graph of steps corresponding to it.

Since the gates and the faults have a bounded fan-in/out⁴ In each step, the maximal size of an error cluster can increase by at most a constant factor, with the maximal increase occurring when multiple clusters are merged. Equipped with that insight, we are able to infer the intermediate theorem.

Section 3 formalize the computation model and the computation graph presentation (splitting to steps). In particular, see ?? and ?? for the formal proof of the intermediate theorem.

Description of the Construction. Similarly to other fault tolerance approaches, we use the concatenation construction in order to initialize zeros and resource states encoded in the 4D-Toric code, as referred to in [Gro19], [Got14]. From there, we assume that all the quantum registers are encoded in the 4D-Toric code, and we name each of the encoded block a quantum register. The resources states are spread across two registers.

Then, after the initialization step, the logical cycles are divided into error correction, which is a single sweep-cycle, and 'logical operation' cycle. For our universal logical gate set, we choose CX (controlled NOT), H , S , and T gates. Except for the 'Controlled NOT' gate, which is realized by transversal wire-wise physical 'Controlled NOT' gates, all the others are implemented via gate teleportation [ZLC00].

The general realization by gate teleportation is done as follows: first, we XOR the target state into a resource state via CX , measure the input state and the first register of the resource state with a Bell measurement, the input register in the X basis, and the first register of the resource state in the Z basis. Then, on the classical outcomes, run a classical decoding procedure to extract the logical codeword. Meanwhile, on the second register of the resource, which is intended to contain the result of the action, perform a sweep cycle. At the end of the decoding, depending on the result, we apply either a transversal Pauli operator or the S gate (again by teleportation) depending on

⁴The faults are bounded-fan because we can assume that in the time units representation they are Pauli products and therefore in the steps presentation they are decomposed into single qubit Pauli matrices.

the original gate that was asked to simulate. The gate teleportation lasts for multiple logical cycles, depending on the depth required by the classical decoding procedure.

We emphasize that for measuring the input register in the X -basis, we apply the Hadamard through its fold-transversal realization [BB24], so the measurement is done in constant depth. However, the reason we don't implement the Hadamard by fold-transversal from the beginning is that we have failed to provide proof that restricts the expansion of the bounded-triangle⁵ by the fold-transversal realization. Since it guarantees that after the measurement the register will not observe more faults, we can tolerate a one-time blowup in the size of the error and still be able to decode the logical state in the register correctly.

In Section 4, we discuss in detail the structures of the logical codeword of the 4D-Toric code and the realization of its logical gates.

Toom's Contours. A sufficiently simple definition of Toom's contours in [BS88] would be a directed graph that describes a noisy run with the additional marking of diverged bits. In our case—the quantum case—the vertices of Toom's contour are neither bits nor checks; instead, they are the vertices on the Torices that 'diverged'. A proper conceptual definition would be:

A graph that describes a noisy run from the Torices' perspective. In particular, the vertices of the graph are the vertices of the Torices, equipped with a time coordinate, that are adjacent to unsatisfied checks.

We adjust their construction as follows: Denote the graph by $\mathcal{H} := (V_{\mathcal{H}}, E_{\mathcal{H}})$ and

1. If at time t , a vertex v , on the one the Toric complexes⁶ corresponding to one of the quantum registers, sees an unsatisfied check on it's local view, then add (v, t) to the vertices.
2. If at time t , vertex v sees an unsatisfied check, and at time $t + 1$ its neighbor u sees an unsatisfied check, then we think of the situation as ' v leads' to u ' and connect (v, t) and $(u, t + 1)$ with an 'arrow'. The vertices v and u might belong to different registers (which matches the case that at time t we apply an operator on both registers). For convenience, we don't require u at time t to see only satisfied checks.
3. As in the classical case, connect any two vertices in $V_{\mathcal{H}}$ that are connected to a third by 'arrows' with a 'fork'.

Later, we incorporate the 'splitting into turns' idea into the formation of Toom's counter $\mathcal{H}$ -graphs, by gluing the layers corresponding to the first $t - 1$ layers with a final layer that matches a partial execution of the t th time unit. The layer encodes the situation of the running up to a certain step. Then, combining with the intermediate

⁵The permutation used in the Hadamard realizations mirrors the errors

⁶Actually the infinite grid.



Figure 5: Above: The application of a sweep-cycle, where the green faces represent the bits assignment z , and the blue edges correspond to unsatisfied checks ∂z . The vertices in ∂z are labeled as $v_1, v_2, \dots, v_8$. Below: The Toom's contour derived from the single computation layer that matches the cycle. Arrows are colored in blue, while forks are colored in green. Note that $(v_1, t+1)$ doesn't belong to the Toom's contour (since v_1 doesn't belong to the syndrome at time $t+1$).

theorem, we have that any running that admits a logical error derives a Toom's contour, with sufficiently large vertices at final layer.

The rest of the analysis follows [BS88]. We define *slices* at time t to be connected (by arrows) components of vertices at time t . Then, in the merging of two slices into one, it is guaranteed that faults induced by each are disjoint; otherwise, those two slices will be connected by arrows, namely - the same slice. For a *slice*, we define its size to be the perimeter of the triangle that would bound the vertices of the slice if they were projected on the same Toric. Then, by induction, we prove that Toom's contour that induces at time t a slice with size s has a spanning tree with at most and at least a constant times s edges and leaves (and therefore faults). That allows us to bound the probability of such a slice occurring by counting the number of possible spanning trees and assigning them the probability that the minimal number of faults occurs.

One technical change that we made and haven't discussed yet is that in the gluing of the last layer, which was defined to capture a partial layer execution up to a turn, we connect by arrows any two vertices that were added to $V_{\mathcal{H}}$ and share a check. By doing so, we ensure that connected components in the Tanner graph would also be connected by arrows in Toom's contours. We emphasize that such edges connection can be made only in the last stage, as it doesn't pass the induction step. However,

the final slice that is formed passes the desired inequalities only by constants and is therefore sufficiently good to derive the probability bound.

Section 4.7 defines Toom's contours, Section 4.8 defines the slices and states a claim connecting a slice to its predecessors, and Section 4.9 provides the induction proofs.

3 Notations and Preliminaries

We follow the standard notations commonly used in the literature. For an integer n , we denote by $[n]$ the set of integers $\{1, 2, \dots, n\}$. To present a countable set, which might or might not be finite, where the number of elements does not matter, we use curly brackets with a subscript denoting the first index of the set. For example, $\{u_i\}_{i=1}$ represents the set $u_1, u_2, \dots$. For finite sets, we might add a superscript to present the size of the set. If the first index is clear from the context, then we might omit the assignment. We use a colon inside curly brackets to specify a set of elements satisfying a condition, i.e., for sets A, B , the notation $\{a : a \in A, a \in B\}$ equals $A \cap B$. Curly brackets without subscripts or conditions refer to singletons. For example, for a given integer i , the set $\{u_i\}$ is the set that contains only the element u_i . The size of a set A is denoted by $|A|$. For an integer i , we denote by $\mathbf{1}_i$ the vector consisting of one at the i th coordinate and zero elsewhere. The char $|$ symbolizes a restriction, for example $x|_y$, is the restriction of the 'entity x ' to ' y '. The particular behavior of the restriction would change depending on the target type.

When not otherwise mentioned, we consider quantum states over qubits, each lying in a two-dimensional Hilbert space $\mathcal{H}_2$, and denote by $\mathcal{H}_{2^n}$ the Hilbert space of an n -qubit system. The $\otimes$ operator stands for tensor product, for example: $\mathcal{H}_{2^n} = \mathcal{H}_2^{\otimes n}$. The space of linear operators on the Hilbert space is denoted by $L(\mathcal{H})$. Pure quantum states and their dual presentations are denoted by $|\psi\rangle, \langle\psi|$, and in the matrix representation, we use $|\psi\rangle\langle\psi|$ to denote its density matrix. We will use ρ to denote a mixed state. The action of an operator A on a mixed state is written as $A \circ \rho$. For example, if A is a unitary operator, then $A \circ \rho = A\rho A^\dagger$. The letters, X and Z , will usually denote the single qubit X and Z Pauli operators. Yet, where the Pauli matrices are out of context, we use these letters for other variables. The gate 'Controlled Not' is denoted by **CX**, and the gates Hadamard, S, and T are denoted by:

$$\mathbf{H} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \mathbf{S} = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}, \mathbf{T} = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\frac{\pi}{4}} \end{bmatrix}$$

For a quantum gate U , we denote by $|U\rangle$ the resource state that can be used to simulate U . Notice that, in general, the resource state is not unique. In this work, we address specific states that will be discussed later. The function $\mathbf{Pr}[\cdot]$ is the probability function. In the language of groups, for a group $\mathcal{G}$ and elements $g_1, \dots, g_l$, we denote by $\langle\{g_1, \dots, g_l\}\rangle$ the subgroup generated by $g_1, \dots, g_l$.

Throughout the paper, we will deal with CSS codes [CS96], [Ste96]. For a linear subspace A , we use $|A\rangle$ to denote the uniform superposition over the vectors in A . A CSS code will usually be denoted by the pairs $\mathcal{C}_X, \mathcal{C}_Z$ such that the Z -stabilizers

match the codewords of $\mathcal{C}_Z^\perp$, and the logical codewords in the computational basis are the elements in $\mathcal{C}_X/\mathcal{C}_Z^\perp$. The logical zero codeword is $|\mathcal{C}_Z^\perp\rangle$. We denote the equivalence relation up to Z -stabilizer addition by $\sim_{\mathcal{C}_X}$. Namely, for $x_1, x_2 \in \mathbb{F}_2^n$, we have $x_1 \sim_{\mathcal{C}_X} x_2$ if $x_1 + x_2 \in \mathcal{C}_Z^\perp$. By the same way we define $\sim_{\mathcal{C}_Z}$.

The Computation Model. Informally, a quantum circuit is a placement of a collection of quantum gates, taken from a finite set, at different space-time locations. Each such location records when the gate is applied and on which qubits it acts. Formally:

Definition 3.1 (Quantum Circuit). *A space-time location is a pair $l = (t, S)$, where t is a time coordinate and $S = (s_1, \dots, s_\Delta)$ is an ordered tuple of qubit indices. For a finite collection of quantum gates*

$$\mathcal{G} \subset \left\{ g: L(\mathcal{H}_2^{\otimes \Delta}) \rightarrow L(\mathcal{H}_2^{\otimes \Delta}) \right\},$$

and integers $w, d \in \mathbb{N}$, a quantum circuit C of depth d and width w is a collection of tuples

$$C = \{u_i = (g_i, l_i)\}_i,$$

where each $g_i \in \mathcal{G}$ and each location $l_i = (l_{i,1}, l_{i,2})$ belongs to $\mathbb{Z}_d \times \mathbb{Z}_w^\Delta$. We call the tuples u_i the gates of C . They must satisfy the following compatibility condition: if two gates have the same time coordinate, then their supports are disjoint. Namely, for $u_i = (g_i, l_i), u_j = (g_j, l_j) \in C$, the equality $l_{i,1} = l_{j,1}$ implies that the sets of qubit indices appearing in $l_{i,2}$ and $l_{j,2}$ are disjoint. We denote by

$$\text{Loc}(C) := \{l_i : u_i = (g_i, l_i) \in C\}$$

the set of locations occupied by the gates of C . We define the t -layer of C to be its restriction to gates at time t :

$$C|_t := \{u_i = (g_i, l_i) : u_i \in C \text{ and } l_{i,1} = t\}.$$

Later, we will restrict $\mathcal{G}$ to contain **CX**, **H**, **S**, **T**, and measurements. To define the action of a quantum circuit C on a given mixed state, we first associate C with a directed acyclic graph.

Definition 3.2 (Associated Computation DAG). *Let $C = \{u_i\}_i$ be a quantum circuit with depth d and width w . We define the associated computation DAG of C , denoted by π_C , to be the directed acyclic graph $\pi_C = (\mathcal{U}, E)$ as follows:*

1. *The vertex set is the gate set of C , namely $\mathcal{U} = \{u_i\}_i$.*
2. *For $u_i = (g_i, l_i)$ and $u_j = (g_j, l_j)$, there is a directed edge $(u_i, u_j) \in E$ if and only if j is one of the minimizers of*

$$\{l_{k,1} : (g_k, l_k) \in \mathcal{U}, l_{k,1} > l_{i,1}, l_{k,2} \cap l_{i,2} \neq \emptyset\}.$$

Equivalently, u_j is one of the earliest later gates whose support intersects the support of u_i . There may be more than one such minimizer, so the out-degree may exceed one.

Suppose that $J = (j_1, j_2, \dots, j_k)$ is an ordering of the gate indices that agrees with a topological order induced by π_C . Then we define the associated partial circuits $C_J = \{C_{J,i}\}_{i=1}^k$ by

1. $C_{J,1} = \{u_{j_1}\}$.
2. For $1 \leq i < k$, set $C_{J,i+1} = C_{J,i} \cup \{u_{j_{i+1}}\}$.

We call $C_{J,i}$ the partial computation circuit up to step i .

Definition 3.3. Let ρ be a density matrix over w qubits, and let $C = \{u_i\}_{i=1}$ be a quantum circuit with depth d and width w . For any density matrix σ , quantum gate $g \in \mathcal{G}$, and location l , the application of the gate-location tuple (g, l) on σ , denoted by $(g, l) \circ \sigma$, is the application of g on σ where wires are ordered such that the first qubit in the input to g is $l_{2,1}$, the second is $l_{2,2}$, and so on. The result of the application of C on ρ , denoted by $C \circ \rho$, is given by iterative applications of the gates $\{u_i\}_{i=1}$ on ρ , according to a topological order induced by π_C .

We say that $\rho_1, \rho_2, \dots, \rho_k$ is a computation prefix if $\rho_1 = \rho$ and $\rho_{i+1} = u'_i \circ \rho_i$, where $\{u'_i\}_{i=1}$ is a prefix of a topological ordering of $\{u_i\}_{i=1}$. If $\{u'_i\}_{i=1}$ contains all the gates in $\{u_i\}_{i=1}$, then we call $\rho_1, \rho_2, \dots, \rho_k$ a running.

For a quantum circuit C , we would like to discuss its action in the presence of different outcomes of faults. The following definition encodes the running in the presence of a specific set of faults $\mathcal{E}$.

Definition 3.4 (C subjected to $\mathcal{E}$ every m computational layers). Let $C = \{u_i\}_i$ be a quantum circuit with depth d and width w , let $m \in \mathbb{N}$, and set

$$d_m := \left\lceil \frac{d}{m} \right\rceil.$$

Let $\mathcal{L} \subset [d_m] \times [w]$ be a set of locations, and let $\mathcal{E}$ be a quantum circuit, such $\text{Loc}(\mathcal{E}) = \mathcal{L}$. We name $\mathcal{E}$ subsets of faults, and we name the layers of $\mathcal{E}$ fault layers. The circuit C subjected to $\mathcal{E}$ every m computational layers, denoted by $C[\mathcal{E}]_m$, is obtained by setting the first layer to be the first layer of $\mathcal{E}$, then placing in order the layers of C and inserting one fault layer after every m consecutive C 's layers. Thus the second fault layer is inserted after the layers $1, \dots, m$ of C , the third is inserted after the layers $m+1, \dots, 2m$ of C , and so on. Concretely, each gate $u_i = (g_i, (t_i, S_i))$ of C is moved to the time coordinate

$$t_i + \left\lfloor \frac{t_i - 1}{m} \right\rfloor,$$

while each fault gate $(g, (t, S)) \in \mathcal{E}$ is placed at time coordinate

$$t(m+1).$$

When $m = 1$, the gates occupy the even time coordinates and the fault layers occupy the odd time coordinates. When the parameter is omitted, we mean $m = 1$. We denote the associated computation DAG of $C[\mathcal{E}]_m$ by $\pi_{C[\mathcal{E}]}$.

Remark 3.5. Formally, a fault circuit is just a particular quantum circuit whose gates are interpreted as faults rather than as computational operations. Thus the distinction between a quantum circuit and a fault circuit is semantic rather than structural: both are circuits in the sense of Definition 3.1, but they play different roles in the analysis.

Example 3.6. When $m = 3$, the ‘computational’ layers are sent to the times

$$2, 3, 4, 6, 7, 8, 10, 11, 12, \dots,$$

while the fault layers are sent to the times

$$1, 5, 9, 13, \dots$$

Definition 3.7 (Noise channel and local stochastic noise channel). A noise channel $\mathcal{N}_{\mathcal{E}}$ is a distribution over sets of fault locations. Let $p \in (0, 1)$, we say that the noise channel $\mathcal{N}_{\mathcal{E}}$ is local stochastic with rate p if for every finite set of locations $S \subset \mathbb{N} \times \mathbb{N}$,

$$\Pr_{\mathcal{E} \sim \mathcal{N}_{\mathcal{E}}} [S \subset \text{Loc}(\mathcal{E})] \leq p^{|S|}.$$

So, $\mathcal{N}_{\mathcal{E}}$ defines a probability space. The set of faults $\mathcal{E}$ is an atomic outcome, and for a given circuit C , the subjected circuit $C[\mathcal{E}]$ is a random variable. The following claim states that for local stochastic noise, the subjection of every 1-layer is equivalent to the subjection of every m -layer, up to a rescaling of the noise rate. Here, by equivalence, we mean that the actions of the subjected circuits are the same⁷.

We give the proof in the appendix for several reasons. First, the argument is not the core of this work. Second, versions of it appear in almost any fault tolerance work. Additionally, it relies on ‘propagation of entire layers,’ a term that conflicts with the ‘propagation to a step’ (Definition 4.42) needed for the intermediate theorem.

Claim 3.8. Let $\mathcal{N}_{\mathcal{E}}$ be a local stochastic noise channel with rate p , and let C be a quantum circuit whose gates have arity at most Δ . Fix an even integer c . Then there exists a local stochastic noise channel $\mathcal{N}_{\mathcal{E}_q}$ with rate

$$q := \left(2^{H(\Delta^{-2c})\Delta^c} p \right)^{\Delta^{-c}},$$

such that for any given state ρ it holds that:

$$C[\mathcal{E}]_1 \circ \rho = C[\mathcal{E}_q]_{c/2} \circ \rho$$

In particular, up to a constant depending only on Δ and c , the rate q behaves like $p^{\Delta^{-c}}$.

Equipped with this claim, we are allowed to assume having constant depth cycles that don’t observe faults.

⁷For a given input (mixed) state ρ , they yield the same output

4 The (d, d') -Dimensional Toric Code

The main error correction code that we will use is the Toric code invented by Kitaev. In the 2D case, the code is obtained by taking a Toric, tiling it with squares, setting qubits on the edges, and checks as follows: For every vertex, the parity of the qubits adjoining it is even, and for any square, the parity of the qubits lying on its sides, in the phase basis, is even. A high-dimensional version can be obtained by taking a high-dimensional Toric, encoded as a cubical complex, and then setting the X -checks, the qubits, and the Z -checks to be faces at sequenced dimensions. Throughout this paper, qubits will be set on d' -dimensional faces, and the X -checks and Z -checks on the $d' \mp 1$ dimensional faces. For a review, we refer to [Kit97], [Den+02].

For defining a decoder and proving its 'correctness,' we will have to overload notation. We derive the Toric complex from the skeleton of the Cayley graph of the cyclic group, using the generating subset as a collection of positive directions. The $\theta_{v,S'}$ encodes a face, one vertex v that serves as its 'root,' and a subset of directions S' . Formally:

Definition 4.1 (d -Dimensional Toric Complex). *For integers d and n , denote the group resulting from the d -directed power of the cyclic group of order n by $\mathcal{G}_n^d = \mathbb{Z}_n^{\times d}$, and denote the standard generating set of $\mathcal{G}_n^d$ by $S = \{1_i : i \in [d]\}$. We define the d -dimensional toric complex to be the hypergraph obtained from the Cayley graph $\text{Cay}(\mathcal{G}_n^d, S)$ by taking its vertices as the 0-faces and for any $i \in [d-1]$ we define the i -faces as follow: First, for a vertex $v \in \mathcal{G}_n^d$ and for a subset of generators $S' \subset S$ of size $|S'| = i$, the i -face rooted at v and spanned by S' , denoted by $\theta_{v,S'}$, is:*

$$\theta_{v,S'} := \bigcup_{I \subset S'} \left\{ \left(\prod_{g \in I} g \right) v \right\}$$

The i -faces of the complex are the collection of all the possible rooted spanned i -faces induced by $\text{Cay}(\mathcal{G}_n^d)$. We will denote them by $\mathcal{F}_i$.

Additionally, we name n and d , the side length and the dimension of the complex.

Definition 4.2 (Positive Edges and Cubes). *For $u, v \in \mathcal{G}_n^d$, and $S'_1, S'_2 \subset [d]$, with sizes $|S'_1| = |S'_2| = i$, we say that the i -faces θ_{v,S'_1} and θ_{u,S'_2} are adjacent if the intersection $\theta_{v,S'_1} \cap \theta_{u,S'_2}$ is an $i-1$ -face. In addition, if θ_1 and θ_2 are i and $i-1$ -faces such that $\theta_2 \subset \theta_1$, then we say that θ_2 is a positive edge relative to face θ_1 if they are rooted at the same vertex. Furthermore, we say that θ_1 is a positive cube of θ_2 . When the i -face is clear from the context, we will abbreviate and say that θ_2 is positive.*

Example 4.3. *For example, let $v, u \in \mathcal{G}_n^d$ and $g_1 \neq g_2 \in S$. Consider the 2-face $\theta = \{v, g_1v, g_2g_1v, g_2v\}$. Then, for $x, y \in \mathcal{G}_n^d$, we say that an edge (1-face) $\{x, y\}$ is positive relative to the face θ if $x = v$ and y is either g_1v or g_2v .*

We denote the complex by $G = (V, E, F)$, where V , E , and F are the 0-, 1-, and 2-faces. Below, we formalize binary assignments on the toric complex and introduce boundary operators. These boundary operators are the same as standard boundary operators in the chain complex formalism.

Definition 4.4 (Assignments on a toric complex). *Let $\mathcal{T}$ be a d -dimensional toric complex. For each i , the set of i -faces induces a vector space $\mathbb{F}_2^{|\mathcal{F}_i|}$. An element $z \in \mathbb{F}_2^{|\mathcal{F}_i|}$ is called an assignment on the i -faces of $\mathcal{T}$.*

For a vertex v and a subset of generators $S' \subset S$ of size $|S'| = i$, let $\theta_{\mathbf{v}, S'}$ denote the standard basis vector corresponding to the i -face $\theta_{v, S'}$. Thus the family

$$\{\theta_{\mathbf{v}, S'} : v \in \mathcal{F}_0, S' \subset S, |S'| = i\}$$

is the standard coordinate basis of $\mathbb{F}_2^{|\mathcal{F}_i|}$.

Let z be an assignment on the i -faces. Let $v_1, \dots, v_k \in \mathcal{F}_0$ and $S'_1, \dots, S'_k \subset S$ be such that $\theta_{v_1, S'_1}, \theta_{v_2, S'_2}, \dots, \theta_{v_k, S'_k}$ are precisely the i -faces in the support of z , namely:

$$z = \sum_{i=1}^k \theta_{\mathbf{v}_i, S'_i}$$

We refer to $\theta_{v_1, S'_1}, \theta_{v_2, S'_2}, \dots, \theta_{v_k, S'_k}$ as the collection of faces that form z , or briefly, the faces form z . For $i > 0$, we define the linear map $\partial : \mathbb{F}_2^{|\mathcal{F}_i|} \rightarrow \mathbb{F}_2^{|\mathcal{F}_{i-1}|}$ such that

$$\partial \theta_{\mathbf{v}, S'} := \sum_{g \in S'} \theta_{\mathbf{v}, S'/g} + \sum_{g \in S'} \theta_{\mathbf{g}\mathbf{v}, S'/g}$$

For $i < d$, we define the linear map $\partial^ : \mathbb{F}_2^{|\mathcal{F}_i|} \rightarrow \mathbb{F}_2^{|\mathcal{F}_{i+1}|}$ such that*

$$\partial^* \theta_{\mathbf{v}, S'} := \sum_{g \in S/S'} \theta_{\mathbf{v}, S' \cup g} + \sum_{g \in S'} \theta_{\mathbf{g}^{-1}\mathbf{v}, S' \cup g}$$

We define the restriction of z to a vertex v to be the linear map $|_v : \mathbb{F}_2^{|\mathcal{F}_i|} \rightarrow \mathbb{F}_2^{|\mathcal{F}_i|}$ such that:

$$\theta_{\mathbf{u}, S'}|_v := \begin{cases} \theta_{\mathbf{u}, S'} & v \in \theta_{\mathbf{u}, S'} \\ 0 & \text{else} \end{cases}$$

We define the front of z to a vertex v to be the linear map $|_{v+} : \mathbb{F}_2^{|\mathcal{F}_i|} \rightarrow \mathbb{F}_2^{|\mathcal{F}_i|}$ such that:

$$\theta_{\mathbf{u}, S'}|_{v+} := \begin{cases} \theta_{\mathbf{u}, S'} & u = v \\ 0 & \text{else} \end{cases}$$

We say that an assignment z is connected if the graph that is induced by taking the faces that support z as vertices and defining the edges to be the adjacency relation between them is connected. We define the connected components of z to be its subsets such that each of them is a connected assignment. For a j -face f , we say that $f \in z$ if z has a support on $\theta_{\mathbf{v}, S'}$ such that $f \in \theta_{v, S'}$. Consider an assignment $z \in \mathbb{F}_2^{|\mathcal{F}_i|}$ and a vertex $v \in \mathcal{G}_n^d$. We say that z is a rooted assignment at vertex v if there are subsets $S'_1, \dots, S'_k \subset S^{\times k}$ such that each of S'_j is at size i and z is decomposed as:

$$z = \sum_j^k \theta_{\mathbf{v}, S'_j}$$

Put differently, z is the result of taking a collection of i -faces that are rooted at v .

We are ready to give a formal definition of the (d, d') -Toric code.

Definition 4.5 ((d, d') Toric Code). *Let d, d' be integers such that $1 < d' < d$. For any integer n , we construct the n th instance of the quantum code family (d, d') Toric Code as follows: Let $G = (\mathcal{F}_0, \dots, \mathcal{F}_d)$ be the d -dimensional toric complex obtained from $\mathcal{G}_n^d$ as defined in Definition 4.1. We associate the (q) bits with the $\mathcal{F}_{d'}$, the X -checks with the $\mathcal{F}_{d'-1}$, and the Z -checks with the $\mathcal{F}_{d'+1}$. An X -check, c_x , is satisfied if the parity of (q) bits set on the d' -face containing c_x is even. Similarly, a Z -check, c_z , is satisfied if the parity, in the Hadamard basis, of the (q) bits set on the d' -faces contained in c_z is even.*

We argue that the (d, d') -Toric code is indeed a CSS, and prove that by showing that boundary operators $\partial^\pm : \mathbb{F}_2^{|\mathcal{F}_i|} \rightarrow \mathbb{F}_2^{|\mathcal{F}_{i\pm 1}|}$ induce a chain complex.

Claim 4.6 (CSS structure). *The (d, d') -Toric code is a CSS code.*

Proof. We will show that $\mathbb{F}_2^{|\mathcal{F}_i|} \xrightarrow{\partial^\pm} \mathbb{F}_2^{|\mathcal{F}_{i\pm 1}|}$ are a⁸ chain complex and its dual chain. First, we show that $(\partial^+)^T = \partial^-$. Consider vertices $v, u \in V$ and two generator subsets $A, B \subset S$. Consider the basis elements defined by them: $\theta_{v,A}$ and $\theta_{u,B}$. Suppose that:

$$(\partial^+ \theta_{v,A}) \theta_{u,B} = 1$$

Means that there exists $g \notin A$ such that $A \cup \{g\} = B$ and either $v = u$ or $g^{-1}v = u$. But that is equivalent to saying that there is $g \in B$ such that $B \setminus \{g\} = A$ and either $v = u$ or $v = g^{-1}u$. In other words:

$$\begin{aligned} (\partial^+ \theta_{v,A}) \theta_{u,B} &= \theta_{v,A} (\partial^- \theta_{u,B}) \\ \Rightarrow (\partial^+)^T &= \partial^- \end{aligned}$$

So it's left to show that the boundary of a boundary is zero. Consider a basis vector $\theta_{v,S'}$:

$$\begin{aligned} \partial^- (\partial^- \theta_{v,S'}) &= \partial^- \left(\sum_{g \in S'} \theta_{v,S'/g} + \sum_{g \in S'} \theta_{gv,S'/g} \right) \\ &= \sum_{\substack{g', g \in S' \\ g' \neq g}} (\theta_{v,S' \setminus \{g, g'\}} + \theta_{g'v, S' \setminus \{g, g'\}} + \theta_{gv, S' \setminus \{g, g'\}} + \theta_{g'gv, S' \setminus \{g, g'\}}) \\ &= 0 \end{aligned}$$

Where we used that the term in the closure is invariant to sweeping between g and g' , and therefore any term in the sum appears twice. Similarly:

$$\begin{aligned} \partial^+ (\partial^+ \theta_{v,S'}) &= \partial^+ \left(\sum_{g \in S/S'} \theta_{v, S' \cup g} + \sum_{g \in S/S'} \theta_{g^{-1}v, S' \cup g} \right) \\ &= \sum_{\substack{g', g \in S/S' \\ g' \neq g}} (\theta_{v, S' \cup \{g, g'\}} + \theta_{g'^{-1}v, S' \cup \{g, g'\}} + \theta_{g^{-1}v, S' \cup \{g, g'\}} + \theta_{g'^{-1}g^{-1}v, S' \cup \{g, g'\}}) \\ &= 0 \end{aligned}$$

⁸Check if that is grammatical correct.

The conditions $(\partial^\pm)^{\circ 2} = 0$ imply that $\mathbb{F}_2^{|\mathcal{F}_i|} \xrightarrow{\partial^\pm} \mathbb{F}_2^{|\mathcal{F}_{i\pm 1}|}$ are chain complexes. Therefore, for any d' , every $d' - 1$ face and every $d' + 1$ face overlap on an even number of d' -faces, so the corresponding X - and Z -checks commute. This is exactly the CSS condition. $\square$

The $\theta_{\mathbf{u}, S'}$ notation gives a nice interpretation of the logical code words. Particularly, an X -type logical codeword is given as follows: Take a subset of generators $S' \subset S$ and consider the subgroup generated by it, $\langle S' \rangle$; let us denote the subgroup by K . Then the logical codeword is the X -product over all $\theta_{\mathbf{u}, S'}$ such that $\mathbf{u} \in K$. Moreover, replacing K by a coset whose base subgroup is $\langle S' \rangle$, we get another presentation of the same logical codeword.

Claim 4.7. *For any generator subset S' of size d' , denote by $x_{S'}$ and $z_{S'}$ the following assignments:*

$$x_{S'} := \sum_{\mathbf{u} \in \langle S' \rangle} \theta_{\mathbf{u}, S'} \quad z_{S'} := \sum_{\mathbf{u} \in \langle S/S' \rangle} \theta_{\mathbf{u}, S'}$$

Then $x_{S'}$ and $z_{S'}$ are binary representations of anticommuting logical codewords of the (d, d') -Toric code. In particular, $x_{S'} \in \mathcal{C}_X / \mathcal{C}_X^\perp$, $z_{S'} \in \mathcal{C}_Z / \mathcal{C}_Z^\perp$, and $x_{S'} z_{S'} = 1$. In addition, let $X_{S'}$ and $Z_{S'}$ be the Pauli matrices obtained by multiplying each non-trivial coordinate of them by X or Z , respectively.

Proof. By definition, $x_{S'}$ and $z_{S'}$ are binary assignments on the d' -dimensional faces. We start by showing that $x_{S'} \in \mathcal{C}_X$ and $z_{S'} \in \mathcal{C}_Z$; namely, that the boundaries $\partial^- x_{S'}$ and $\partial^+ z_{S'}$ vanish.

$$\begin{aligned} \partial^- \sum_{\mathbf{u} \in \langle S' \rangle} \theta_{\mathbf{u}, S'} &= \sum_{\mathbf{u} \in \langle S' \rangle} \sum_{g \in S'} \theta_{\mathbf{u}, S'/g} + \theta_{\mathbf{g}\mathbf{u}, S'/g} \\ &= \sum_{g \in S'} \sum_{\mathbf{u} \in \langle S' \rangle} \theta_{\mathbf{u}, S'/g} + \sum_{\mathbf{u} \in \langle S' \rangle} \theta_{\mathbf{g}\mathbf{u}, S'/g} \\ &= \sum_{g \in S'} \sum_{\mathbf{u} \in \langle S' \rangle} \theta_{\mathbf{u}, S'/g} + \sum_{\mathbf{u} \in \langle S' \rangle} \theta_{\mathbf{u}, S'/g} = 0 \\ \partial^+ \sum_{\mathbf{u} \in \langle S' \rangle} \theta_{\mathbf{u}, S/S'} &= \sum_{\mathbf{u} \in \langle S' \rangle} \sum_{g \in S'} \theta_{\mathbf{u}, S/S' \cup g} + \theta_{\mathbf{g}^{-1}\mathbf{u}, S/S' \cup g} \\ &= \sum_{g \in S'} \sum_{\mathbf{u} \in \langle S' \rangle} \theta_{\mathbf{u}, S/S' \cup g} + \sum_{\mathbf{u} \in \langle S' \rangle} \theta_{\mathbf{g}^{-1}\mathbf{u}, S/S' \cup g} \\ &= \sum_{g \in S'} \sum_{\mathbf{u} \in \langle S' \rangle} \theta_{\mathbf{u}, S/S' \cup g} + \sum_{\mathbf{u} \in \langle S' \rangle} \theta_{\mathbf{u}, S/S' \cup g} = 0 \end{aligned}$$

Thus $x_{S'} \in \mathcal{C}_X$ and $z_{S'} \in \mathcal{C}_Z$, now for showing that they are not binary representations of stabilizers, it's enough to show they admit no vanishing product.

$$x_{S'} z_{S'} = \sum_{\mathbf{u} \in \langle S' \rangle} \theta_{\mathbf{u}, S'} \sum_{\mathbf{u} \in \langle S/S' \rangle} \theta_{\mathbf{u}, S'} = \theta_{\mathbf{e}, S'} \theta_{\mathbf{e}, S'} = 1$$

Therefore, the Paulis $X_{S'}$ and $Z_{S'}$ are X - and Z -type anticommuting logical code words. $\square$



Figure 6: In blue and green are the binary presentations x_S and z_S of logical code words of the $(2, 2)$ -Toric code. The edge $\{(5, 4), (0, 4)\}$ is omitted for clarity (it should be a green edge that hovers over the green line).

The (d, d') -Toric code admits a fold-transversal Hadamard-type realization for any dimensions d, d' . Particularly, the gate exchanges logical Pauli $X_{S'}$ with logical Pauli $Z_{S/S'}$ and vice versa. It is realized by an application of transversal Hadamards followed by permuting the qubits according to a transformation which we name the ϕ -transpose map. We define the transpose map as follows:

Definition 4.8 (The Transpose Map). *Let d, d' be integers, such that $d' < d$, and let $\mathcal{G}$ be a (d, d') -Toric complex generated by the generator set S . We define the transpose map $\phi : \mathcal{F}^{d'} \rightarrow \mathcal{F}^{d-d'}$ by acting on the basis elements as follows:*

$$\phi(\theta_{\mathbf{v}, \mathbf{S}'}) \rightarrow \theta_{\mathbf{v}-\mathbf{1}, \mathbf{S}/\mathbf{S}'}$$

Notice that the transpose map is a permutation over the bits and therefore can be realized at constant depth.

Definition 4.9 (Hadamard-Type Gate $\mathbf{H}_\phi$). *We define the logical Hadamard-type gate $\mathbf{H}_\phi : L(\mathcal{H}) \rightarrow L(\mathcal{H})$ as follows, for any $S' \subset S$, of size d' :*

$$\mathbf{H}_\phi(X_{S'}) \rightarrow Z_{S/S'} \quad \mathbf{H}_\phi(Z_{S'}) \rightarrow X_{S/S'}$$

Claim 4.10. *The Hadamard-type gate $\mathbf{H}_\phi$ has a fold-transversal implementation by, first applying $\mathbf{H}^n$ over all the physical qubits, and then permuting them by the transpose map ϕ , namely $\hat{\mathbf{H}}_\phi = \phi \circ \mathbf{H}^n$.*

Proof. Clearly, for any subset of generators $S' \subset S$, the map ϕ sends $x_{S'}$ to $z_{S/S'}$:

$$\phi(x_{S'}) = \phi\left(\sum_{u \in \langle S' \rangle} \theta_{u, S'}\right) = \sum_{u \in \langle S' \rangle} \theta_{u, S/S'} = z_{S/S'}$$

Moreover, ϕ sends binary representations of Z -stabilizers to binary representations of X -stabilizers, and vice versa. To see this, consider a $d' + 1$ dimensional face, and consider the image of the stabilizer it defines.

$$\begin{aligned} \phi(\partial^- \theta_{v, S'}) &= \phi\left(\sum_{g \in S'} \theta_{v, S'/g} + \theta_{gv, S'/g}\right) \\ &= \sum_{g \in S'} \theta_{v^{-1}, (S/S') \cup g} + \theta_{g^{-1}v^{-1}, (S/S') \cup g} \\ &= \partial^+ \theta_{v^{-1}, S/S'} \end{aligned}$$

And since $\phi \circ \phi = I$, we also have that $\phi(\partial^+ \theta_{v^{-1}, S/S'})$. Thus, $\mathbf{H}^{\otimes n} \circ \phi$ sends Z -stabilizers to X -stabilizers and vice versa, thereby preserving the code space. Combining this with the above, $\mathbf{H}^{\otimes n} \circ \phi$ realizes the logical form $\mathbf{H}_\phi$ in the $(d, d/2)$ -Toric code. $\square$

Example 4.11. *Consider the $(2, 1)$ -Toric code. Name the generators in S by $g_1 = (1, 0)$ and $g_2 = (0, 1)$. Consider the check corresponding to the Z -stabilizer $\partial^+ \theta_{(2, 3), \{\}}.$*

$$\partial^+ \theta_{(2, 3), \{\}} = \{\theta_{(2, 3), \{(1, 0)\}}, \theta_{(2, 3), \{(0, 1)\}}, \theta_{(1, 3), \{(1, 0)\}}, \theta_{(2, 2), \{(0, 1)\}}\}$$

The ϕ -transpose map acts on it as follows:

$$\begin{aligned} \phi(\theta_{(2, 3), \{(1, 0)\}}) &= \theta_{(4, 3), \{(0, 1)\}} \\ \phi(\theta_{(2, 3), \{(0, 1)\}}) &= \theta_{(4, 3), \{(1, 0)\}} \\ \phi(\theta_{(1, 3), \{(1, 0)\}}) &= \theta_{(5, 3), \{(0, 1)\}} \\ \phi(\theta_{(2, 2), \{(0, 1)\}}) &= \theta_{(4, 4), \{(1, 0)\}} \end{aligned}$$

What gives edges on a rectangle that converges with the bits on the support of the X -stabilizer $\theta_{(4, 3), \{(0, 1), (1, 0)\}}.$

Remark 4.12. *If $d' = d/2$, then for a fixed $S' \subset S$, a realization of a Hadamard-type gate that exchanges $X_{S'} \leftrightarrow Z_{S'}$ can be obtained by concatenating to $\mathbf{H}_\phi$ a second permutation over the qubits (which is induced by a permutation over the generating set S). For our purposes, using $\mathbf{H}_\phi$ is sufficient.*

Equipped with $\mathbf{H}_\phi$, we gain a mapping into the dual code that both keeps the Toric skeleton and sends the Z -checks to $d' - 1$ dimensional faces. Later (in Section 4.4.1),



Figure 7: An outline of Example 4.11. The ϕ -transpose sends Z -stabilizers to X -stabilizers and vice versa. Here we consider the $(2, 1)$ Toric code; namely, the Z -stabilizers (parity checks) are set on the vertices, and the X -stabilizers (phase checks) are set on the squares. In blue and green, we have the edges corresponding to checks $\partial^+ \theta_{(2,3),\{\cdot\}}$ and $\theta_{(4,3),\{(0,1),(1,0)\}}$. The example shows that indeed ϕ maps between the two.

this will be advantageous, enabling the use of the same local decoder (with a slight change) to act on both $\mathcal{C}_X$ and its dual.

Throughout the construction, we encode each qubit in a code block; namely, we restrict ourselves to using only a single codeword of the (d, d') -Toric code. From now on, we will think of $S' \subset S$ as a fixed d' -size subset of generators that were picked from the head. And we will decode the logical $|1\rangle$ state by $x_{S'}$. To simplify terminology, we call such usage 'nice encoding'.

Definition 4.13 (The Nice Encoding). *Consider (d, d') -Toric code and fix a subset $S' \subset S$, at size d' . We define the nice encoding to be the subsystem code, which uses $x_{S'}$ to encode a single logical qubit. Namely, the logical codewords are:*

$$\begin{aligned} |\bar{0}\rangle &= |\mathcal{C}_Z^\perp\rangle \\ |\bar{1}\rangle &= |x_{S'} + \mathcal{C}_Z^\perp\rangle \end{aligned}$$

We are now ready to give a full description of the realization of the logical operators. The logical Paulis X and Z are realized by the application of $X_{S'}$ and $Z_{S'}$. For the gates $\mathbf{S}$ and $\mathbf{T}$, we use single-qubit gate teleportation as presented in Figure 8. In single-qubit gate teleportation, the resource $|\mathbf{S}\rangle$ is initialized as $\mathbf{S}|+\rangle$, similarly for $|\mathbf{T}\rangle$. The gate $\mathbf{1}_{S'}$ stands for the classical procedure that, given the measurement results, checks whether the logical bit $|1\rangle$ is turned on. We will return to this gate with a higher resolution depiction of it after the description of the Hadamard teleportation.



Figure 8: Simulating the $\mathbf{S}$ and $\mathbf{T}$ gates, using the $|\mathbf{S}\rangle$, $|\mathbf{T}\rangle$ magic states.

For the Hadamard gate, we use a modification of the standard two-qubit gate teleportation as shown in Figure 9. The realization differs from the standard realization as the measurement in the phase basis is done by first moving to the dual space via the application of physical $\mathbf{H}^{\otimes n}$, measuring, reorganizing the qubits through ϕ , and then checking for the $x_{S/S'}$ outcome (thus, it can be thought of as the application of $\mathbf{H}_\phi$ and then measurement). In addition, unlike the single-qubit gate teleportation, here $|\mathbf{H}\rangle$ is initialized as $I \otimes H(|00\rangle + |11\rangle)$ (up to normalization).



Figure 9: Gate teleportation of the Hadamard gate, using the resource $|\mathbf{H}\rangle$. We emphasize again that although the Hadamard (at least Hadamard-type) has an $O(1)$ -depth gate realization, we still need all the operations on the quantum register to be strictly transversal for the sake of the proof. Thus, we realize the Hadamard via gate teleportation.

The classical procedure of logical codeword checking, namely $\mathbf{1}_{x_{S'}} / \mathbf{1}_{x_{S/S'}}$, is divided into four stages. In the first, we find all the deviation clusters of the Tanner graph. This requires finding connected components, which is known to be done in $\log^2(n)$ depth (See [SV82] and [AS87] for logarithmic time connected components in the PRAM model and [Lev80] for the simulation of PRAM machines using bounded-fan circuits). Then, we broadcast the connected components. Conceptually, each deviation cluster is sent to its own register; technically, we copy the restriction of

the syndromes to the deviation cluster. The broadcast also requires at most $\log(n)$ depth since there can be at most $\#\text{checks} = \Theta(n)$ deviation clusters and therefore at most $\Theta(n)$ copies. Then, we decode each of the copies in parallel. The separation of the clusters guarantees that during decoding, clusters will not merge into a larger cluster. For the decoding, we iterate sweep-rule cycles (defined in Definition 4.26) until erasing the syndrome. Later, we will see that the decoding depth is bounded by $\Theta(n)$ (See Corollary 4.41). Finally, we combine the results of all the decoding sessions together by Xoring their correction suggestions. Each of the Xor operations is between at most n qubits and therefore requires at most $\log(n)$ depth. So the overall depth for performing a logical measurement is $\Theta(n)$. The additional classical width consists of restricting the syndrome for each of the deviation clusters. We will store them as check assignments $\mathbb{F}_2^{\mathcal{F}_{d'}-1}$, namely an addition of $\Theta(n)$ bits per deviation cluster and in total $\Theta(n^2)$.

4.1 Infinite Sides Torics

The Toric skeleton provides a notation for positive directions. However, to demonstrate that the deviation cluster size shrinks as a result of the decoding cycle, or at least does not expand too much during the operation of the logical operator, we will need a way to mark extremities (for example, the northernmost point). To achieve this, we will analyze the view over the finite Toric as a projection derived from a view over the infinite plane. For this purpose, given our initial (subjected) quantum circuit, we construct a representation in which any gate that originally acted on a quantum register, namely on the finite Toric, is mapped to an infinite set of quantum gates that act on the infinite number of qubits set on the infinite plane. This set respects a periodic condition such that any restriction of it to a finite rectangle at the size of the original Toric yields the original view, and therefore describes well the original running.

The following two definitions simplify decoding gates by replacing them with the Pauli 'correction' they suggest at the end of their cycle. The first defines what a decoding gate is, and the second defines the replacement.

Definition 4.14 (Decoding Gadget). *Let C be a quantum circuit, and let u be a gate in C with fan-in at most Δ . We say that u is a decoding gadget if it has the following form:*

1. *The wires of u can be decomposed into two subsets A and B such that the wires in A are reset wires, and there exists an encoded register R in C such that the wires in B are all set in R .*
2. *For a set of faults $\mathcal{E}$, there is a Pauli gate u' that acts only on B such that the action of u in $C[\mathcal{E}]$ equals u' . Put differently, the quantum circuit obtained from $C[\mathcal{E}]$ by replacing u with u' equals $C[\mathcal{E}]$.*

Definition 4.15 (Unitary Decoding Representation). *Let $C = \{u_i\}_{i=1}$ be a quantum circuit, and let $\mathcal{U} = \{u'_i\}_{i=1} \subset C$ be a subset of the gates in C that are decoding gadgets.*

For a set of faults $\mathcal{E}$, we define the unitary decoding representation of $C[\mathcal{E}]$ to be the quantum circuit obtained by replacing each of the gates in $\mathcal{U}$ with their corresponding Pauli gate according to $\mathcal{E}$.

Until the end of the subsection, we will assume that the subjected circuits are given in their unitary decoding representation.

The following definition extends the notion of location to differentiate between a coordinate that encodes a register and a coordinate of the location inside the register. This will be useful, as in the infinite plane representation, each register will be mapped to a unique infinite register. From a technical perspective, the map will stretch only inner locations to infinity.

Definition 4.16 (Toric register coordinate system). *Let $C = \{u_i\}_i$ be a quantum circuit of width w with registers $\mathcal{R} = R_1, \dots, R_k$, each encoded in the (d, d') -Toric code, induced by a d -dimensional Toric with a side length n . We define the 'Toric register coordinate system' to be the coordinate system that encodes spatial location with two coordinates as follows: In the first coordinate, the register number is recorded, and in the second, the qubit is encoded by the face associated with it. We call the first the register coordinate and the second the inner coordinate.*

We will need two types of gate translation. The first is the standard spatial translation, which means that any of the inner coordinates of the gate are advanced by the same vector. The second is 'rooted translation.' Here, we assume that the locations of the gate are given in a relative-to-root fashion, meaning that there is a root vertex, and all the locations are given by positive paths from the root. Then, we first translate the root and then advance the inner coordinate of any of the other locations to be on the path from the translated root. When considering a product gate, the translations act the same.

Definition 4.17 (Translation by vertex v). *Let u be a gate, and denote the inner coordinates of u 's locations by $\theta_{v_1, J_1}, \dots, \theta_{v_\Delta, J_\Delta}$. The translation of u by vertex v is defined to be the gate obtained from u by replacing the inner coordinates of its locations by: $\theta_{vv_1, J_1}, \dots, \theta_{vv_\Delta, J_\Delta}$. Furthermore, for a d -dimensional Toric complex with side length n , we define a translation by an integer vector $a \in \mathbb{Z}^d$ as a translation by the vertex:*

$$(g_1^n)^{a_1}, (g_2^n)^{a_2}, \dots, (g_d^n)^{a_d}$$

We denote the translation by an integer vector a as $a * u$.

Definition 4.18 (Rooted gate at vertex v). *Let u be a quantum gate that acts on a register whose qubits are associated with a Toric complex at dimension d . We say that a gate u is rooted at vertex v if the inner coordinate of its location can be encoded by lists of pairs $(I_1, J_1), (I_2, J_2), \dots, (I_l, J_l)$ such that $I_i \in S^d, J_i \subset S$, and the i th qubit on which it acts is in the face $\theta_{(\prod_{g \in I_i} g)v, J_i}$.*

Definition 4.19 (Root translation in Register System). *For a gate u that is a rooted gate at vertex v , we define its translation by a vertex v' to be the gate obtained by copying u and replacing its inner location encoding the root by $v'v$.*

Similarly to the standard translation by an integer vector, we extend the action to a root translation by an integer vector $a \in \mathbb{Z}^d$, as the root translation by the vertex:

$$(g_1^n)^{a_1}, (g_2^n)^{a_2}, \dots, (g_d^n)^{a_d}$$

*We denote the root translation by an integer vector a as $a *_r u$.*

Now we are ready to give an explicit description of the mapping. Given a quantum circuit C , we name its presentation over the infinite plane the ∞ -extension of C .

Definition 4.20. *Let n be an integer, and let $C = \{u_i\}_{i=1}$ be a quantum circuit with width w and quantum registers $\mathcal{R} = R_1, \dots, R_k$ such that each register R_i encodes a (d, d') -Toric code with side at length n . We define the ∞ -extension of C , denoted by C^∞ , as follows:*

1. *Each register R_i in $\mathcal{R}$ is mapped to a register R'_i , which its qubits are associated with the d' -dimensional faces of the infinite d -dimensional Toric.*
2. *Each gate $u_i \in C$ is mapped to the infinite collection of gates, where the gates are obtained by either the standard translation or the rooted translation by an integer vector, depending on whether u_i is a decoding gate as follows:*

$$u_i \mapsto \begin{cases} \bigcup_{a \in \mathbb{Z}^d} \{a *_r v\} & \text{if } u_i \text{ is not a decoding gadget} \\ \bigcup_{a \in \mathbb{Z}^d} \{a_r *_r v\} & \text{else} \end{cases}$$

The restriction of the qubits in C^∞ 's registers to finite Torices with side length n will be denoted by $\cdot|_{[w]}$, and it includes all the faces whose root vertex has 'powers' (the powers of $g_1, \dots, g_d$) in the range $[0, n-1]$. For a set of faults $\mathcal{E} = \{f_i\}_{i=1}$, we denote by $C^\infty[\mathcal{E}]$ the ∞ -extension of $C[\mathcal{E}]$.

Obviously, translation by the zero vector (or by the unital vertex) keeps the gate in its place, or in other words, in the restriction of C^∞ to $[w]$ - the original Toric - any non-decoding gate is restored. The following claim states that this is also true for a decoding gate.

Claim 4.21. *Let $u = (g, l)$ be a decoding gate. The restriction of the image of u in C^∞ to $[w]$ is u (when applied to the infinite domain). Namely:*

$$\left(\prod_{a \in \mathbb{Z}^d} a *_r u \right) |_{[w]} = u$$

Proof. Denote by $\hat{a}^j \in \mathbb{F}_2^d$ the vector which assigns in its i th coordinate 1 if the root that supports the face of u_j has a g_i power greater than n . We argue that the j th

face of $-\hat{a}^j *_r u$ is in $[w]$ and that the j th face of any other root translation of u is not in $[w]$. That implies that the restriction

$$\left(\prod_{a \in \mathbb{Z}^d} a *_r u\right)|_{[w]}$$

contains each of u 's faces exactly once, and combined with the fact that u is a Pauli product, it follows that they are the same operator.

By the definition of root translation, we get that $(\hat{a}^j *_r u)_j$ is in $[w]$. Let $a' \in \mathbb{Z}^d$ be such that $a' \neq \hat{a}^j$, and denote by i the coordinate on which they differ. Then, we get that the g_i power of the root of $(a' *_r u)_j$ is at a distance n from the g_i power of the root of $(\hat{a}^j *_r u)_j$; therefore, their roots can't both be in the range $[0, n-1]$, which implies that $(a' *_r u)_j \notin [w]$. □

Next, we would like to prove that the infinite presentation of a circuit agrees with it in terms of action. We will prove that when in the zero state, the action of the circuit on them is 'equivalent' in the sense that the restriction of C^∞ 's output to $[w]$ yields the same mixed state as the output of C . The following provides a formal definition of that 'equivalence', which we name ∞ -compatible.

Definition 4.22. For a quantum circuit C , of the form defined in Definition 4.20, and a set of faults $\mathcal{E}$, denote:

$$\begin{aligned}\rho &:= C[\mathcal{E}] |0\rangle\langle 0| C[\mathcal{E}]^\dagger \\ \rho_\infty &:= C^\infty[\mathcal{E}^\infty] |0\rangle\langle 0| (C^\infty[\mathcal{E}^\infty])^\dagger\end{aligned}$$

We say that a quantum circuit C is ∞ -compatible if, for any set of faults $\mathcal{E} = \{f_i\}_{i=1}$, it holds that:

$$\rho = \mathbf{Tr}_{/[w]}(\rho_\infty)$$

Claim 4.23. Let C be quantum circuit of the form defined in Definition 4.20. Then C is ∞ -compatible.

Proof. We prove it by induction on the number of the gates.

1. Base. For an empty circuit, ρ_∞ is just the infinite string of zeros; thus, the restriction of it to $[w]$ locations, namely $\mathbf{Tr}_{/[w]}(\rho_\infty)$, yields a single matrix element corresponds to the w -length string of zeros, which is exactly ρ .
2. Step. Assume the claim holds for every circuit with fewer than τ gates, and consider a circuit C with τ gates. Denote by C' the circuit obtained from C by erasing its last gate; then C' has $\tau - 1$ gates. Denote by ρ' and ρ'_∞ the output states of $C'[\mathcal{E}]$ and $(C')^\infty[\mathcal{E}^\infty]$, respectively. By the induction assumption we have:

$$\rho' = \mathbf{Tr}_{/[w]}(\rho'_\infty)$$

Denote the last gate in C by u , and by $I = \{v_j\}$ the collection of gates in C^∞ to which u is mapped (notice that the collection is countable). Let us split into cases, depending on whether u is or is not a decoding gate.

- (a) Suppose that u is not a decoding gate. Let g and l be the gate element and the location of u . By the definition of ∞ -extension, I equals:

$$I = \bigcup_{a \in \mathbb{Z}^d} \{a * u\}$$

So, there is only one gate in I , corresponding to a being the zero vector, that has support on qubits with spatial locations in the range $[w]$. Denote that gate by v_0 .

$$\begin{aligned} \text{Tr}_{/[w]}(\rho_\infty) &= \text{Tr}_{/[w]} \left(\prod_{j=-\infty}^{\infty} v_j \rho'_\infty \prod_{j=-\infty}^{\infty} v_j^\dagger \right) \\ &= v_0 \text{Tr}_{/[w]} \left(\prod_{j=-\infty, j \neq 0}^{\infty} v_j \rho'_\infty \prod_{j=-\infty, j \neq 0}^{\infty} v_j^\dagger \right) v_0^\dagger \\ &= v_0 \text{Tr}_{/[w]} \left(\prod_{j=-\infty, j \neq 0}^{\infty} v_j^\dagger \prod_{j=-\infty, j \neq 0}^{\infty} v_j \rho'_\infty \right) v_0^\dagger \\ &= v_0 \text{Tr}_{/[w]}(\rho'_\infty) v_0^\dagger \\ &= v_0 \rho' v_0^\dagger \\ &= \rho \end{aligned}$$

- (b) In the case that u is a decoding gate, each of the v_j is a Pauli product. Thus, we can decompose it into the product of two terms: the first has support on $[w]$, and the second is its complement. Exactly as in the above case, the second v_j component can be treated as a disjoint gate that has no support on qubits in $[w]$ and therefore does not contribute to the trace. For the rest, we get:

$$\text{Tr}_{/[w]}(\rho_\infty) = \left(\prod_{-\infty}^{\infty} v_j \right) |_{[w]} \rho' \left(\left(\prod_{-\infty}^{\infty} v_j \right) |_{[w]} \right)^\dagger$$

Then, by Claim 4.21, we have that:

$$\left(\prod_{-\infty}^{\infty} v_j \right) |_{[w]} = u$$

therefore:

$$\text{Tr}_{/[w]}(\rho_\infty) = u \rho' u^\dagger = \rho$$

□

The computation DAG lifts naturally to the ∞ -extension: gates that arise as translates of the same local gate act on disjoint translated supports inside different periods, so they may be executed simultaneously. The same periodic lifting applies to the underlying geometric objects, passing from the finite Toric complex to its infinite cover.

We close that section by introducing the *potential walls*. Analyzing the infinite presentation enables us to bound a subset of errors by 'triangles'. A potential wall is a linear function $\mathbb{R}^d \rightarrow \mathbb{R}$. For the subset of vertices (that might be the subset that supports some error), the maximal value of the potential wall on it is achieved at the point of tangency between the subsets and its bounding triangle. The next definition explains how the embedding of vertices of the infinite in $\mathbb{R}^d$ is handled.

Definition 4.24 (The standart embedding). *Consider the d -dimensional infinite grid $\mathcal{G}^d$. The standard embedding $\mathcal{G}^d \rightarrow \mathbb{R}^d$ is given by fixing a vertex in $\mathcal{G}^d$ to be sent to the origin and then mapping the generators of the grid to the standard basis. We extend notation as follows: for a vertex $v \in \mathcal{G}^d$, a generator $g \in S$, and a coordinate $i \in [d]$, such that under the embedding $g \rightarrow i$, we denote by $v_i = v_g$ the i 'th coordinate of the image of v .*

Now we are ready to define the *potential walls*:

Definition 4.25 (Potential Walls). *We define the potential walls to be the $d + 1$ functions $L_1, L_2, \dots, L_{d+1}$, from the vertices of $\mathcal{G}_n^d$ to the reals as follow:*

1. *For any coordinate $i \in [d]$, we define $L_i: \mathcal{G}^d \rightarrow \mathbb{R}$ as $L_i(v) := v_i$. In addition, for the generator $g \in S$ which is mapped to i , i.e $g \rightarrow i$, we identify $L_g = L_i$.*
2. *We define $L_{d+1}: \mathcal{G}^d \rightarrow \mathbb{R}$ by $L_{d+1}(v) := -\sum_{i=1}^d v_i$.*

Consider an assignment z over the faces. For a vertex $v \in z$, and a potential wall $l \in \{L_1, \dots, L_{d+1}\}$, we say that v is a local maximum of l in z if for any neighbor u of v , such that $u \in z$, we have $l(v) \geq l(u)$. When the assignment z is clear from the context, we might omit it and briefly say that v is a local maximum of l . In addition we define the function $\mathbf{Size}(\cdot): \mathcal{G}^d \rightarrow \mathbb{R}$, to act on a subset of points $A \subset \mathcal{G}^d$, as follows:

$$\mathbf{Size}(A) := \sum_{g \in S} \max_{v \in A} L_g(v) + \max_{v \in A} L_{d+1}(v)$$

and extend the function to act over assignment on faces z , by taking its action on the set of vertices that support z .

With the assistance of potential walls, we can discuss the endpoint of a bunch of qubits and the 'size' of the (hyper)triangle that bounds them. Soon, the qubits will be associated with the support of an error, and showing that an action decreases the maximums of the potential walls implies that the error shrinks.

4.2 Sweep Rule

The sweep-rule is the core of our construction. A decoding cycle begins with each of the vertices reading the syndrome on its positive X -checks (checks that are associated with the positive $d' - 1$ dimensional faces of the vertex) and, based on what it sees, applying a Pauli correction. The sweep-rule defines the correction to be over the positive qubits such that it will turn off the positive syndrome. It's not hard to see that for the $(2, 2)$ -Toric complex, the Toom's and the sweep rules are the same. Compared to other correction cycles on topological codes, the main advantage of the sweep is that it is independent of time. The invention of the sweep-rule is attributed to Aleksander Kubica and John Preskill [KP19]. Here we extend their work to cubical complexes.

Definition 4.26 (Sweep Rule). *Let σ be a $d' - 1$ -dimensional chain. The sweep rule decoding procedure at a vertex $v \in \mathcal{G}_n^d$ is defined as follows: Measure the checks rooted at v and look for a rooted chain $\tilde{z}$ at v such that:*

1. *The vertices in $\tilde{z}$ are contained in the vertices of σ , namely: $\mathbf{Str}_{0+}(v, \tilde{z}) \subset \mathbf{Str}_{0+}(v, \sigma)$*
2. *The boundary of $\tilde{z}$ agrees with σ on the positive checks of v , namely: $(\partial^- \tilde{z})|_{v+} = \sigma|_{v+}$.*

If such an assignment exists, flip $\tilde{z}$.

The first property is analogous to the second property of 'causal structure' defined in [KP19]⁹. There, they conjectured that vertices that see a syndrome only on their positive checks can suggest a correction that is 'supported' on the vertices of their unsatisfied checks. For lower dimensions, they mentioned that the property can be easily checked. Here, we provide a proof for any dimensions where the face dimension is greater than one. We use the language of cubical complexes; however, since the 'positive' local view at a vertex is identical for both cubical and simplicial complexes, generalizing the proof for simplicial complexes is a matter of adjusting notations. By that, we affirmatively answer their conjecture. Moreover, the proof will provide us with an algorithm to find $\tilde{z}$, which is significantly better than running all options.

Claim 4.27. *Consider a nontrivial $d' - 1$ -dimensional chain σ . If there is a d' -dimensional chain z that is rooted at v and whose boundary agrees with $\sigma|_{v+}$, then there is a d' -dimensional chain $\tilde{z}$ rooted at v that satisfies the sweep-rule conditions.*

Proof. We prove it by induction on d' .

1. Base. Consider $d' = 2$ and let d be any integer equal to or greater than 3. We argue that the support size of σ has to be even. To see that, denote the faces in z by $\theta_{\mathbf{v}, \mathbf{S}'_i}$, where $\mathbf{S}'_i = \{g_{i,1}, g_{i,2}\}$. Using the fact that σ is the boundary of z , we have:

$$\sigma = \partial^-(z) = \sum_{\theta_{\mathbf{v}, \mathbf{S}'_i} \in z} \sum_{g \in \mathbf{S}'_i} \theta_{\mathbf{v}, \mathbf{S}'_i / g} = \sum_i \theta_{\mathbf{v}, \{\mathbf{g}_{i,1}\}} + \theta_{\mathbf{v}, \{\mathbf{g}_{i,2}\}}$$

⁹In their language: $\diamond(\tilde{z}) = \diamond(\sigma|_v)$.

So the number of terms in the sum is even, and since a term is zeroed out only if it appears an even number of times, we have that after excluding the vanishing terms, σ has an even number of faces. Since the number of faces is even, there is a perfect matching between them. It's easy to see that the 2-chain $\tilde{z}$, whose each face corresponds to the union of the two edges in a match, satisfies the sweep-rule conditions.

2. Assume the correctness of the claim for every face dimension lower than d' . Now for a face dimension d' and an arbitrary generator $g \in S$ such $gv \in \sigma$, divide σ into a sum of two chains, the first contain the faces of σ that supported on the vertex gv where the second contains the compliment. Denote them by $\sigma|_{gv}$ and $\sigma|_{/gv}$, so $\sigma = \sigma|_{gv} + \sigma|_{/gv}$.

Now let σ^* the chain obtained from $\sigma|_{/gv}$ by removing from any face of it the vertex gv , thus σ^* is a $d' - 2$ -dimensional chain. Moreover, let z^* be the $d' - 1$ -dimensional chain complex obtained from $z|_{/gv}$ by excluding gv from its faces. Observes that

$$\partial^- z^* = \sum_{\theta_{\mathbf{v}, \mathbf{S}' \in z|_{/gv}}} \sum_{g' \in S'/g} \theta_{\mathbf{v}, \mathbf{S}'/\{\mathbf{g}, \mathbf{g}'\}} = \sum_{\theta_{\mathbf{v}, \mathbf{S}' \in (\partial^- z)|_{/gv}}} \theta_{\mathbf{v}, \mathbf{S}'/\{\mathbf{g}\}} = \sigma^*$$

Thus, by the induction assumption, there is a $d' - 1$ -dimensional chain $\tilde{z}^*$ such that:

- (a) $\mathbf{Str}_{0+}(v, \tilde{z}^*) \subset \mathbf{Str}_{0+}(v, \sigma^*)$
- (b) $(\partial^- \tilde{z}^*)|_{v+} = \sigma^*|_{v+}$

The first condition implies that $gv \notin \tilde{z}^*$. Hence, we can define the d' -dimensional chain $\tilde{z}$ by adding the vertex gv to each face of $\tilde{z}^*$. From the second it follows that:

$$\begin{aligned} \partial^- \tilde{z} &= \partial^- \left(\sum_{\theta_{\mathbf{v}, \mathbf{S}' \in \tilde{z}^*}} \theta_{\mathbf{v}, \mathbf{S}' \cup \{\mathbf{g}\}} \right) = \sum_{\theta_{\mathbf{v}, \mathbf{S}' \in \tilde{z}^*}} \left(\sum_{g' \in S'/g} \theta_{\mathbf{v}, \mathbf{S}'/\{\mathbf{g}'\} \cup \{\mathbf{g}\}} \right) + \theta_{\mathbf{v}, \mathbf{S}'} \\ &= \sum_{\theta_{\mathbf{v}, \mathbf{S}' \in \partial^- \tilde{z}^*}} \theta_{\mathbf{v}, \mathbf{S}' \cup \{\mathbf{g}\}} + \tilde{z}^* = \sigma|_{/gv} + \tilde{z}^* \end{aligned}$$

Therefore,

$$\begin{aligned} \sigma + \partial^- \tilde{z} &= \sigma|_{gv} + \sigma|_{/gv} + \sigma|_{/gv} + \tilde{z}^* \\ &= \sigma|_{/gv} + \tilde{z}^* \end{aligned}$$

Thus we reduce the question to handle the $d' - 1$ -dimensional chain $\sigma + \partial^- \tilde{z}$. However, since $gv \notin \sigma + \partial^- \tilde{z}$ it follows that:

$$|\mathbf{Str}_{0+}(v, \sigma + \partial^- \tilde{z})| \leq |\mathbf{Str}_{0+}(v, \sigma)| - 1$$

Thus, we can repeat recursively until the resulting $d' - 1$ chain is 'supported' on fewer than d' vertices. In that case, the chain must be trivial.

□

The algorithm given in the proof of Claim 4.27 for finding $\tilde{z}$ is as follows: given σ , pick a $g \in S$ such that $gv \in \sigma$. Then, define σ^* by removing gv from each face of $\sigma|_{gv}$. Let $\tilde{z}^*$ be the result of the recursive call on σ^* , and return ' $\tilde{z}^* \cup \{gv\}$ ' plus the recursive call on $\sigma|_{/gv} + \tilde{z}^*$.

We stop the recursion whenever either $d' = 2$ or $|\mathbf{Str}_{0+}(v, \sigma)| \leq d'$. The work in each recursion step consists of decomposing σ into $\sigma|_{gv}$ and $\sigma|_{/gv}$ and summing $\binom{d}{d'}$ -length vectors. We summarize by introducing the following claim which gives the running time of the algorithm in the RAM computation model.

Claim 4.28. *Let $T(d, d', k) : \cdot \rightarrow \mathbb{R}$ be the upper bound function for the running time of the algorithm given in Claim 4.27, where d is the dimension of the skeleton, d' is the face dimension, and k is the number of vertices supporting the local view, namely $k = |\mathbf{Str}_{0+}(v, \sigma)|$. Then:*

$$T(k, d') = O\left(2^k \binom{d}{d'}\right)$$

Proof. In each recursive call, the vertex support of σ matches a different subset of the vertices in $\mathbf{Str}_{0+}(v, \sigma)$; thus, the number of recursive calls is at most the number of subsets, namely 2^k . The decomposition of σ into $\sigma|_{gv}$ and $\sigma|_{/gv}$, and the summation $\sigma|_{/gv} + \tilde{z}^*$ are linear in the vector's length, namely linear in $\binom{d}{d'}$. So we bound the total work by the number of recursive steps multiplied by the maximal work at each recursive step. That becomes at most:

$$T(d, d', k) \leq O\left(2^k \binom{d}{d'}\right)$$

□

A full decoding cycle is composed of first applying the sweep-rule regarding the X -syndrome, then moving into the dual code (and reordering the qubits) by $\mathbf{H}_\phi$, and then applying the sweep-rule again with the change that the positive direction is flipped to be the negative direction. At the end, we return to the original code by applying $\mathbf{H}_\phi$.

Definition 4.29 (Sweep cycle). *We define the sweep cycle to be the following procedure:*

1. *For each vertex, perform the sweep rule.*
2. *Apply the Hadamard transform $\mathbf{H}^{\otimes n}$ followed by the ϕ -transpose, and flip the positive direction.*
3. *Again, apply the sweep rule at each vertex.*
4. *Reverse stage 2 by reordering the qubits according to the ϕ -transpose, and then apply the Hadamard transform $\mathbf{H}^{\otimes n}$. Flip the positive direction again.*

To have our constructions yield circuits that have ∞ -extensions Definition 4.20, the sweep-rule cycles have to be 'decoding gadgets' Definition 4.14.

Claim 4.30. *The local gates that perform the sweep rule are decoding gadgets.*

Proof. A local sweep update acts on a constant-size neighborhood around a vertex, uses only local syndrome information, and may be implemented with fresh ancillas that are reset after the update. Any dependence on faults inside the gadget can therefore be pushed onto the data wires as an effective Pauli operator supported on the encoded register, while the reset ancillas carry no residual information. This is exactly the form required in the definition of a decoding gadget. $\square$

Next, we prove that the sweep rule shrinks error subsets.

4.3 Reduced Error

Errors can be thought of as assignments on the d' -dimensional faces that have non-vanishing boundaries. In particular, X -errors can be characterized as d' -dimensional chains $z \in \mathbb{F}_2^{\mathcal{F}_{d'}}$ such that $\partial^- z \neq 0$. In this section, we assume dealing with the ∞ -extension of a circuit (see Definition 4.20), so the cubical complex is the d -dimensional grid with infinite side length.

Rewrite. The $(d + 1)$ th potential wall L_{d+1} gets is maximum on the 'corner' vertex of the error, and therefore we would like to have a stronger claim stating that its maximum value decreases. Thereby implying that the corner 'advances' and the error shrinks. For this, we start by showing that any error is equivalent, up to stabilizer addition, to an error whose 'corner' vertex - the maximal point of L_{d+1} - sees a syndrome.

Rewrite. We present two codes that describe local views. They are the *stabilizers code at vertex v* and the *small code at vertex v* . The first corresponds to stabilizers rooted at the vertex, while the second is the collection of local views on its positive faces that satisfy its positive checks. Then, in Claim 4.33, we consider a vertex whose X -checks don't admit a syndrome, meaning that its local view is in its *small code*, and show that there is a codeword of its *stabilizers code* that agrees with its positive faces. Addition by that stabilizer yields a new assignment excluding v .

Rewrite. If v was a unique maximum of L_{d+1} , then the vertices on the yielded assignments have a lower value of L_{d+1} . Therefore, one can assume that for a given error, the vertex that is the maximal point of L_{d+1} sees a syndrome.

Definition 4.31 (Stabilizers Code at Vertex v). *We define stabilizer code at vertex v , denoted by $Stab_v$, as the restriction of the (d, d') -Toric code to the d' -dimensional faces such that their vertices are at a distance of at most one positive step in each direction from v . That is equivalent to the union of all the positive d' -dimensional faces of v with the d -dimensional faces of each of its sibling gv , excluding faces containing g as*

their generator. Namely:

$$\begin{aligned} \text{Stab}_v := \{ & z : \partial^- z = 0 \text{ and} \\ & z \in \text{Span}(\{\theta_{\mathbf{v}, \mathbf{S}'} : S' \subset S, |S'| = d'\} \\ & \bigcup_{g \in S} \{\theta_{\mathbf{g}\mathbf{v}, \mathbf{S}'/\mathbf{g}} : S' \subset S/g, |S'| = d'\})\} \end{aligned}$$

The checks of the code, are of the following three types:

1. Checks that are rooted at v , each identified by choosing $d - 1$ generators from S . Namely:

$$\{\theta_{\mathbf{v}, \mathbf{S}'} : S' \subset S, |S'| = d' - 1\}$$

For each check, the bits that connect to it, are given by restricting the coboundary of the check to bits domain of Stab_v :

$$(\partial^+ \theta_{\mathbf{v}, \mathbf{S}'})|_{\text{Stab}_v} = \left(\sum_{g \in S/S'} \theta_{\mathbf{v}, \mathbf{S}' \cup \mathbf{g}} + \sum_{g \in S'} \theta_{\mathbf{g}^{-1}\mathbf{v}, \mathbf{S}' \cup \mathbf{g}} \right)|_{\text{Stab}_v} = \sum_{g \in S/S'} \theta_{\mathbf{v}, \mathbf{S}' \cup \mathbf{g}}$$

2. Checks are rooted at the positive sibling of v . Consider such a sibling, let's say gv (namely the vertex that is given by stepping in the direction $g \in S$ from v). Notice that any face rooted at gv that contains ggv cannot be contained in faces rooted at v . Thus, the checks of gv correspond to choosing subsets from S/g . Namely:

$$\bigcup_{g \in S} \{\theta_{\mathbf{g}\mathbf{v}, \mathbf{S}'} : S' \subset S/g, |S'| = d' - 1\}$$

And each check is supported by:

$$\begin{aligned} (\partial^+ \theta_{\mathbf{g}\mathbf{v}, \mathbf{S}'})|_{\text{Stab}_v} &= \left(\sum_{g' \in S/S'} \theta_{\mathbf{g}\mathbf{v}, \mathbf{S}' \cup \mathbf{g}'} + \sum_{g' \in S'} \theta_{\mathbf{g}'^{-1}\mathbf{g}\mathbf{v}, \mathbf{S}' \cup \mathbf{g}'} \right)|_{\text{Stab}_v} \\ &= \left(\sum_{\substack{g' \in S/S' \\ g' \neq g}} \theta_{\mathbf{g}\mathbf{v}, \mathbf{S}' \cup \mathbf{g}'} \right) + \theta_{\mathbf{v}, \mathbf{S}' \cup \mathbf{g}} \end{aligned}$$

3. Checks that their roots are at a two-step distance from v , namely vertices of the form $g'gv$ where $g, g' \in S$, and $g \neq g'$. By the arguments presented in the second type, the checks are the $d' - 1$ faces which do not have the generators on the path leading from v to them, i.e., g, g' , in their generator set.

$$\bigcup_{g, g' \text{ in } S'} \{\theta_{\mathbf{g}'\mathbf{g}\mathbf{v}, \mathbf{S}'} : S' \subset S/\{g, g'\}, |S'| = d' - 1\}$$

And for each check, its support is:

$$\begin{aligned} (\partial^+ \theta_{\mathbf{g}'\mathbf{g}\mathbf{v}, \mathbf{S}'})|_{\text{Stab}_v} &= \left(\sum_{g'' \in S/S'} \theta_{\mathbf{g}\mathbf{g}'\mathbf{v}, \mathbf{S}' \cup \mathbf{g}''} + \sum_{g \in S'} \theta_{\mathbf{g}''^{-1}\mathbf{g}\mathbf{g}'\mathbf{v}, \mathbf{S}' \cup \mathbf{g}''} \right)|_{\text{Stab}_v} \\ &= \theta_{\mathbf{g}\mathbf{v}, \mathbf{S}' \cup \mathbf{g}'} + \theta_{\mathbf{g}'\mathbf{v}, \mathbf{S}' \cup \mathbf{g}} \end{aligned}$$

Definition 4.32 (Small Code at Vertex v). For a vertex v , we denote by $\mathcal{C}_v$ the small code at vertex v , which consists of all the d' -dimensional chains rooted at v that satisfy the positive checks of v , namely:

$$\mathcal{C}_v := \left\{ z : (\partial^- z)|_{v+} = 0 \text{ and } z \in \text{Span}(\{\theta_{\mathbf{v}, \mathbf{S}'} : \mathbf{S}' \subset S, |\mathbf{S}'| = d'\}) \right\}$$

The checks of the code are exactly the first type checks of Stab_v .

Claim 4.33. Let v be a vertex. For any $d' \geq 2$, $d \geq d' + 1$, and any $z \in \mathcal{C}_v$, there is a stabilizer $w \in \text{Stab}_v$ such that $z = w|_v$; namely, z and w agree on the faces rooted at v .

Proof. By induction on the dimensions d, d' . We extend the notation of the stabilizer and the small codes at vertex v to $\mathcal{C}_v^J, \text{Stab}_v^J$, where J is a subset of S , to refer to the codes derived from the skeleton generated by J .

1. Base. For the base cases, let d be either three or four, and let d' be two. In the appendix, we show that the generator matrix of $\mathcal{C}_v$ is a submatrix of the generating matrix of Stab_v . See Section 5.1.
2. Assume the correction of the claim for dimensions lower than d, d' . Decompose $z = z|_{gv} + z|_{/gv}$ where $z|_{gv}$ and $z|_{/gv}$ are the chains consisting of all the faces in z that have support on gv and its complement. Define z^* to be the $d' - 1$ -dimensional chain obtained by excluding gv from any face of $z|_{/gv}$. Notice that $\partial^- z^*$ has to be zero since it contains all the faces of $\partial^- z$ that contain gv (up to the removal). In other words, $z^* \in \mathcal{C}_v^{S/g}$. Thus, we can use the induction assumption and infer the existence of $w^* \in \text{Stab}_v^{S'/g}$ such that $z^* = w^*|_v$.

Now, since $w^* \in \text{Stab}_v^{S/g}$, then $\partial^- w = 0$. In particular, w^* can be decomposed into a sum of stabilizers rooted in v . Let $\theta_{\mathbf{v}, \mathbf{S}'_1}, \theta_{\mathbf{v}, \mathbf{S}'_2}, \theta_{\mathbf{v}, \mathbf{S}'_3}, \dots, \theta_{\mathbf{v}, \mathbf{S}'_l}$ be the d' -dimensional chains corresponding to the stabilizers composing w^* . Namely:

$$w^* = \sum_i \partial^- \theta_{\mathbf{v}, \mathbf{S}'_i}$$

Since those stabilizers are basis elements of $\text{Stab}_v^{S/g}$, we have that $g \notin \mathbf{S}'_i$ for any i . Hence, we can define the extended d' -dimensional chain w to be the chain obtained by adding g to each of the stabilizers deriving w^* . Namely:

$$w = \sum_i \partial^- \theta_{\mathbf{v}, \mathbf{S}'_i \cup \mathbf{g}}$$

However, $\theta_{\mathbf{v}, \mathbf{S}'_1 \cup \mathbf{g}}, \dots, \theta_{\mathbf{v}, \mathbf{S}'_l \cup \mathbf{g}}$ are $d' + 1$ -dimensional chains corresponding to the stabilizers, which are basis elements of $\text{Stab}_v^S = \text{Stab}_v$. Therefore, $w \in \text{Stab}_v$. In addition, opening up the boundary operator yields:

$$\begin{aligned} w|_v &= \sum_i (\partial^- \theta_{\mathbf{v}, \mathbf{S}'_i \cup \mathbf{g}})|_v = \sum_i \left(\sum_{g' \in \mathbf{S}'_i / g} \theta_{\mathbf{v}, \mathbf{S}'_i / g' \cup \mathbf{g}} + \theta_{\mathbf{v}, \mathbf{S}'_i} \right) \\ &= \sum_{\theta_{\mathbf{v}, \mathbf{S}'_i} \in w^*|_v} \theta_{\mathbf{v}, \mathbf{S}'_i \cup \mathbf{g}} + \sum_i \theta_{\mathbf{v}, \mathbf{S}'_i} = z|_{gv} + \sum_i \theta_{\mathbf{v}, \mathbf{S}'_i} \end{aligned}$$

Thus, adding $w|_v$ to z results in:

$$z + w|_v = z|_{/gv} + \sum_i \theta_{\mathbf{v}, \mathbf{S}'_i}$$

Which is a d' -dimensional chain, supported on vertices of the form $g'v$ for $g' \in S/g$, namely a d' -dimensional chain on the complex induced by the $d-1$ -dimensional skeleton. In addition, since the boundary operator can only advance the root of a face forward, we have that $\partial^- w|_v = (\partial^- w)|_v$ and therefore $\partial^-(z + w|_v) = \partial^- z + (\partial^- w)|_v$.

To summarize, we have both $z + w|_v \in \mathcal{C}_v^{S/g}$ and $z + w|_v$ sets on a $d-1$ -dimensional skeleton. Therefore, we can repeat in a recursive manner, with each step resulting in a d' -dimensional chain in a skeleton with dimension decreased by one. We stop when $d = d'$; in that case, the resulting chain has to be trivial.

□

Claim 4.34. *Let $x \in \mathbb{F}_2^{\mathcal{F}_{d'}}$ be a d' -dimensional finite chain, consider the stabilizer $z \in \mathcal{C}_Z^\perp$ which adding it to x minimize the size, Namely:*

$$z = \arg \min_{z \in \mathcal{C}_Z^\perp} \mathbf{Size}(x + z)$$

Then:

$$\mathbf{Size}(x + z) = \mathbf{Size}(\partial^-(x + z))$$

Proof. First, observe that since the vertex domain of $\partial^- x$ is contained in the vertex domain of x , it follows that for any x the inequality $\mathbf{Size}(x) \geq \mathbf{Size}(\partial^- x)$ holds. Combining this with the fact that $\partial^- z = 0$ for any $z \in \mathcal{C}_Z^\perp$, we have that $\mathbf{Size}(x + z) \geq \mathbf{Size}(\partial^-(x + z))$. Now suppose that $\mathbf{Size}(x) > \mathbf{Size}(\partial^- x)$; that implies the existence of $v \in x/\partial^- x$ such that v is a local maximum of one of the potential walls in x and $(\partial^- x)|_v = 0$. Let's split into cases, depending on the potential wall type.

1. If v is a maximum of L_{d+1} , then by Claim 4.33, there exists a stabilizer z rooted at v , such that $z|_v = z|_{v+} = x|_{v+} = x|_v$ (where we used that v is a local maximum of L_{d+1} in x to infer $x|_v = x|_{v+}$). Therefore, $(x + z)|_v = 0$, meaning adding z to x excludes v . In addition, since the rooted stabilizer z is supported only on the vertices in $\mathbf{Str}_{0+}(v, x)$, the addition cannot introduce a vertex with a higher L_g value than the maximal L_g value in $\mathbf{Str}_{0+}(v, x)$ into x . So, repeating the process until excluding all the vertices that get the maximal L_{d+1} -value in x , and summing the stabilizers, results in a stabilizer z for which $L_{d+1}(x + z) < L_{d+1}(x)$.
2. If, for some generator $g \in S$, v is a maximum of L_g , then without loss of generality, pick v among the maximum points of L_g with the highest value of $-L_{d+1}$ (maximize $\sum_{g \in S} L_g(v)$). Since stepping through the positive direction can only increase the value of $-L_{d+1}$, it follows that v is also a local max of

$-L_{d+1}$. Now consider the skeleton system where the directions are flipped, and $\theta_{v,S'}$ is set to be the face containing the vertices that are reached by the selection of reverse steps from S' (i.e., $g^{-1}v \in \theta_{v,S'}$). In that system, we can apply Claim 4.33 (in the reversed direction) and get that there is a stabilizer z such that adding it to x excludes v from x . By the same argument as above, no vertex with a higher $L_{g'}$ -value (although L_{d+1} can increase, that's fine) than what it already has in $\mathbf{Str}_{0-}(v, x)$ is introduced into x . Repeating the process excludes all the vertices with the maximal value of L_g in x , so eventually, we end up with $x + z$, such that $L_g(x + z) < L_g(x)$.

We keep excluding vertices until we get the desired equality. □

We close the section by defining the *reduced* form of finite d' -dimensional chain.

Definition 4.35. Let $x \in \mathbb{F}_2^{\mathcal{F}^{d'}}$ be a d' -dimensional finite chain. We will say that x is *reduced* if for any vertex $v \in x$ that is a maximum point of a potential wall $L \in L_1, \dots, L_{d+1}$, v is also in ∂x . In particular, we have that $\mathbf{Size}(x) = \mathbf{Size}(\partial x)$ and that the reduced form of a given chain is obtained by the construction presented in the proof of Claim 4.34.



Figure 10: Consider the $(3, 2)$ -Toric code, with qubits and X -checks set on the 2- and 1-dimensional faces. In blue, we have a 2-dimensional chain where the vertex achieving the minimum of L_{d+1} , denoted by v , does not see any unsatisfied checks in its positive local view. The green faces represent a 2-dimensional chain, which is the **reduced** form of the blue. In particular, any local maximum of L_{d+1} (vertices u and w) sees a non-trivial syndrome in its positive local view. Namely, the maximal points of L_{d+1} in the chain are also the maximal points in the boundary ∂^- .

4.4 Shrinking

Rewrite. We are about to prove that the sweep rule shrinks finite errors. The following claim relates the local maximum points of the potential walls $L_1, \dots, L_d$ in an assignment z to the structure of the syndrome they observe. Later, we will use

that relation to infer that the decoding step of the sweep rule can't spread the error to a vertex¹⁰ with a higher value than the current global maximum of $L_1, \dots, L_d$. In other words, the error does not spread in the positive directions.

Claim 4.36. *Let z and ξ be d' -dimensional chains, such that ξ results from the application of the sweep rule on z . For any generator $g \in S$, the maximal values of L_g in ξ and $\partial^-\xi$ are lower than the maximal values in z and ∂^-z . Namely:*

$$\begin{aligned} \max \{L_g(u) : u \in \xi\} &\leq \max \{L_g(u) : u \in z\} \\ \max \{L_g(u) : u \in \partial^-\xi\} &\leq \max \{L_g(u) : u \in \partial^-z\} \end{aligned}$$

Proof. First, we argue that for a vertex v in ∂^-z , and a generator $g \in S$, such that v is a local maximum of L_g in either z or ∂^-z , it implies that $gv \notin \partial^-z$. The contradiction where v is a maximum of L_g in z follows immediately, since containment in ∂^-z implies containment in z . Where v is a maximum in ∂^-z , suppose through contradiction that $gv \in \partial^-z$, then

$$L_g(gv) = (gv)_g = 1 + v_g = 1 + L_g(v)$$

In contradiction to v being a local maximum of L_g in ∂^-z .

For a vertex $v \in \partial^-z$ denote the correction it applies by $\varphi_v \in \mathbb{F}_2^{\mathcal{F}_{d'}}$, so

$$\xi = \sum_{v \in \partial^-z} \varphi_v + z \quad \text{and} \quad \partial^-\xi = \sum_{v \in \partial^-z} \partial^-\varphi_v + \partial^-z$$

So for any vertex $u \in \xi/z$, there is a vertex $v \in \partial^-z$ such that $u \in \varphi_v$. Moreover, since φ_v is a rooted chain, it follows that $L_g(u) \leq L_g(v) + 1$, where the equality holds when g doesn't belong to the generator's subset of the face φ_v . Similarly for $u \in \partial^-\xi/\partial^-z$.

Now, combining with the sweep rule's first condition, which restricts the correction applied by a vertex to have a vertex support that is contained in the positive syndrome it sees, we have that for any vertex v which is a local maximum of L_g (in either z or ∂^-z), any $u \in \varphi_v$ satisfies $L_g(u) = L_g(v)$. In other words, ξ and $\partial^-\xi$ don't contain vertices of the form gv where v is a local maximum of L_g in z and ∂^-z , and the desired inequalities follow. $\square$

Next, we prove that for a finite X -error, the sweep-cycle advances the corner (the maximum of L_{d+1}) of its reduced form. We assume we are given a finite error in its reduced form, and therefore whose corner sees a syndrome, and show that the sweep-rule application excludes the corner. In other words, the sweep-rule shrinks X -errors.

Claim 4.37. *Let z be a finite reduced d' -dimensional chain, and let ξ be the reduced result of applying the sweep rule to it. Then:*

$$1. \text{ For any generator } g \in S: \max \{L_g(v) : v \in z\} \geq \max \{L_g(v) : v \in \xi\}$$

¹⁰To faces whose vertices have higher values of $L_1, \dots, L_d$ than the current global maximum values.

2. And for $L_{d+1} : \max \{L_{d+1}(v) : v \in z\} > \max \{L_{d+1}(v) : v \in \xi\} + 1$

In particular, $\mathbf{Size}(\xi) \leq \mathbf{Size}(z) - 1$.

Proof. The first inequality was proved in Claim 4.36, so it's left to prove the second. Denote by φ the applied correction and by v the vertex on which L_{d+1} gets the maximal value in z , if there are more than one vertex then choose v arbitrarily. Since z is reduced, then by Claim 4.34 it holds that $v \in \partial^- z$. Moreover, since v is a local max of L_{d+1} in z the two follows:

1. Claim 4.27 guarantees that $\partial^- \varphi|_v = \partial^- z|_v$. Thus, adding $\varphi|_v$ to z excludes v from the boundary.
2. For any other vertex $u \in z$, there is no face $\theta_{\mathbf{u}, \mathbf{S}'} \in z$ such that $v \in \theta_{\mathbf{u}, \mathbf{S}'}$. Otherwise, it would contradict v being a local maximum of L_{d+1} in z . Thus, $v \notin \mathbf{Str}_{0+}(u, \partial^- z)$, so by Claim 4.27, $v \notin \varphi|_u$.

Combining the two, we get that $v \notin \partial^-(z + \varphi)$, so the reduced form of $z + \varphi$ does not contain v . Namely, $v \notin \xi$. Repeating the above for all the other vertices which get the maximal L_{d+1} -value in z yields that $L_{d+1}(\xi) \leq L_{d+1}(z) - 1$. □

We finished analyzing the first part of the sweep-cycle that acts in the computational basis. In the next section, we show that the second part of the cycle shrinks reduced phase errors.

4.4.1 The Sweep-Rule Phase in the Dual Code

Rewrite. Now, we will think of the binary assignment $z \in \mathbb{F}_2^{\mathcal{F}_{d'}}$ as the positions of the Z -errors, after permuting the qubits by the ϕ -transpose. Namely, $\partial^- z$ is the assignment boundary that we would like to shrink, and $\partial^+ \phi(z)$ is its source prior to the reordering.

In this stage of the sweep-cycle, the sweep-rule acts in opposite directions. Thus, it will be sufficient to show that the extremities of the potential walls in $\partial^- z$ mirror the extremities of the potential walls of $\partial^+ \phi(z)$. Subsequently, since the sweep-rule shrinks $\partial^- z$ (with sign adjustments), we have that the ϕ -transpose of the yielded assignment also admits a shrunk coboundary relative to $\partial^+ \phi(z)$.

However, we would not be able to show an exact mirroring, yet we show that the situation is close, and that would be enough. Specifically, we will show that for $g \in S$, any of the extremities of L_g in $\partial^- z$ are higher (sent closer) than the mirrored extremities of $\partial^+ \phi(z)$. For L_{d+1} , the extremities of $\partial^- z$ have an equal L_{d+1} value to the mirrored extremities in $\partial^+ \phi(z)$ plus $d - d' - 1$.

Considering redefine ϕ to flip the positive direction, such the root of $\phi(\theta_{v, \mathbf{S}'})$ is the up right corner. It seems to gives equality between $\max L_{g/d+1}$ and the $-\min L_{g/d+1}$ in the dual picture.

Claim 4.38. *Let z be an assignment. Then:*

1. For any generator $g \in S$, and potential wall L_g it holds that:

$$\min_{v' \in \partial^- z} L_g(v') \geq - \max_{v' \in \partial^+ \phi(z)} L_g(v')$$

2. The following equality holds:

$$\max_{v' \in \partial^- z} L_{d+1}(v') = \left(- \max_{v' \in \partial^+ \phi(z)} L_{d+1}(v') \right) + d - d' - 1$$

Proof. For the first part, observes that a vertex that is minimizing L_g in $\partial^- z$ is a root of unsatisfied check. Since ϕ -transpose sends checks to checks, we have that the check $\partial^- \theta_{\mathbf{v}^{-1}, \mathbf{S}/\mathbf{S}'} = \phi(\partial^+ \theta_{\mathbf{v}, \mathbf{S}'})$ is not satisfied in $\partial^- z$ if and only if $\partial^+ \theta_{\mathbf{v}, \mathbf{S}'}$ is not satisfied in $\partial^+ \phi(z)$. Thus,

$$\begin{aligned} \min_{v' \in \partial^- z} L_g(v') &= \min \{ L_g(v^{-1}) : \theta_{\mathbf{v}, \mathbf{S}'} \in \partial^+ \phi(z) \} \\ &= - \max \{ L_g(v) : \theta_{\mathbf{v}, \mathbf{S}'} \in \partial^+ \phi(z) \} \\ &\geq - \max \{ L_g(u) : u \in \partial^+ \phi(z) \} \end{aligned}$$

For the second part, observes that if $\theta_{\mathbf{v}, \mathbf{S}'}$ is unsatisfied check in $\partial^- z$ then the vertex which gets the minimal value among the vertices on $\theta_{\mathbf{v}, \mathbf{S}'}$ is the vertex that is the farthest from v . Namely the vertex:

$$u = \left(\prod_{g \in S'} g \right) v$$

So, similarly to the first part, and combining that the checks induce $\partial^+ \phi(z)$ are at dimension $d' + 1$ we have:

$$\begin{aligned} \min_{v' \in \partial^- z} L_{d+1}(v') &= \min \{ L_{d+1} \left(\left(\prod_{g \in S/S'} g \right) v^{-1} \right) : \theta_{\mathbf{v}, \mathbf{S}'} \in \partial^+ \phi(z) \} \\ &= (d - d' - 1) - \max \{ L_{d+1}(v) : \theta_{\mathbf{v}, \mathbf{S}'} \in \partial^+ \phi(z) \} \\ &= (d - d' - 1) - \max \{ L_g(u) : u \in \partial^+ \phi(z) \} \end{aligned}$$

Where the last transition is an equality since the maximum of L_{d+1} is achieved at a root vertex. $\square$

Rewrite. Consider removing, that is because when considering reduced forms there is also equality for L_g . Clearly, the equality regarding L_{d+1} means that advancing 'down' a minimal vertex in $\partial^- z$ implies advancing 'up' a corresponding maximal vertex in $\partial^+ \phi(z)$. The following claim states that the maximum value of L_g in $\partial^+ \phi(z)$ can only decrease (the error doesn't spread in the g -direction) by the application of the sweep-rule in the dual picture ($\partial^- z$).

Claim 4.39. For $g \in S$ denote $M = \min_{v \in \partial^+ \phi(z)} L_g(v)$, and denote by ξ the result of the sweep-rule over z . Then $\min_{v \in \partial^+ \phi(\xi)} L_g(v) \leq M$.

Proof. Assume through contradiction that $\min_{v \in \partial^+ \phi(\xi)} L_g(v) \leq M$. Thus, there has to be a check that is unsatisfied in $\phi(\xi)$ and satisfied in $\phi(z)$. Denote it by $\theta_{v,S'}$, and from the inclusion/exclusion relation, it follows that $\theta_{v^{-1},S/S'} \in \xi$ while $\theta_{v^{-1},S/S'} \notin z$. Thus, one of the qubits adjacent to $\theta_{v^{-1},S/S'}$ was flipped by the sweep-rule. Denote the vertex that flipped the qubits, while performing the sweep-rule, by w (if there is more than one pick arbitrary) and the vertex with the maximal L_g value among the vertices of $\theta_{v,S'}$ by u .

For the sake of the proof, we will say that g^{-1} is on the path from vertex w to vertex v^{-1} , if one has to go through an edge corresponding to generator g (in the opposite direction) for getting to v^{-1} from w . There are two options, let's splitting on whether g^{-1} is on the path from w to v^{-1} .

In the first case, it must be that $L_g(w) = \min L_g(v^{-1})$, and in addition, $g \notin S/S'$, so

$$L_g(u) = L_g(v) = -L_g(v^{-1}) = -(L_g(w) - 1) \leq M$$

In the second case, where g^{-1} is not on the path from w to v^{-1} , it follows that $L_g(w) = L_g(v^{-1})$. Denote the flipped face by f . By the definition of the sweep-rule, there has to be at least one check in the negative local view of w such that it's contained in f , and it's unsatisfied in $\partial^-(z)$ (Otherwise, w would not flip f). Denote it by $\theta_{x,J}$, and since g^{-1} is not on the path between w and v^{-1} , then it's also not on the paths from w to x and from x to v^{-1} , namely $L_g(x) = L_g(w)$ and $g \notin J$. On the other hand, $\theta_{x^{-1},S/J} \in \partial^+ \phi(z)$, and $g \in S/J$, so we get that the maximal value of L_g in $\theta_{x^{-1},S/J}$ is $1 - L_g(x) = 1 - L_g(v^{-1}) \geq L_g(u)$, in contradiction to the fact that u exhibits L_g value greater than M . $\square$

Rewrite. Combining the relation of extremis of an error to its extremis in the dual image, with the argument that the sweep-rule shrinks X -error (∂^- -boundaries), we infer that a full sweep-cycle shrinks an error composed of both X - and Z -components.

Corollary 4.40. *Consider an error whose X - and Z -product components lie at positions corresponding to finite reduced chains $z_X, z_Z \in \mathbb{F}_2^{\mathcal{F}_{d'}}$. Denote by $\xi_X, \xi_Z \in \mathbb{F}_2^{\mathcal{F}_{d'}}$ the finite reduced chains corresponding to the X, Z positions of the result after the application of a full sweep-cycle. Then:*

1. For any $i \in [d]$:

$$\max \{L_i(v) : v \in z_X \cup z_Z\} = \max \{L_i(v) : v \in \xi_X \cup \xi_Z\}$$

2. And for $i = d + 1$:

$$\max \{L_{d+1}(v) : v \in z_X \cup z_Z\} > \max \{L_{d+1}(v) : v \in \xi_X \cup \xi_Z\} + 1$$

4.4.2 Ideal decoder

Rewrite. Although the main role of the section is to show that a single sweep-cycle keeps error clusters under control during a noisy run (by admitting a modest shrinking),

an immediate corollary of this is that the application of non-noisy multiple iterations of the sweep-cycle has the potential to erase errors, namely a decoder. Here, a classical measurement is given, we act on classical bits, and the sweep-cycle consists only of the first stage that works over the computational basis, without moving to the dual code. We formulate the corollary as follows:

Corollary 4.41. *Consider the (d, d') -Toric code, induced by a Toric with side length n . Given a finite reduced error $z \in \mathbb{F}_2^{\mathcal{F}^{d'}}$ on that Toric, with $\mathbf{Size}(z) \leq n - 1$. Then an application of $\mathbf{Size}(z)$ sweep cycles (partial cycles consisting only of the first stage) erases the error.*

4.5 Intermediate

The following definition of propagation describes the evolution of a fault. It assumes an order of steps and defines an operator that is obtained by commuting a given fault with the gates (and faults) prior to a given step.

Definition 4.42 (Fault propagation to a step τ). *Let $C = \{u_i\}_i$ be a quantum circuit, let $J = (j_1, \dots, j_k)$ be an order agreeing with π_C , and let $u_i \in C$. Suppose that $u_i = u_{j_l}$ in this order. If $l > \tau$, we define the propagation of u_i to step τ to be the identity operator. Otherwise, define operators $\mathcal{D}_0, \mathcal{D}_1, \dots, \mathcal{D}_{\tau-l}$ recursively by setting $\mathcal{D}_0 = u_{j_l}$ and requiring that for every $1 \leq s \leq \tau - l$,*

$$\mathcal{D}_{s-1} u_{j_{l+s}} = u_{j_{l+s}} \mathcal{D}_s.$$

We then define the propagation of u_i to step τ to be the operator

$$\mathcal{D} := \mathcal{D}_{\tau-l}.$$

Now let $C[\mathcal{E}]$ be a noisy circuit, where all of its quantum registers are encoded in the (d, d') -Toric code with side length n . Let J be an order agreeing with $\pi_{C[\mathcal{E}]}$, and let τ be a step in that order. For each faulty gate $e \in \mathcal{E}$ that occurs no later than step τ , denote by $\mathcal{D}_e^\tau$ its propagation to step τ . We define the propagated error at step τ by multiplying these propagated faults in reverse topological order.

$$\mathcal{D}^\tau := \prod_{e \preceq \tau} \mathcal{D}_e^\tau.$$

Moreover, we say that a step τ admits an infinite error if there is a register such that, in the view of the ∞ -extension of $C[\mathcal{E}]$, the Pauli decomposition of the propagated error at step τ , $\mathcal{D}^\tau$, has a term with support on a connected, reduced, d' -dimensional chain x such that $\mathbf{Size}(x) \geq n$.

Rewrite. The propagation up to step τ allows us to define the deviation from an ideal run by propagating all the faults and marking all the locations in the result.

Claim 4.43. Let $\alpha = (\Delta d'(d - d' + 1))^{-1}$. Denote by A the event that there is a step that admits an infinite error. In addition, for any step $\tau \in J$, denote by B_τ the event that step τ is the first step for which there is a register such that, in the view of the ∞ -extension, $\mathcal{D}^\tau$ has support on a connected, reduced, d' -dimensional chain x such that $\text{Size}(x) = \alpha n$. Denote by w and μ the width and the depth of $C[\mathcal{E}]$. Then the probability that A occurs is bounded by:

$$\Pr[A] \leq w\mu \max_{\tau \in [w\mu]} \Pr[B_\tau].$$

Proof. For the sake of the proof, we say that a chain x is in operator O if, when considering the Pauli decomposition of the image of O in the ∞ -extension, there is a correspondence between the faces of x and the (partial) support of one of the terms in O 's image.

Denote by x the maximal, in size, reduced connected d' -dimensional chain in $\mathcal{D}^{\tau+1}$. Assume that τ is the first step to admits an infinite error, thus any chain in $\mathcal{D}^\tau$ has a size lower than n . Denote by u the gates which is applied at step τ , so there have to be Paulis P_1, P_2 such that:

$$P_1 \in \text{Supp } \mathcal{D}^\tau, P_2 \in \text{Supp } \mathcal{D}^{\tau+1}, x \text{ is both in } P_1, P_2 \text{ and } \langle P_1 | u | P_2 \rangle \neq 0$$

Since u has a fan at most Δ , and since any qubit (d' -dimensional face) is connected to at most $d'(d - d' + 1)$ qubits (by stepping through a check¹¹), we get that the application of u on P_1 touches at most α^{-1} chains, and the result of the modification yields x . Therefore, x resides inside a union of at most α^{-1} , and since x is connected, we have that its size is bounded by summing the size of each chain in the union. Therefore, if any of the chains is at size at most αn , it implies that the size of x is lower than n . Hence, a necessary condition to step $\tau + 1$ to be the first step to admit an infinite error is that step $\tau - 1$ admits errors at size greater than αn , namely the occurrence of B_τ . The inequality follows by using the union bound over all the steps in $C[\mathcal{E}]$. $\square$

We close the section by presenting Pauli-paths, which can be thought of as the collapsing errors due to stabilizer measurements.

Definition 4.44. For a fixed quantum circuit C , where all its quantum registers are encoded with the Toric code, and a set of faults $\mathcal{E}$, such that the depth of $C[\mathcal{E}]$ is μ , we will say that the Paulis $P_1, \dots, P_{\mu+1} \in \mathcal{P}$ are fault paths of $C[\mathcal{E}]$ if P_1 is the identity and for every $t \leq \mu + 1$ the Pauli P_{t+1} supports $C[\mathcal{E}]|_t P_t$. Namely:

$$\langle P_t | C[\mathcal{E}]|_t | P_{t+1} \rangle \neq 0$$

We extend the notation to Pauli-path up to step τ by considering the quantum circuit obtained from $C[\mathcal{E}]$ by taking all the gates at steps lower than or equal to τ . A Pauli-path of that circuit is a Pauli-path up to step τ of the original circuit.

¹¹a $d' - 1$ dimensional face

Finally, we define a reduced Pauli-path $\tilde{P}_1, \dots, \tilde{P}_{\mu+1}$ to be the Paulis obtained by taking a Pauli-path, considering its view in the ∞ -extension, and for any finite connected chain that corresponds to a subset of the X component of P_t 's support, substitute it with its reduced form (Definition 4.35). Then do the same for the Z component in the dual picture.

4.6 Gottesman argument

Even better. Assume that the clusters always have a size lower than γn and that there are m bits. Run sweep-cycles for $m\gamma n$ times. Since there can be at most $m - 1$ merges (since the number of clusters is bounded by the number of bits), there has to be a continuous sequence of γn cycles for which no merge occurs. Since it's given that the size of all the clusters is lower than γn , we have that all the clusters are erased after those cycles. To prove that no cluster reaches a size greater than γn , we use explanation trees (where trees in which all the leaves are at least l from the root are contained in the trees without that restriction).

4.7 Toric Encoded Circuits and Toom's Contours

We are about to zoom out from a single register encoded in the Toric code to describe a fully fault-tolerant computation. For a given quantum circuit, assumed to consist of gates **CX**, **H**, **S**, **T** and measurements, we define its Toric encoded circuit as its version where each qubit is encoded by the Toric code. Formally:

Definition 4.45 (Toric Encoded Circuit). *Let C be a quantum circuit. We say that C is a Toric encoded circuit if the following holds:*

1. *Any layer of the circuit consists of classical and quantum registers, such that there is a Toric code $\mathcal{C}$ such that the quantum registers are encoded by the nice encoding of $\mathcal{C}$. At time $t = 0$, the quantum registers are initialized to be either $|\bar{0}\rangle$, encoded $|\mathbf{H}\rangle$, encoded $|\mathbf{S}\rangle$, or encoded $|\mathbf{T}\rangle$. The classical registers are initialized to be zeros.*
2. *The layers of C on the quantum registers at even times are sweep-cycles, and the layers at odd times are realizations of either logical Paulis, **CX**, or measurements in the computational and parity bases. On the classical registers, including the measured register, we allow any two-bit gates.*

For Toric encoded circuits, we define their *base graph*, which can be thought of as a recording of a running from the Toric's point of view. The *base graph* is obtained by taking the Toric's vertices at different times as the vertices and connecting any two sequential in time if they are support faces — qubits — which are either the same or are entangled by a quantum gate.

Definition 4.46 (Base graph G_o). *Let C be a Toric encoded circuit at depth μ . Notice that measurements take quantum registers into classical, Thus any quantum register has a maximal depth, which we denote by $\{\mu_i\}$. Let's denote by $\{\mathcal{G}_i\}$ the skeleton of*

the ∞ -extension of the Toric complexes that induce the codes. We define the base graph of C , denoted by $G_o = (V_o, E_o)$, as follows:

1. The vertices V_o are tuples, where the first coordinates correspond to a vertex in the disjoint union over the vertices in $\{\mathcal{G}_i\}$, and the second coordinate is associated with a time. Namely:

$$V_o := \bigcup_i (V(\mathcal{G}_i) \times [\mu_i])$$

2. The edges E_o are derived from the original edges in $\mathcal{G}_1, \dots, \mathcal{G}_k$, with the addition that any two vertices in preceding times t and $t + 1$ that support qubits (faces) such that C^∞ acts non-trivially on those qubits at time t are also connected by an edge. Namely:

$$E_o := \{ \{(v, t), (u, t')\} : \text{if} \\ \text{either there exists } i \text{ such } \{v, u\} \in \mathcal{G}_i \text{ and } t' = t + 1 \\ \text{or there exists } u_i = (g_i, l_i) \in C^\infty \text{ such } v, u \in l_{i,2} \text{ and } t = l_{i,1}, t' = l_{i,1} + 1 \}$$

Where we define a vertex v to belong to location l if there is a qubit-face f in l such that v belongs to its associated face.

As a concept, if the base graph is a recording of a run from the Torices' point of view, then a *Tooms contour* is a recording of the deviation's evolution in the eyes of the Torices vertices. More technically, for a reduced Pauli-path $P_1, \dots, P_{\mu+1}$ (Definition 4.44), the Tooms contour is the restriction of the base graph to vertices that witness a deviation, namely, those that belong to qubits supporting $P_1, P_2, \dots, P_{\mu+1}$.

Definition 4.47 (Tooms Contour Graph of $C(\mathcal{E})$). For a quantum circuit C , set of faults $\mathcal{E}$ and a reduced Pauli-path $P_1, \dots$, let us denote the base graph of $C[\mathcal{E}]$ by G_o . We define the Tooms contour graph $\mathcal{H} = (V_{\mathcal{H}}, E_{\mathcal{H}})$ as follows:

1. The vertices $V_{\mathcal{H}}$ are the vertices of faces supporting the reduced Pauli-path. Namely:

$$V_{\mathcal{H}} := \{v_t : v_t = (v, t) \in V_o \text{ and there exists a face } f \in \text{supp } P_t \text{ such } v \in f\}$$

2. The edges $E_{\mathcal{H}} \subset E_o$ is partitioned into two types $A_{\mathcal{H}}$ and $F_{\mathcal{H}}$, where for any logical gate g which applied in time t in $C[\mathcal{E}]^\infty$ we add edge to $A_{\mathcal{H}}$ as follow:

- (a) If g is either a logical Pauli, a measurement (both in the computational and phase basis), or an identity, then for any face f in the register on which g acts, we add:

$$\{ \{(u, t), (v, t + 1)\} : u, v \in f \text{ and } v, u \in V_{\mathcal{H}} \}$$

- (b) If g is a logical **CX** then for any physical **CX** in its realization, between faces f_1 into f_2 at time t we add:

$$\{ \{ (u, t), (v, t+1) \} : u \in f_1 \cup f_2, v \in f_1 \cup f_2 \text{ and } v, u \in V_{\mathcal{H}} \}$$

In addition for any fault g of $\mathcal{E}^\infty$, that is applied at time t , and any face $f \in \text{Locc}(g)$, we add the edges:

$$\{ \{ (u, t), (v, t+1) \} : u, v \in f \text{ and } u, v \in V_{\mathcal{H}, X} \}$$

To construct $F_{\mathcal{H}}$, we connect any pair of vertices (v, t) , (u, t) if there exists a vertex $(w, t+1)$ such both of them are connected to it through $A_{\mathcal{H}}$. Namely:

$$F_{\mathcal{H}} = \{ e = \{u, v\} \in E_o : \text{exists } w \in V_{\mathcal{H}} \text{ such } \{v, w\}, \{u, w\} \in A_{\mathcal{H}} \}$$

We call $A_{\mathcal{H}}$ and $F_{\mathcal{H}}$ the arrows and forks edges respectively.

For the analyses, we would like to extend Toom's contour to record in their final layer a partial time-step, namely a computation up to a certain step Definition 3.2. For this, we substitute in the definition above the Pauli-path with a Pauli-path up to a step. Moreover, since we would like to refer to the connected chain at the final layer as the 'final error clusters', we add at the end a layer whose purpose is to connect in time (by arrows) the connected components of Toom's contour in the prior layer, which are connected by checks (their union is identified with a connected chain).

Definition 4.48. For a Toric encoded quantum circuit C , a set of faults $\mathcal{E}$, and a step τ , according to the standard order, let $C[\mathcal{E}]_\tau$ be the partial computation circuit of $C[\mathcal{E}]$ up to step τ . We define the analysis Toom's contour at step τ $\mathcal{H}_\tau = (V_{\mathcal{H}}, E_{\mathcal{H}})$ to be the graph obtained by taking the Toom's contour of $C[\mathcal{E}]_\tau$ and adding vertices and edges as follows:

1. For vertices, we define t^* as the timeunit of the gate at step τ plus one, and then add duplication of the previews layer, namely the subset:

$$\{ v_{t^*} : v_{t^*} = (v, t^*) \text{ such there exists a face } f \in \text{supp } P_{t^*-1} \text{ such } v \in f \in P_{t^*-1} \}$$

2. For the edges, we add the following arrows:

$$\begin{aligned} & \{ \{ (u, t^*-1), (v, t^*) \} \text{ if there exists a face } f \in \partial P_{t^*-1} \\ & \text{ such } v_{t^*}, u_{t^*-1} \in f \} \end{aligned}$$

3. Finally, we add forks for vertices at time t^* to preserve the property that any two vertices with the same time coordinate, which are connected to a third via arrows, are also connected by a fork.

4.8 Space-Time Graph

Not a few of the results about Tooms contours are derived from their directed structure, regardless of their connection to Torices. Thus, we split the discussion to talk about *space-time graphs*, which are graphs whose edges can be partitioned into arrows and forks.

Here, we present the results and proofs first given by Berman and Simon [BS88] and later by Gacs [Gac21] about *space-time graphs*. Although the proofs we give are almost identical, we chose to present them rather than referring to the sources. The reason for this is that in the language appearing in [BS88] and [Gac21], vertices are bits¹², so conceptually their Toom's contours track different creatures. Thus, jumping into their text while keeping our image in mind might cause confusion, and we want to avoid that.

Definition 4.49. Consider a graph $G = (V, A, F)$, where V is a set of vertices, and A and F are sets of undirected edges. For $v \in V$, denote by $\text{pre}(v) = \{u \in V : \{u, v\} \in A\}$, we extend the notion for subsets of vertices $X \subset V$ by $\text{pre}(X) = \bigcup_{v \in X} \text{pre}(v)$. Let $T: V \rightarrow \mathbb{N}$. We say that (G, T) is a time graph if and only if:

1. $\{x, y\} \in A \Rightarrow T(y) = T(x) + 1$
2. For $x, y \in V$ $\{x, y\} \in F$ if and only if there exists $z \in V$ such both $\{x, z\}, \{y, z\} \in A$.

We will call to T the time coordinate, to A arrows, and to F forks. Observe that from the second requirement it follows that forks can connect only vertices that have the same time value (set by T).

Remark 4.50. The Toom's contour graph defined in Definition 4.47 is, by definition of its construction, a time-graph.

The history of a vertex consists of all the vertices that are connected to it by arrows. Here we allow passing through arrows in both directions, so it might be possible that the history of vertex u contains vertices with the same time coordinate as u . That motivates to define *slices*. A *slice* is a (maximal) subset of vertices with the same time coordinate such they share the same history. In [BS88], slices take the role of error droplets.

Definition 4.51 (History, slice.). For a time graph (G, T) and a vertex $u \in V$, let the history of u , denoted by H_u , be the restriction of G to vertices that both precede u in time and connect to u via path of arrows. A slice is a maximal subset of vertices $K \subset V$ such that for any two vertices $u, v \in K$, $H_u = H_v$ is satisfied. As a consequence, the time coordinate in K has to be constant. Thus, we can extend the time coordinate and the history to slices and write for a slice K , $T(K)$ and H_K .

¹²It is also true that the gates are different, yet that difference alone does not constitute a conceptual change.

The following claim ensures us an 'independent' argument about slices. Here we state that two slices with the same time coordinate have to hold disjoint histories. Another way to think about this is that if the first and the second slices are 'induced by n_1 and n_2 faults,' then the slice obtained by merging them 'is induced by $n_1 + n_2$ faults.'

Claim 4.52. *Let K_1, K_2 be two slices such that $T(K_1) = T(K_2)$, then either $K_1 = K_2$ or H_{K_1}, H_{K_2} are disjoint.*

Proof. Assume a non-empty intersection, so there exists $z \in H_{K_1} \cap H_{K_2}$ such that for any $x \in K_1$ and $y \in K_2$, there exist arrow-paths $z \rightarrow x$ and $z \rightarrow y$. Thus, for any $w \in H_{K_1}$, one can concatenate a path $w \rightarrow x \rightarrow z \rightarrow y$ and conclude that $y \in H_{K_2}$. $\square$

Next, we would like to study how slices look a moment before they merge. For this, we define the *graph of a slice*. We construct this graph by taking a slice and considering all the slices that precede it by a single time unit. We take those slices as vertices and connect any two of them that have vertices sharing a fork.

Definition 4.53 (The graph of slice). *Let K be a slice. The graph of K , denoted by $G_K = (V_K, E_K)$, is the graph whose vertices are slices $R \subset H_K$ with $T(R) = T(K) - 1$. Two vertices are connected by an edge if and only if there is a fork whose ends belong to the slices associated with them. Namely, $S, R \in V_K$ are connected in G_K if and only if there is $\{s, r\} = f \in F$ such that $s \in S$ and $r \in R$.*

The graph of slices is connected. That fact means that a spanning tree of a (subgraph of) a slice's history can be found by combining the spanning trees of its processors. The following gives the proof:

Claim 4.54. *G_K is connected.*

Proof. Let $R, S \in V_K$ and consider $r, s \in V$ such that $r \in R, s \in S$. Recall that $R, S \subset H_K$ and therefore $r, s \in H_K$. Thus, by definition of the 'History' there exists a path, passing through arrows only, from r to s in H_K . Consider a path that is minimal in the number of vertices with a time coordinate equal to $T(K)$; denote that number by k . We prove by induction on k that R and S are connected in G_K .

1. Base. $k = 0$, In this case, any vertex on the path between r and s has an equal or lower time coordinate than r and is also connected to r via a path of arrows, namely belongs to H_r . In particular, s is also a vertex on the path, and therefore $s \in H_r \Rightarrow H_r = H_s$, so $R = S$.
2. Induction step. Since $k > 0$, there exists a vertex y on the path from r to s such that $T(y) = T(K)$. Let x be the vertex which precedes y on the path, and z be the vertex that follows y . Since the values of T at the two ends of an arrow differ by 1, and since y gets the maximal T value in H_K it follows that:

$$(a) \quad \{x, z\} \subset \text{pre}(y) \Rightarrow \{x, z\} \in F.$$

- (b) The slices of x and z , say K_x and K_z has time value $T(K_x) = T(K_z) = T(K) - 1$, Namely they are both members of V_K .

Since the path connecting r, x in H_K has $k - 1$ vertices with $T(\cdot) = T(K)$, then by the induction hypothesis, R and K_x are connected in G_K , and by similar argument so are K_z and S . On the other hand, $\{x, z\} \Rightarrow \{K_x, K_z\} \in E_K$. Thus, R and S are connected in G_K .

□

Now, we would like to plug in the fact that the vertices originate from the infinite plane. A spanned slice is a presentation of a slice equipped with poles. The poles are the vertices in the slice, which are the maximal points of potential walls.

Definition 4.55 (Spanned Set). *Consider a subset $A \subset \mathbb{R}^d$, and $d + 1$ points $x_1, \dots, x_{d+1} \in \mathbb{R}^d$. We say that $\langle A, x_1, \dots, x_{d+1} \rangle$ is a spanned set of A if for any $i \in [d + 1]$ and $s \in A$, it holds that $L(s) \leq L_i(x_i)$ and in addition there are $v_1, \dots, v_{d+1} \in V$, not necessary different, that achieve the equalities $L_i(x_i) = L_i(v_i)$. We name $x_1, \dots, x_{d+1}$ the poles of A . Furthermore, we extend the notation to a spanned slice when A is a slice of some time-graph. Notice that saying $\langle A, x_1, \dots, x_{d+1} \rangle$ is a spanned set means that $\mathbf{Size}(A) = \mathbf{Size}(\{x_1, \dots, x_{d+1}\})$.*

Observe that in the case where the subset A is a slice. Each of these poles can be associated with a vertex of G that gives the equality. If there are multiple vertices whose give equality, associate the point with one of them arbitrarily. We will use the notion of applying functions originally defined over vertices to the poles, referring to the application over the vertices associated with them. For example $\text{pre}(x_i) = \text{pre}(v)$ for the vertex $v \in A$ which satisfies $L_i(v) = L_i(x_i)$ and therefore is associated with the pole x_i . Another example is referring to x_i as a vertex, so when saying 'For $x_i \in V_k$ ' or 'For $\{x_i, u\} \in F$ ', we refer to the associated vertex or the edge connecting it to u .

The following claim relates the size of a given slice to the sizes of its predecessor slices.

Claim 4.56. *Consider a time graph (G, T) , Let K be a slice in G , and let $\hat{K} = \langle K, x_1, \dots, x_{d+1} \rangle$ be a spanned slice of K such that $K \neq H_K$. Then for some integer k there exist spanned slices $\hat{K}_0 = \langle K_0, \cdot \rangle, \dots, \hat{K}_k = \langle K_k, \cdot \rangle$ and Forks $F_1, \dots, F_k$ such that:*

1. *For $i \in [d + 1]$ $\text{pre}_i(x_i)$ belongs to the poles $\hat{K}_0, \dots, \hat{K}_k$.*
2. *Introduce an edge between $\hat{K}_i$ and $\hat{K}_j$ if and only if one of the Forks $F_1, \dots, F_k$ consists of a pole of $\hat{K}_i$ and a pole of $\hat{K}_j$. Then the graph formed from $\hat{K}_0, \dots, \hat{K}_k$ is connected.*
3. $\sum_{i=0}^k \mathbf{Size}(\hat{K}_i) + \sum_{i=1}^k \mathbf{Size}(F_i) \geq \sum_{i=1}^{d+1} L_i(\text{pre}_i(v_i))$

Proof. First Notice that K doesn't contain leaves in (V, A) , since a leaf is connected only to itself via arrows, and therefore its 'slice' equals to its 'History'. Thus $\text{pre}_i(x_i)$ is not empty for $i = 1, \dots, d + 1$. Next consider the graph $G_K = (V_K, E_K)$ from

definition 4.53. Since $\{x_i, \text{pre}_i(x_i)\}$ is an arrow $\text{pre}_i(x_i)$ belongs to a slice in V_K . For $i = 1, \dots, d+1$ denote by R_i the slice in V_K that contains $\text{pre}_i(x_i)$.

By claim 4.54 G_K is connected and therefore there exist a minimal collection $\{K_0, \dots, K_k\}$ of slices form V_K which contains $R_1, R_2, \dots, R_{d+1}$, and which is connected in G_K . We pick a set of forks $\mathbf{F} = \{F_1, \dots, F_k\}$ which realizes a spanning tree for this collection of slices. Later we will identify edges from this spanning tree with the forks from $\mathbf{F}$.

For $i = 1, \dots, d+1$ we set $\text{pre}_i(v_i)$ to be the i th pole of R_i , This assures (1). The set of remaining poles for $\hat{K}_0, \dots, \hat{K}_k$ will be equal to $\cup_{i=k}^k F_i$. This will assure (ii), We will also select poles for $F_1, \dots, F_k$ to form 'spanned forks' $\hat{F}_1, \dots, \hat{F}_k$ in such way that:

$$\sum_{i=0}^k \text{Size}(\hat{K}_i) + \sum_{i=1}^k \text{Size}(\hat{F}_i) \geq \sum_{i=1}^{d+1} L_i(\text{pre}_i(v_i))$$

This will assure (3).

Let us denote by $J_0, \dots, J_{2k}$ the enumerated union $K_0, \dots, K_k, F_1, \dots, F_k$. Observes that for any choice of $(d+1)(2k+1)$ elements $z_0^1, z_0^2, z_0^3, \dots, z_0^{d+1}, \dots, z_{2k}^{d+1}$ in $\mathbb{R}^d$, such that $z_j^i \in J_j$ it holds that:

$$\sum_{i=0}^k \text{Size}(\hat{K}_i) + \sum_{i=1}^k \text{Size}(\hat{F}_i) \geq \sum_{j,i} L_i(z_j^i) \quad (1)$$

Later, we are going to take the points $\{z_j^i\}$ from the intersection of J_j 's. For convenience, let us assume that for the index subset $X \subset [2k] \cup \{0\}$, the intersection $\bigcap_{j \in X} J_j$ returns a single point. If there is more than one, then decide on the point in a way that all the chosen points simultaneously satisfy that for any index subsets Y, X , such that $X \cap Y$ and they both introduce a non-trivial intersection $\bigcap_{j \in Z} J_j : Z \in \{X, Y\}$, then:

$$\bigcap_{j \in X} J_j = \bigcap_{j \in Y} J_j$$

Now consider a non-trivial maximal intersection $\bigcap_{j \in X} J_j$, the intersection is maximal in the sense that for any $j' \notin X$ it holds that $\bigcap_{j \in X \cup \{j'\}} J_j = \emptyset$. We claim that for any i and for any such X , there is $t \in X$ such that J_t is the successor of J_j on the path to R_i for any $j \in X/\{t\}$. Assume not. Let $x, y \in X$, and denote the paths from J_x, J_y to the i th pole by $\langle J_x, J_{x_2}, \dots, R_i \rangle$, $\langle J_y, J_{y_2}, \dots, R_i \rangle$, such that $J_y \neq J_{x_2}$. If $y = x_2$, $y_2 = x$, or $x_2 = y_2 \in X$, then pick other x, y (if there is no such, we get an immediate contradiction). In that case, $\langle J_x, J_y, J_{y_2}, \dots, R_i \rangle$ is a path from J_x to R_i which is different from $\langle J_x, J_{x_2}, \dots, R_i \rangle$. Therefore, there are at least two paths from J_x to the i th pole, namely, there is a cycle in contradiction to the minimality of $K_0, \dots, K_k$.

We will choose the z_j^i by the following process. Pick a z_j^i that has not been set yet. If $J_j = R_i$, then set $z_j^i \leftarrow \text{pre}_i(x_i)$ - we call this a type-1 assignment. Otherwise, consider the path from J_j to R_i (by connectivity, it has to be such a one, and by

minimality, there is exactly one). Let J_t be the successor on the path, then pick $z_j^i \in J_j \cap J_t$ - similarly, we call this assignment a type-2 assignment. Since any type-2 assignment is associated with a non-trivial intersection, and any non-trivial intersection induces a type-2 assignment for all i 's (since for all i 's, one of the J_j on its support is a successor of the rest on their path to the i th pole), we have that:

$$\begin{aligned}
\sum_{j,i} L_i(z_j^i) &= \sum_{j,i,z_j^i \in \text{type-1}} L_i(z_j^i) + \sum_{j,i,z_j^i \in \text{type-2}} L_i(z_j^i) \\
&= \sum_i^{d+1} L_i(\text{pre}_i(x_i)) + \sum_{X: \bigcap_{j \in X} J_j \neq \emptyset} (|X| - 1) \sum_i^{d+1} L_i \left(\bigcap_{j \in X} J_j \right) \\
&= \sum_i^{d+1} L_i(\text{pre}_i(x_i))
\end{aligned}$$

Plugging it into eq. (1) gives the desired. $\square$

Until now, the claims stated were true for any kind of space-time graph (where their vertices can be associated with coordinates in $\mathbb{R}^d$). The next claims assume that the given time graph is a Toom's contour induced by the noisy running of a Toric encoded circuit.

Claim 4.57. *For Toom's contours, the maximal size of a fork is $2d'$.*

Proof. Consider the rules of forks connecting in Definition 4.47. For forks obtained by rules (a), (b), or as a result of a fault, it's clear that their vertices at their ends are on intersecting faces (when projecting the vertices on $\mathcal{G}$). The maximal fork, in those cases, has the form of:

$$F = \{v, (\prod_{g \in S'_1} g)(\prod_{g \in S'_2} g)v\}$$

Where S'_1 and S'_2 are subsets of S of size d' , and therefore the size of such forks is $\text{Size}(F) = 2d'$. $\square$

While in the construction presented in [BS88], all the layers were the same (consisting of the same gates up to wiring), here different layers might differ both in realization and role. Thus, we would like to distinguish between them. We will define the *predecessor decomposition* of a slice to be the collection of slices and forks which satisfy the condition in Claim 4.56. The order of a slice is a number in $\{0, 1, 2, 3\}$ that matches the time modulo four of the layer that results in the slice. Therefore, it encodes the layer type.

Definition 4.58. *Let $\hat{K}$ be a spanned slice, and let $\hat{K}_0, \dots, \hat{K}_k$ be spanned slices in H_K , and let $F_1, \dots, F_k$ be spanned forks, such that $\hat{K}_0, \dots, \hat{K}_k, F_1, \dots, F_k$ satisfy the conditions in Claim 4.56. We call $\hat{K}_0, \dots, \hat{K}_k, F_1, \dots, F_k$ the predecessor decomposition of $\hat{K}$.*

Furthermore, we say that $\hat{K}$ is a result of a sweep-cycle, faults, or a logical operation if the layer at time $T(\hat{K}) - 1$ consists of sweep-cycle gates, fault gates (whose source is $\mathcal{E}$), or either measurement or logical Pauli. The order of the slice is its time modulo 4, namely $\text{order}(\hat{K}) = T(\hat{K}) \bmod 4$. Observe that slices that are results of faults have an order that is either 0 or 2, whereas slices that are results of a sweep-cycle or logical operation are of orders 1 and 3, respectively.

The distinction between slices resulting from different layers allows us to adjust Claim 4.56 conditioning on the slice type.

Corollary 4.59. *Consider a spanned slice $\hat{K}$, at finite size, such that $K \neq H_K$. Since $K \neq H_K$, it follows that $\hat{K}$ has a predecessor decomposition, denoted by $\hat{K}_0, \dots, \hat{K}_k, F_1, \dots, F_k$. The inequality in Claim 4.56 becomes:*

1. *If $\hat{K}$ is a result of a ξ -sweep-cycle (namely, the result of ξ sequential sweep cycles), then:*

$$\sum_{i=0}^k \text{Size}(\hat{K}_i) + \sum_{i=1}^k \text{Size}(F_i) \geq \text{Size}(\hat{K}) + \xi$$

2. *If $\hat{K}$ is a result of either faults or a logical operation, then:*

$$\sum_{i=0}^k \text{Size}(\hat{K}_i) + \sum_{i=1}^k \text{Size}(F_i) \geq \text{Size}(\hat{K})$$

Proof. For the sweep-cycle part, we use Corollary 4.40, which states that after each application of the sweep-cycle over a finite size assignment, the maximal L_{d+1} value of the resulting boundary decreases by 1, while the values of each L_g do not increase.

When $\hat{K}$ is a result of either faults¹³ or a logical operator, we use the fact that since the operators are transversal, we have that the support of $\hat{K}$ is contained in the support of $\cup_0^k \hat{K}_i$, and therefore for any $i \in [d+1]$:

$$\max\{L_i(v) : v \in \hat{K}\} \leq \max\{L_i(v) : v \in \bigcup_{i=0}^k \hat{K}_i\}$$

□

The following claim extends the corollary to slices at the final layer of an *analysis Toom's contour* at step τ .

Claim 4.60. *For a step τ , let $\mathcal{H}_\tau$ be analysis Toom's contour at step τ , and let t^* be the maximal time at that contour. For a spanned slice $\hat{K}$, at time $T(K) = t^*$, where $K \neq H_K$, the inequality in Claim 4.56 becomes:*

$$\sum_{i=0}^k \text{Size}(\hat{K}_i) + \sum_{i=1}^k \text{Size}(F_i) \geq \text{Size}(K) - d - d' - 1$$

¹³Notice that $\hat{K}$ does not consist of faults since otherwise $K = H_K$, so the meaning here is that $\hat{K}$ contains vertices which weren't flipped in the fault layer.

Proof. We allow vertices at the final time layer to be connected by an arrow if their counterpart on the skeleton shares a check. Therefore, for any $i \in [d]$, the value of L_i for v_i might be greater by at most 1 than the value of $\text{pre}_i(v_i)$, and the L_{d+1} for v_{d+1} might be greater by at most $d' + 1$ than the value of the $d + 1$ pole of $\hat{K}$. Hence, overall, we get that:

$$\begin{aligned} \sum_{i=1}^{d+1} L_i(\text{pre}_i(v_i)) &\geq \sum_{i=1}^d (L_i(v_i) - 1) + L_{d+1}(v_{d+1}) - \max\{d' + 1, d' - 1\} \\ &= \mathbf{Size}(K) - d - d' - 1 \end{aligned}$$

□

4.9 Explantation of a Spanned Slice

We are about to prove the main lemma stating that the number of faults causing an occurrence of slice is linear in both the slice's size and the minimal number of edges required to span a Tooms contour that induces it.

First, we define the explanation tree of a slice to be the subtree of its history that connects its poles. The relation between the leaves of the subtree and its arrows, forks, and the size of its origin slice is embedded as a requirement in the definition. Then, we prove that any slice has an explanation tree.

Definition 4.61. *Consider a Tooms contour graph induced by Toric encoded circuit, with sweep-cycle parameter $\xi = 1$, and let α, β be constants and $\mathcal{X}_0, \mathcal{X}_1, \mathcal{X}_2, \mathcal{X}_3 \in \mathbb{A}$ be quadruple, defining such:*

$$\begin{aligned} \alpha &:= 40(d + 1)^2, \quad \beta := 5(d + 1) \\ \mathcal{X}_r &:= 1 - r \frac{d + 1}{\alpha} = 1 - \frac{r}{40(d + 1)} \end{aligned}$$

Given sets W of points, A of arrows and F of forks, we define U as the set of vertices of the graph induced by the edges of $A \cup F$. The sets W, A and F form an explanation of a spanned slice $\hat{K} = \langle K, v_1, \dots, v_{d+1} \rangle$ of order $\text{order}(\hat{K}) = r$, if and only if:

1. $W \cup \{v_1, \dots, v_{d+1}\} \subset U \subset H_K$ and the graph $(U, A \cup F)$ is connected.
2. All elements of W are leaves.
3. For $m = |W|$ and $s = \mathbf{Size}(K)$ we have $|A| \leq \alpha(m - \mathcal{X}_r) - \beta s$ and $|F| < m - 1$.

Furthermore, for a step τ and a spanned slice $\hat{K}$ in the analysis Toom's contour at step τ , we extend the definition of explanation as above, but require the following weaker inequality instead: For $m = |W|$ and $s = \mathbf{Size}(K)$, we have $|A| \leq \alpha(m + 1) - \beta s$ and $|F| < m - 1$.

Claim 4.62. *Every spanned slice $K = \langle K, v_1, \dots, v_{d+1} \rangle$ has an explanation.*

Proof. Let $h = T(K) - \min_{u \in H_K} T(u)$. For spans derived from the standard Toom's contours (namely not a analysis Toom's contours), we prove by induction on h .

1. Base: $h = 0$, meaning T is constant on H_K , so no arrows connect points of H_K . Thus, H_K , as well as $\mathbf{K}$, consists only of a single leaf, namely $\hat{K}$ is a result of a fault, and $r = \text{order}(\mathcal{X}_r) \in \{0, 2\}$. In that case, there is a single vertex $u \in V_{\mathcal{H}}$ such that $U = W = \{u\}$, so $m = 1$, $s = 0$, and $|A|, |F| = 0$. It is easy to check that those values satisfy the requirement.
2. Induction Step: $h \neq 0$ implies $K \neq H_K$. Thus, by Claim 4.56, we know that for some k there exist spanned slices $\hat{K}_0, \dots, \hat{K}_k$ and forks $F_1, \dots, F_k$ that satisfy Claim 4.56. In particular, for $r < 3$ it holds that:

$$\sum_{i=0}^k \text{Size}(\hat{K}_i) + \sum_{i=1}^k \text{Size}(F_i) \geq \text{Size}(\hat{K})$$

Whereas, for $r = 3$:

$$\sum_{i=0}^k \text{Size}(\hat{K}_i) + \sum_{i=1}^k \text{Size}(F_i) \geq \text{Size}(\hat{K}) + \xi = \text{Size}(\hat{K}) + 1$$

Consider $i \in \{0, \dots, k\}$, and denote $s_i = \text{Size}(\hat{K}_i)$. By the inductive hypothesis and since $T(K_i) = T(K) - 1$, we know that $\hat{K}_i$ has an explanation formed by a set of leaves $\mathbf{W}_i$, a set of arrows $\mathbf{A}_i$, and a set of forks $\mathbf{F}_i$. Let $m_i = |\mathbf{W}_i|$. Thus, the sets $\mathbf{A}_i$ and $\mathbf{F}_i$ have at most $\alpha(m_i - \mathcal{X}_{r-1}) - \beta s_i$ arrows and $m_i - 1$ forks. We define:

- (a) $\mathbf{W} = \cup_{i=0}^k \mathbf{W}_i$, Notice that $\mathbf{W}_i \subset K_i$ and that $T(K_i) = T(K_j)$ for all i, j , thus by Claim 4.52 $\mathbf{W}$ is a union of disjoint set, Hence W contains $m = \sum_{i=0}^k m_i$ leaves.
- (b) $\mathbf{F} = \{F_1, \dots, F_k\} \cup_{i=0}^k \mathbf{F}_i$ with at most $k + \sum_{i=0}^k m_i - 1 = m - 1$ elements.
- (c) $\mathbf{A} = \{\{v_i, \text{pre}_i(v_i)\}\} \cup_{i=0}^k \mathbf{A}_i$. For bounding the number of arrows in $\mathbf{A}$ we condition upon whether $r = 3$, For convenient, let us denote by Y the indicator, which indicate if $r = 3$.

$$\begin{aligned} & d + 1 + \sum_{i=0}^k \alpha(m_i - \mathcal{X}_{r-1}) - \beta s_i \\ & \leq d + 1 + \alpha m - \mathcal{X}_{r-1} \alpha(k + 1) - \beta(s + Y\xi - k \cdot \text{Size}(\text{fork})) \\ & = \alpha(m - \mathcal{X}_r) - \beta s + (\alpha(\mathcal{X}_r - \mathcal{X}_{r-1}) + d + 1 - \mathcal{X}_{r-1} k \alpha - \beta Y \xi + \beta k \cdot \text{Size}(\text{fork})) \end{aligned}$$

So it's left to show that the term in the closure is negative for all integers k :

$$\alpha(\mathcal{X}_r - \mathcal{X}_{r-1}) + d + 1 - \mathcal{X}_{r-1} k \alpha - \beta Y \xi + \beta k \cdot \text{Size}(\text{fork})$$

We do that by showing that for the choice of the parameter values, $\alpha, \beta, \mathcal{X}_0, \dots, \mathcal{X}_3$, the slope is negative, and then the value of the term at k equals zero is negative. For the slope to be negative, we require:

$$-\alpha \mathcal{X}_{r-1} + \beta \mathbf{Size}(\text{fork}) \leq 0 \Rightarrow \mathbf{Size}(\text{fork}) \leq \frac{\alpha}{\beta} \mathcal{X}_{r-1}$$

For negativity at the origin, $k = 0$, we require for $r < 3$:

$$\begin{aligned} (\mathcal{X}_r - \mathcal{X}_{r-1})\alpha + d + 1 &\leq 0 \\ \Rightarrow \mathcal{X}_r &\leq \mathcal{X}_{r-1} - \frac{d+1}{\alpha} \end{aligned}$$

And for $r = 3$:

$$\begin{aligned} (\mathcal{X}_3 - \mathcal{X}_2)\alpha + d + 1 - \beta\xi &\leq 0 \\ \Rightarrow \mathcal{X}_3 &\leq \mathcal{X}_0 - 3\frac{d+1}{\alpha} + \frac{\beta}{\alpha}\xi \end{aligned}$$

So in total:

$$\begin{aligned} \mathcal{X}_1 &\leq \mathcal{X}_0 - \frac{d+1}{\alpha} \\ \mathcal{X}_2 &\leq \mathcal{X}_0 - 2\frac{d+1}{\alpha} \\ \mathcal{X}_3 &\leq \mathcal{X}_0 - 3\frac{d+1}{\alpha} + \frac{\beta}{\alpha}\xi \\ \mathcal{X}_0 &\leq \mathcal{X}_0 - 4\frac{d+1}{\alpha} + \frac{\beta}{\alpha}\xi \\ \mathbf{Size}(\text{fork}) &\leq \frac{\alpha}{\beta} \mathcal{X}_r \end{aligned}$$

The first three inequalities hold by the definitions of the $\mathcal{X}_r$, the fourth is satisfied since $\frac{\beta}{\alpha}\xi = 5\frac{d+1}{\alpha}$, the last is obtained due to Claim 4.57, that states that the maximal size of a fork equals $2d'$, and that $\frac{\alpha}{\beta}\mathcal{X}_r$ is greater than $4d$.

Finally, by Claim 4.56, the graphs induced by $\mathbf{A}_i \cup \mathbf{F}_i$ are connected, thus the graph induced by $\mathbf{A} \cup \mathbf{F}$ is also connected. Therefore, $\mathbf{W}, \mathbf{A}, \mathbf{F}$ form an explanation of K .

It's left to prove for spanned slices in the analysis Toom's contours. Notice, that h has to be greater than zero, We prove the inequality by repeating on the induction bases, where all the spanned slices that are back in the history of K are spanned slices in the standard Toom's contour. So the only different is in the bounding of $\mathbf{A}$'s size. We use Claim 4.60 and get:

$$\begin{aligned} |\mathbf{A}| &\leq \alpha(m+1) - \beta s \\ &\quad + (-\alpha(1 + \mathcal{X}_{r-1}) + d + 1 - \mathcal{X}_{r-1}k\alpha + d + d' + 1 + \beta k \cdot \mathbf{Size}(\text{fork})) \\ &\leq \alpha(m+1) - \beta s \\ &\quad + (-\alpha + d + 1 - \mathcal{X}_{r-1}k\alpha + d + d' + 1 + \beta k \cdot \mathbf{Size}(\text{fork})) \end{aligned}$$

Where we used the fact that $\mathcal{X}_{r-1} \in (0, 1)$ for $r \in \{0, 1, 2, 3\}$. Clearly, the bound is satisfied since $\alpha \geq 2d + d' + 1$ and $\mathcal{X}_{r-1}\alpha \geq \beta \cdot \mathbf{Size}(\text{fork})$. $\square$

We summarize by using the linear relation between the leaves of a Toom's contour and the faults that induce it. That gives an upper bound on the probability of a slice occurring when the noise is local stochastic.

Corollary 4.63. *Let $\hat{K}$ be a spanned slice with size s , and explanation with $|A|$ arrows, and $|F|$ forks. Then it is induced by at least the following number of faults:*

$$\frac{1}{2^{d'}} \frac{1}{2} (|F| + 1 + \alpha^{-1}(|A| - 1) + \beta s)$$

Furthermore, the ratio of leaves to edges in an explanation is greater than $\frac{1}{2\alpha} = \frac{1}{80(d+1)^2}$.

Proof. We bound the Claim 4.62 by the number of leaves in the explanation, and use the fact that any d' -face has $2^{d'}$ vertices, so each fault contributes at most $2^{d'}$ leaves to the explanation. $\square$

4.10 Proof of the Theorem

We are ready to prove the main theorem (Theorem 4.67). We start by giving a formal definition of a simulation.

Definition 4.64 (Simulation and Logical Errors). *For a quantum code $\mathcal{C}$ of length n and a decoder for $\mathcal{C}$, denoted by D , we say that a Pauli operator $P \in \mathcal{P}^{\otimes n}$ is a logical error with respect to D if DP does not equal the logical identity.*

For quantum circuits C and $\tilde{C}$, a noise channel $\mathcal{E}$, and $p \in (0, 1)$, we say that $\tilde{C}$ simulates C with probability $1 - p$ if

1. *If there is a mapping between the qubits of C and the registers of $\tilde{C}$, along with a decoding procedure such that, at the end of running $\tilde{C}$, decoding the registers results in the same quantum states as those produced by running C .*
2. *The probability induced by $\mathcal{E}$ that the deviation on one of the registers has a logical error is less than p .*

Lemma 4.65. *For any d, d' , there are constants $a, b \in \mathbb{R}^+$ and $p_0 \in (0, 1)$ that depend only on d, d' and satisfy the following. Consider a quantum circuit C with depth μ and width w , and let $\mathcal{N}_{\mathcal{E}}$ be a local stochastic noise channel with a noise rate $p < p_0$. There is a Toric encoded circuit $\tilde{C}$ such that each of its quantum registers is encoded by an n -length, (d, d') -Toric code, and it simulates C with probability:*

$$1 - \Theta(((w + 2\mu)\mu n^2)^3) (ap^b)^{\frac{1}{d}}$$

To prove the lemma, we will need the following two claims. The first bounds, for a given regular graph, the number of its subgraphs having at most k edges. This claim will be used to infer the number of possible explanation trees for a given slice. The

second states that each tree has a subtree with fewer than half of the edges and a leaves-to-edge ratio that is at least half of the ratio of the original tree. This claim will be used to infer that any explanation has a subtree where, under the projection to the finite Toric, its faults are mapped uniquely, meaning no fault is counted twice.

Claim (Claim 5.4). *Let $G = (V, E)$ be a $\Delta + 1$ -regular graph, and let k be an integer less than $|E|$. Fix a vertex $r \in V$. There is a constant number α , depending only on Δ , such that the number of subgraphs rooted at r with fewer than k edges is at most α^k .*

Claim (Claim 5.7). *Let $\gamma, \beta \in \mathbb{R}_{>0}$ and n be a positive integer. For any tree T with $\tau \geq \beta n$ edges and m leaves such that $\tau \leq \gamma m$, there is a subtree T' of T with a number of edges τ' less than $\frac{\beta}{2}n$ and a number of leaves greater than $\frac{\beta}{4\gamma}n$.*

The proofs of the claims are given in the appendix. We continue with the proof of the lemma.

Proof. We construct $\tilde{C}$ as follows: for each input wire of C , we initialize the logical zero $|0\rangle$ of the (d, d') -Toric code. For each of the gates **H** and **S**, we initialize the encoded $|\mathbf{H}\rangle$ and $|\mathbf{S}\rangle$ states, and for each **T**, we initialize copies of both $|\mathbf{T}\rangle$ and $|\mathbf{S}\rangle$ states. The even layers of $\tilde{C}$ are sweep cycles, while on the odd layers, we set the realization of the logical gates. For **CX**, we set the transversal realization, and for **H**, **S**, and **T**, we use gate teleportation. Thus the width of $\tilde{C}$ expands to $wn + 2\mu n$. For gate teleportation, we use the decoder in Corollary 4.41, which runs at most at a depth of n . Thus, the depth of $\tilde{C}$ expands to $2(3\mu n)$.

The decoder successfully decodes any error that does not contain a deviation cluster larger than $\gamma n^{\frac{1}{d}}$. Thus, it remains to prove that with probability $1 - p$, no deviation cluster exceeds that size. To prove this, we use ??, which states that the probability of there being a time t with an error cluster larger than $\gamma n^{\frac{1}{d}}$ is bounded by the number of locations multiplied by the probability that step τ is the first to admit a deviation cluster of size $\gamma' n^{\frac{1}{d}}$, maximized over τ . This leads us to prove the following claim:

Claim 4.66. *There exist constants $c_3, c_4, c_5, c_6, \alpha \in \mathbb{R}^+$, such that for any integer s and for any step τ in the standard order of $\tilde{C}[\mathcal{E}]$, the probability that τ is the first step where there is a deviation cluster of size greater than s is less than:*

$$c_3(w + 2\mu)\mu n^2 (\alpha p^{c_4})^s + c_5((w + 2\mu)\mu n^2)^2 (\alpha p^{c_6})^{\frac{1}{2d}n^{\frac{1}{d}}}$$

Proof. Denote by B_τ the event where τ is the first step to have a deviation cluster of size greater than $\frac{1}{2}n^{\frac{d'}{d}}$, and by A_τ the event where all explanations of deviation clusters up to and including step τ have fewer than $\frac{1}{d}n^{\frac{1}{d}}$ edges. Thus:

$$\Pr_{\sim \mathcal{N}_\mathcal{E}}[B_\tau] \leq \Pr_{\sim \mathcal{N}_\mathcal{E}}[B_\tau | A_\tau] + \Pr_{\sim \mathcal{N}_\mathcal{E}}[\bar{A}_\tau]$$

We begin by bounding the contribution of the right term. According to Corollary 4.63, there exists a constant c_1 , depending only on d , such that the ratio of leaves to edges

in the explanation is at least c_1 . By Claim 5.7, it follows that any such explanation has more than $\frac{1}{d}n^{\frac{1}{d}}$ edges containing a subtree with fewer than $\frac{1}{2d}n^{\frac{1}{d}}$ edges, with a leaves-to-edges ratio greater than $\frac{1}{2}c_1$. Using the union bound, we bound the probability that there is a vertex in the base graph that is a root of such a subtree.

$$\Pr_{\sim \mathcal{N}_\varepsilon}[\bar{A}_\tau] \leq \bigcup_{v \in V_o, \tau' \leq \tau} \Pr_{\sim \mathcal{N}_\varepsilon}[v \text{ is a root of subtree in } \mathcal{H}_{\tau'} \\ \text{with less than } \frac{1}{2d}n^{\frac{1}{d}} \text{ edges and more than } \frac{1}{2}c_1 \frac{1}{2d}n^{\frac{1}{d}} \text{ leaves}]$$

Now, since any two connected vertices are at a space distance of at most

$$\max\{\mathbf{Size}(\text{arrow}), \mathbf{Size}(\text{fork})\} \leq d' \leq d$$

It follows that all the vertices in the subtree are at a space distance of at most $n^{\frac{1}{d}}$ from each other. Thus, no two of them have the same space coordinate modulo n . In particular, projecting the infinite plane back to the finite Toric sends each of the leaves of the subtree to a unique target, and therefore the number of faults that induce the explanation is at least:

$$\frac{1}{2^{d'}} \frac{1}{4d} c_1 n^{\frac{1}{d}}$$

Plugging into the union bounds yields

$$\Pr_{\sim \mathcal{N}_\varepsilon}[\bar{A}_\tau] \leq \tau \cdot 3(w + 2\mu)6\mu n^2 \alpha^{\frac{1}{2d}n^{\frac{1}{d}}} p^{\frac{1}{2d'} \frac{1}{4d} c_1 n^{\frac{1}{d}}} \leq ((w + 2\mu)6\mu n^2)^2 \left(\alpha p^{\frac{1}{2d'+1} c_1} \right)^{\frac{1}{2d}n^{\frac{1}{d}}}$$

It remains to bound the term. By Claim 4.62, there is a constant c_2 depending only on d , such that any explanation of a slice of size s has at least $c_2 s$ edges. Therefore, it suffices to bound the probability that the 'analysis Toom's contour', $\mathcal{H}_\tau$ admits an explanation with more than $c_2 s$ edges. Since the event is conditioned on the event A_τ , no explanation has more than $\frac{1}{d}n^{\frac{1}{d}}$ edges. Thus, the same argument as above shows that the projection of each explanation tree into the finite Toric sends its leaves to unique targets. Hence, we obtain:

$$\Pr_{\sim \mathcal{N}_\varepsilon}[B_\tau | A_\tau] \leq \bigcup_{v \in V_o, x \geq c_2 s} \Pr_{\sim \mathcal{N}_\varepsilon}[v \text{ is a root of explanation with number of edges } x | A_\tau] \\ \leq \sum_{x=c_2 s}^{\infty} (w + 2\mu)6\mu n^2 \left(\alpha p^{\frac{1}{2d'} c_2} \right)^x = (w + 2\mu)6\mu n^2 \frac{\left(\alpha p^{\frac{1}{2d'} c_2} \right)^s}{1 - \left(\alpha p^{\frac{1}{2d'} c_2} \right)}$$

Combing the bounds we have:

$$\Pr_{\sim \mathcal{N}_\varepsilon}[B_\tau] \leq (w + 2\mu)6\mu n^2 \frac{\left(\alpha p^{\frac{1}{2d'} c_2} \right)^{c_2 s}}{1 - \left(\alpha p^{\frac{1}{2d'} c_2} \right)} + ((w + 2\mu)6\mu n^2)^2 \left(\alpha p^{\frac{1}{2d'+1} c_1} \right)^{\frac{1}{2d}n^{\frac{1}{d}}}$$

□

Now, combined with ??, with a probability greater than:

$$1 - \Theta \left(((w + 2\mu)\mu n^2)^3 (\alpha p^{c_7})^{\gamma' n^{\frac{1}{d}}} \right)$$

No error cluster has a size greater than $\gamma n^{\frac{1}{d}}$, so the decoder successfully restores the logical state; in other words, no logical error occurs. $\square$

The lemma gives the theorem almost immediately. To complete the proof, we need to plug in the initialization phase, which initializes encoded zeros and resource states subjected to local stochastic noise. We emphasize that the resulting states are subjected to local stochastic noise (in addition to the computation). We neither prove nor describe that phase here; instead, we refer to [Gro19], [Got14], [FGL18], and [NP25].

Theorem 4.67. *There exists a constant $p_0 \in (0, 1)$ such that for any local stochastic noise channel $\mathcal{E}$ with rate $p < p_0$, and a quantum circuit C with depth μ and width w , there exists a Toric encoded circuit $\tilde{C}$, with depth $\Theta(\mu n)$ and width $\Theta((w + \mu)n)$ that simulates C with probability:*

$$1 - \Theta(\text{poly}(w, \mu, n)) p^{\Theta(n^{\frac{1}{4}})}$$

and in addition:

1. *After a short initialization stage (at depth $\Theta(\log n)$), each of the qubits is encoded in the n -length $4D$ Toric code.*
2. *An error correction cycle takes $O(1)$ depth, and the gates in each correction cycle don't depend on time. In particular, if C is the identity circuit, then after the initialization stage, the layers in $\tilde{C}$ are the same.*

References

- [Neu56] J. von Neumann. “Probabilistic Logics and the Synthesis of Reliable Organisms From Unreliable Components”. In: *Automata Studies*. Ed. by C. E. Shannon and J. McCarthy. Princeton: Princeton University Press, 1956, pp. 43–98. ISBN: 9781400882618. DOI: [doi:10.1515/9781400882618-003](https://doi.org/10.1515/9781400882618-003). URL: <https://doi.org/10.1515/9781400882618-003>.
- [Lev80] G. Lev. “Size Bounds and Parallel Algorithms for Networks”. PhD thesis. University of Edinburgh, 1980. URL: <https://era.ed.ac.uk/handle/1842/17015>.
- [SV82] Yossi Shiloach and Uzi Vishkin. “An $O(\log n)$ parallel connectivity algorithm”. In: *Journal of Algorithms* 3.1 (1982), pp. 57–67. ISSN: 0196-6774. DOI: [https://doi.org/10.1016/0196-6774\(82\)90008-6](https://doi.org/10.1016/0196-6774(82)90008-6). URL: <https://www.sciencedirect.com/science/article/pii/0196677482900086>.

- [AS87] Awerbuch and Shiloach. “New Connectivity and MSF Algorithms for Shuffle-Exchange Network and PRAM”. In: *IEEE Transactions on Computers* C-36.10 (1987), pp. 1258–1263. DOI: [10.1109/TC.1987.1676869](https://doi.org/10.1109/TC.1987.1676869).
- [BS88] Piotr Berman and J'anos Simon. “Investigations of fault-tolerant networks of computers”. In: *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing*. STOC '88. Chicago, Illinois, USA: Association for Computing Machinery, 1988, pp. 66–77. ISBN: 0897912640. DOI: [10.1145/62212.62219](https://doi.org/10.1145/62212.62219). URL: <https://doi.org/10.1145/62212.62219>.
- [CS96] A. R. Calderbank and Peter W. Shor. “Good quantum error-correcting codes exist”. In: *Physical Review A* 54.2 (Aug. 1996), pp. 1098–1105. DOI: [10.1103/physreva.54.1098](https://doi.org/10.1103/physreva.54.1098). URL: <https://doi.org/10.1103/2Fphysreva.54.1098>.
- [Ste96] Andrew Steane. In: *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences* 452.1954 (Nov. 1996), pp. 2551–2577. ISSN: 1471-2946. DOI: [10.1098/rspa.1996.0136](https://doi.org/10.1098/rspa.1996.0136). URL: <http://dx.doi.org/10.1098/rspa.1996.0136>.
- [Kit97] A Yu Kitaev. “Quantum computations: algorithms and error correction”. In: *Russian Mathematical Surveys* 52 (1997), pp. 1191–1249. URL: <https://api.semanticscholar.org/CorpusID:250816585>.
- [AB99] Dorit Aharonov and Michael Ben-Or. *Fault-Tolerant Quantum Computation With Constant Error Rate*. 1999. arXiv: [quant-ph/9906129](https://arxiv.org/abs/quant-ph/9906129) [[quant-ph](https://arxiv.org/abs/quant-ph/9906129)].
- [ZLC00] Xinlan Zhou, Debbie W. Leung, and Isaac L. Chuang. “Methodology for quantum logic gate construction”. In: *Physical Review A* 62.5 (Oct. 2000). ISSN: 1094-1622. DOI: [10.1103/physreva.62.052316](https://doi.org/10.1103/physreva.62.052316). URL: <http://dx.doi.org/10.1103/PhysRevA.62.052316>.
- [Den+02] Eric Dennis et al. “Topological quantum memory”. In: *Journal of Mathematical Physics* 43.9 (Sept. 2002), pp. 4452–4505. DOI: [10.1063/1.1499754](https://doi.org/10.1063/1.1499754). URL: <https://doi.org/10.1063/1.1499754>.
- [Ali+08] R. Alicki et al. *On thermal stability of topological qubit in Kitaev's 4D model*. 2008. arXiv: [0811.0033](https://arxiv.org/abs/0811.0033) [[quant-ph](https://arxiv.org/abs/0811.0033)]. URL: <https://arxiv.org/abs/0811.0033>.
- [Got14] Daniel Gottesman. *Fault-Tolerant Quantum Computation with Constant Overhead*. 2014. arXiv: [1310.2984](https://arxiv.org/abs/1310.2984) [[quant-ph](https://arxiv.org/abs/1310.2984)].
- [Bom15] Héctor Bombín. “Gauge color codes: optimal transversal gates and gauge fixing in topological stabilizer codes”. In: *New Journal of Physics* 17.8 (Aug. 2015), p. 083002. DOI: [10.1088/1367-2630/17/8/083002](https://doi.org/10.1088/1367-2630/17/8/083002). URL: <https://doi.org/10.1088/1367-2630/17/8/083002>.

- [FGL18] Omar Fawzi, Antoine Grospellier, and Anthony Leverrier. “Constant Overhead Quantum Fault-Tolerance with Quantum Expander Codes”. In: *2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, Oct. 2018, pp. 743–754. DOI: [10.1109/focs.2018.00076](https://doi.org/10.1109/focs.2018.00076). URL: <http://dx.doi.org/10.1109/FOCS.2018.00076>.
- [Gro19] Antoine Grospellier. “Constant time decoding of quantum expander codes and application to fault-tolerant quantum computation”. Theses. Sorbonne Université, Nov. 2019. URL: <https://theses.hal.science/tel-03364419>.
- [KP19] Aleksander Kubica and John Preskill. “Cellular-Automaton Decoders with Provable Thresholds for Topological Codes”. In: *Physical Review Letters* 123.2 (July 2019). ISSN: 1079-7114. DOI: [10.1103/physrevlett.123.020501](https://doi.org/10.1103/physrevlett.123.020501). URL: <http://dx.doi.org/10.1103/PhysRevLett.123.020501>.
- [Gac21] Peter Gacs. *A new version of Toom’s proof*. 2021. arXiv: [2105.05968](https://arxiv.org/abs/2105.05968) [cs.FL]. URL: <https://arxiv.org/abs/2105.05968>.
- [BB24] Nikolas P. Breuckmann and Simon Burton. “Fold-Transversal Clifford Gates for Quantum Codes”. In: *Quantum* 8 (2024), p. 1372. ISSN: 2521-327X. DOI: [10.22331/q-2024-06-13-1372](https://doi.org/10.22331/q-2024-06-13-1372). URL: <http://dx.doi.org/10.22331/q-2024-06-13-1372>.
- [GV24] Alexandre Guernut and Christophe Vuillot. *Fault-Tolerant Constant-Depth Clifford Gates on Toric Codes*. 2024. arXiv: [2411.18287](https://arxiv.org/abs/2411.18287) [quant-ph]. URL: <https://arxiv.org/abs/2411.18287>.
- [NP25] Quynh T. Nguyen and Christopher A. Pattison. *Quantum fault tolerance with constant-space and logarithmic-time overheads*. 2025. arXiv: [2411.03632](https://arxiv.org/abs/2411.03632) [quant-ph]. URL: <https://arxiv.org/abs/2411.03632>.
- [BDL26] Shankar Balasubramanian, Margarita Davydova, and Ethan Lake. “A local automaton for the 2D toric code”. In: *Quantum* 10 (2026), p. 2125. ISSN: 2521-327X. DOI: [10.22331/q-2026-06-02-2125](https://doi.org/10.22331/q-2026-06-02-2125). URL: <http://dx.doi.org/10.22331/q-2026-06-02-2125>.
- [DST26] Gesa Dünnweber, Georgios Styliaris, and Rahul Trivedi. *Quantum Memory and Autonomous Computation in Two Dimensions*. 2026. arXiv: [2601.20818](https://arxiv.org/abs/2601.20818) [quant-ph]. URL: <https://arxiv.org/abs/2601.20818>.
- [SST26] Jan M. Swart, Réka Szabó, and Cristina Toninelli. *Peierls bounds from Toom contours*. 2026. arXiv: [2202.10999](https://arxiv.org/abs/2202.10999) [math.PR]. URL: <https://arxiv.org/abs/2202.10999>.

5 Appendix

Below, a semi proof that in CSS code partial correction $Z^e \rightarrow Z^{\hat{e}}$ doesn’t add bit flip errors. It might adds a global phase. I should enter it in a section about CSS coding.

$$\begin{aligned}
& X^f Z^e |w + C_z^\perp\rangle \\
&= X^f Z^e X^w |C_z^\perp\rangle \\
&\text{apply } H^{\otimes n} \mapsto Z^f X^e Z^w |C_z\rangle \\
&= (-1)^{e \cdot w} Z^f Z^w X^e |C_z\rangle \\
&\text{partial correction } X^{e+\hat{e}} \mapsto (-1)^{(e+\hat{e})(f+w)} (-1)^{e \cdot w} Z^f Z^w X^{\hat{e}} |C_z\rangle \\
&\text{apply } H^{\otimes n} \mapsto (-1)^{(e+\hat{e})(f+w)} (-1)^{e \cdot w} X^f X^w Z^{\hat{e}} |C_z^\perp\rangle \\
&= (-1)^{(e+\hat{e})(f+w)} (-1)^{e \cdot w} (-1)^{\hat{e} \cdot w} X^f Z^{\hat{e}} X^w |C_z^\perp\rangle \\
&= (-1)^{(e+\hat{e})f} X^f Z^{\hat{e}} |w + C_z^\perp\rangle
\end{aligned}$$

5.1 Base case for Claim 4.33

Proof. The parity check matrices are:

$$H_{C_v} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix} \quad H_{\text{Stab}_v} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Where we mark in blue, brown, and red the first, second, and third types of checks.

$$G_{\mathcal{C}_v} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \quad G_{\text{Stab}_v} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

□

5.2 Time Coarse-Graining

In this section, we prove that when subjecting a circuit to local stochastic noise, changing the frequency at which we introduce the fault layers translates to rescaling the noise rate. For this purpose, we are abusing the notation "propagation" by reusing it to describe a different context.

Definition 5.1 (Layer propagation). *Let $C = \{u_i\}_{i=1}$ be a quantum circuit. For each time coordinate t , write $U_t := C|_t$. The family of non-empty t -cuts $\{U_t\}_t$ is called the layer decomposition of C .*

Let $t < t'$ be such that U_t and $U_{t'}$ are non-empty and $U_s = \emptyset$ for every $t < s < t'$. The forward propagation of the layer U_t across the next layer $U_{t'}$ is the operation that replaces the pair $(U_t, U_{t'})$ by a pair $(U_{t'}, V_{t \rightarrow t'})$ satisfying

$$\prod_{u \in U_t} u \prod_{v \in U_{t'}} v = \prod_{v \in U_{t'}} v \prod_{w \in V_{t \rightarrow t'}} w.$$

The layer $V_{t \rightarrow t'}$ is called the propagation expression of U_t across $U_{t'}$.

More generally, if $t < t'$, the forward propagation of U_t to time t' is obtained by iterating the above operation through the successive non-empty cuts between times t and t' . The resulting propagated layer is denoted by $V_{t \rightarrow t'}$ and is again called the propagation expression of U_t to time t' . The resulting circuit has layer decomposition

$$\dots, U_{t-1}, U_t, U_{t+1}, \dots, U_{t'}, U_{t'+1}, \dots \mapsto \dots, U_{t-1}, U_{t+1}, \dots, U_{t'}, V_{t \rightarrow t'}, U_{t'+1}, \dots$$

After propagation we update the time coordinates so that they agree with the new layer in which each gate is applied.

Definition 5.2 ($C \sim_P C'$). For quantum circuits C and C' , we write $C \sim_P C'$ if one can be obtained from the other by a finite sequence of layer propagations.

By construction, $\sim_P$ preserves the implemented channel. Indeed, if $C \sim_P C'$ and ρ is any mixed state on the input register, then

$$C \circ \rho = C' \circ \rho.$$

We state Claim 3.8 in the language of layer propagation.

Claim 5.3. Let $\mathcal{N}_\mathcal{E}$ be a local stochastic noise channel with rate p , and let C be a quantum circuit whose gates have arity at most Δ . Fix an even integer c . Then there exists a local stochastic noise channel $\mathcal{N}_{\mathcal{E}_q}$ with rate

$$q := \left(2^{H(\Delta^{-2c})\Delta^c} p \right)^{\Delta^{-c}},$$

such that $C[\mathcal{E}]_1 \sim_P C[\mathcal{E}_q]_{c/2}$. In particular, up to a constant depending only on Δ and c , the rate q behaves like $p^{\Delta^{-c}}$.

Proof of Claim 3.8. Partition the subjected circuit $C[\mathcal{E}]_1$ into consecutive blocks of c layers. Inside each block, propagate all the fault layers to the end of the block and then grouping the resultd expressions together. Doing this block by block yields a new circuit C' in which each block of $c/2$ computational layers is followed by a single fault layer. Therefore $C' = C[\mathcal{E}']_{c/2}$ for a suitable random fault set $\mathcal{E}'$, and by construction: $C[\mathcal{E}]_1 \sim_P C[\mathcal{E}']_{c/2}$.

It remains to be proven that $\mathcal{E}'$ is distributed according to a local stochastic channel. Fix one propagation expression $V^{(i)}$ and a subset $S \subset \text{Loc}(V^{(i)})$. Denote by $U_1^i, \dots, U_c^i$ the layers for which $V^{(i)}$ is their (grouped) expression. Denote the light cone of S in $U_1^i, \dots, U_c^i$ by $\Lambda(S)$. Since the fan is bounded by Δ , we have that $|\Lambda(S)| \leq \Delta^c |S|$, and any $S' \subset \Lambda(S)$ whose propagation covers S has size at least $|S'| \geq \Delta^{-c} |S|$.

Now, having $S \subset \text{Loc}(V^{(i)})$, implies that there is a minimal S' (the minimality is regarding inclusion) such that $S' \subset \text{Loc}(\mathcal{E})$. Denote by $\mathcal{W}(S)$ the set of such minimal S' , and observe that its size is at most:

$$|\mathcal{W}(S)| \leq \binom{\Delta^c |S|}{\Delta^{-c} |S|}$$

Hence, using the fact that $\mathcal{E}$ is an outcome of local stochastic noise channel with a noise rate p we have:

$$\begin{aligned} \Pr[S \subset \text{Loc}(V^{(i)})] &\leq \Pr\left[\bigcup_{S' \in \mathcal{W}(S)} \{S' \subset \text{Loc}(\mathcal{E})\}\right] \\ &\leq \sum_{S' \in \mathcal{W}(S)} \Pr[S' \subset \text{Loc}(\mathcal{E})] \\ &\leq \binom{\Delta^c |S|}{\Delta^{-c} |S|} p^{\Delta^{-c} |S|} \leq \left(2^{H(\Delta^{-2c})\Delta^c} p \right)^{|S|/\Delta^c} = q^{|S|}. \end{aligned}$$

□

5.3 Checks Commutation

We give an alternative proof of Claim 4.6 by straightforwardly multiplying checks. Consider subsets of generators $A, B \subset S$ with sizes $d' - 1$ and $d' + 1$, respectively. Thus, $\partial^+ \theta_{\mathbf{v}, \mathbf{A}}$ and $\partial^- \theta_{\mathbf{u}, \mathbf{B}}$ are X - and Z -checks. Let us examine their multiplication:

$$(\partial^+ \theta_{\mathbf{v}, \mathbf{A}})(\partial^- \theta_{\mathbf{u}, \mathbf{B}}) = \left(\sum_{g \in S/A} \theta_{\mathbf{v}, \mathbf{A} \cup \mathbf{g}} + \theta_{\mathbf{g}^{-1} \mathbf{v}, \mathbf{A} \cup \mathbf{g}} \right) \left(\sum_{g \in B} \theta_{\mathbf{u}, \mathbf{B}/\mathbf{g}} + \theta_{\mathbf{gu}, \mathbf{B}/\mathbf{g}} \right)$$

Split into cases based on whether v equals u .

1. If $v = u$, then for any $g, g' \in S$, it holds that $gv \neq u$, $g'^{-1}u$, and $v \neq g'^{-1}u$. Thus, only terms of the form

$$\theta_{\mathbf{v}, \mathbf{A} \cup \mathbf{g}} \theta_{\mathbf{u}, \mathbf{B}/\mathbf{g}'}$$

can contribute to the product, yet for any g, g' for which this term is not zeroing out, it follows that $g \in B$ and $g' \notin A$, and therefore the term

$$\theta_{\mathbf{v}, \mathbf{A} \cup \mathbf{g}'} \theta_{\mathbf{u}, \mathbf{B}/\mathbf{g}}$$

Is not zero as well. Therefore, the number of non-zero terms is even, namely the sum equals zero.

2. If there is g such that $gv = u$, then first notice that for any $g', g'' \in S/g \cup \{e\}$, we have $g'v \neq u$. So only terms of the form

$$\text{either } \theta_{\mathbf{gv}, \mathbf{A} \cup \mathbf{g}} \theta_{\mathbf{u}, \mathbf{B}/\mathbf{g}'} \text{ or } \theta_{\mathbf{v}, \mathbf{A} \cup \mathbf{g}'} \theta_{\mathbf{g}^{-1} \mathbf{u}, \mathbf{B}/\mathbf{g}}$$

can contribute. Similarly to the above, if

$$\theta_{\mathbf{gv}, \mathbf{A} \cup \mathbf{g}} \theta_{\mathbf{u}, \mathbf{B}/\mathbf{g}'} \neq 0$$

then $g \in B$ and $g' \notin A$. So

$$\theta_{\mathbf{v}, \mathbf{A} \cup \mathbf{g}'} \theta_{\mathbf{g}^{-1} \mathbf{u}, \mathbf{B}/\mathbf{g}}$$

is also not zeroing out, and again the number of non-zeros is even.

5.4 Counting Subtrees.

Claim 5.4. *Let $G = (V, E)$ be a $\Delta + 1$ -regular graph, and let k be an integer less than $|E|$. Fix a vertex $r \in V$. There is a constant number α , depending only on Δ , such that the number of subgraphs rooted at r with fewer than k edges is at most α^k .*

Proof. Denote by T_Δ and T_2 the infinity Δ and binary tress. There is an injection between the subtrees of G , rooted at v , with at most k edges, to the subtrees of T_Δ rooted at v , with at most k edges. Moreover, there is an injection from the subtrees of T_Δ rooted at v , with at most k edges to the subtrees of T_2 , rooted at v , with at most $k \log \Delta$ edges. The injection is obtained by replacing each of the vertex with a (complete) binary tree over $\Delta - 1$ -leaves, each leaf is associated with a outgoing edge of the original vertex.

Thus, it's enough to bound the number of of subtrees in T_2 rooted at v , with at most $k \log \Delta$ edges, that number is less than the number of binary tress with $k \log \Delta + 1$ leaves. For exact leaves number we get the $k \log \Delta$ Catalan number, which its limit is:

$$\sim \frac{4^{k \log \Delta}}{(k \log \Delta)^{\frac{3}{2}} \sqrt{\pi}} \leq \frac{\Delta^{2k}}{(k \log \Delta)^{\frac{3}{2}} \sqrt{\pi}}$$

Therefore we can bound by:

$$\sum_{j=1}^k \frac{\Delta^{2j}}{(j \log \Delta)^{\frac{3}{2}} \sqrt{\pi}} \leq \frac{\Delta^{2k}}{\sqrt{\pi} \log^{\frac{3}{2}} \Delta} \sum_{j=1}^k \frac{1}{j^{\frac{3}{2}}} \leq \zeta\left(\frac{3}{2}\right) \cdot \frac{\Delta^{2k}}{\sqrt{\pi} \log^{\frac{3}{2}} \Delta}$$

□

5.5 Tree Separator Lemma

For a tree $T = (V, E)$ and a vertex $v \in V$, we call the subtree obtained by taking the descendants of v the subtree rooted at v .

Claim 5.5. *Let $T = (V, E)$ be a tree with $m > 3$ leaves and τ edges, then it has a subtree with $m' \in (\frac{1}{3}m, \frac{2}{3}m)$ leaves and at most $\frac{m'}{m}\tau$ edges.*

Proof. Order the vertices according to their heights. Denote by $v \in V$ the lowest vertex for which the subtree rooted at v has at least $\frac{1}{3}m$ leaves. Now, denote by T_1 the subtree obtained through the following iterative process over v 's children:

1. Initialize T_1 as the subtree rooted at v .
2. For a child u of v :
 - (a) Check if erasing from T_1 the subtree rooted at u doesn't decrease the number of leaves below $\frac{1}{3}m$. If so, then we update T_1 to be the result of the erasure.

Clearly, the algorithm is defined such that T_1 has to have at least $\frac{m}{3}$ leaves. Furthermore, since any child of v is a root of a subtree with fewer than $\frac{m}{3}$ leaves (otherwise we get a contradiction to the minimality of v 's height), we have that if T_1 would have more than $\frac{2}{3}m$ leaves, then the subtraction of any of its children would yield a tree with at least $\frac{2}{3}m - \frac{1}{3}m = \frac{1}{3}m$ leaves. Namely, v has in T_1 a child u such that the subtraction of the subtree rooted at u from T_1 results in a subtree with more than $\frac{1}{3}m$ leaves. But if indeed there were such a child, then the algorithm would erase it, so there is not.

Now let $T_2 = T/T_1 \cup v$, namely the tree induced by the subtraction of T_1 's edges from T . Observe that since the summation of leaves in T_1 and T_2 is the leaves in T , and T_1 has no more than $\frac{2}{3}m$ leaves, and no less than $\frac{1}{3}$ leaves, it holds that the number of leaves in T_2 is also in the range $(\frac{1}{3}m, \frac{2}{3}m)$. We claim that at least one of the subtrees has fewer than $\frac{m'}{m}\tau$ edges.

To see this, denote by m_1, m_2 and by τ_1, τ_2 the leaves and the edges in T_1, T_2 respectively. Now:

$$\frac{\tau}{m} = \frac{\tau_1 + \tau_2}{m_1 + m_2} = \frac{m_1}{m_1 + m_2} \frac{\tau_1}{m_1} + \frac{m_2}{m_1 + m_2} \frac{\tau_2}{m_2}$$

Namely, $\frac{\tau}{m}$ is a weighted average of $\frac{\tau_1}{m_1}$ and $\frac{\tau_2}{m_2}$, and therefore it is greater than their minimum. Peak the subtree with the minimal quotient. $\square$

By repeating recursively on the above construction, we get the following corollary:

Corollary 5.6. *Let $T = (V, E)$ be a tree with m leaves and τ edges. Then, for any $\alpha > 0$, it has a subtree with $m' \in (\frac{1}{3}\alpha m, \frac{2}{3}\alpha m)$ leaves and at most $\frac{m'}{m}\tau$ edges.*

Claim 5.7. *Let $\gamma, \beta \in \mathbb{R}_{>0}$ and n be a positive integer. For any tree T with $\tau \geq \beta n$ edges and m leaves such that $\tau \leq \gamma m$, there is a subtree T' of T with a number of edges τ' less than $\frac{\beta}{2}n$ and a number of leaves greater than $\frac{\beta}{4\gamma}n$.*

Proof. Since the construction of claim 5.5 preserves the ratio of edges to leaves, we have that for a T' obtained using the construction, the number of edges satisfies $\tau' \leq \gamma m'$. So, it's enough to look for the maximal α such that:

$$\gamma m' \leq \gamma \frac{2}{3}\alpha m \leq \frac{\beta}{2}n \Rightarrow \alpha = \frac{3\beta}{4\gamma} \frac{n}{m}$$

Which gives $m' \geq \frac{1}{3}\alpha m = \frac{\beta}{4\gamma}n$. $\square$

6 Garbage

Now we are ready to prove that any error is equivalent, up to addition by stabilizer, to an error whose maximal L_{d+1} vertex (or its edge) sees a syndrome (and that maximum is equal to or lower than the maximal value of L_{d+1} in the original error). Figure 10 visualize the idea. Claim 6.1 introduces a 'single application' of Claim 4.33 to 'advance' the corner (the vertex that gets the maximal value of L_{d+1}). The corner might not be unique, so in the proof, we advance all the corners simultaneously. Yet, it might occur that the yielded assignment still doesn't admit corners that see a syndrome. Thus, Claim 6.2 repeats 'corners advancing' steps on until it ensures the desired outcome - that the corner vertex of the final yielded assignment sees a syndrome.

Claim 6.1. *Let z be a finite assignment. Suppose that*

$$\max \{L_{d+1}(u) : u \in z\} > \max \{L_{d+1}(u) : u \in \partial^- z\}$$

Then there is $\tilde{z} \sim_{\mathcal{C}_X} z$ such that:

$$\max \{L_{d+1}(u) : u \in \tilde{z}\} \leq \max \{L_{d+1}(u) : u \in z\} - 1$$

Proof. Let $v_1, v_2, \dots, v_k$ be the vertices in z that achieve the maximum for L_{d+1} , namely $v_1, v_2, \dots, v_k = \arg \max_{v \in z} L_{d+1}(v)$, and assume that $v_i \notin \partial^- z$ for any $i \in [k]$. (Notice that the maximum is indeed achieved since z is finite.)

Consider the i th vertex v_i . By definition, since $v_i \notin \partial^- z$, we have $\partial^- z|_{v_i} = 0$, and since v_i is a maximum point of L_{d+1} in z , it is also a local maximum, and therefore $z|_{v_i+} = z|_{v_i}$. Combining the two, we get that $z|_{v_i}$ is a codeword of the small code at v_i , namely $z|_{v_i} \in \mathcal{C}_{v_i}$. Thus, by Claim 4.33, there is a stabilizer $w_i \in \text{Stab}_{v_i}$ such that w_i and z agree on the positive faces of v_i , namely $w_i|_{v_i} = z|_{v_i}$.

Set $\tilde{z} \leftarrow z + \sum_i w_i$. Since $\sum_i w_i$ is a stabilizer, we have $\tilde{z} \sim_{\mathcal{C}_X} z$. In addition, for any $j \neq i$, it follows that $v_j \notin w_i$. Otherwise, there exists $S' \subset S$ such that $v_j \in \theta_{v_i, S'}/v_i$, and therefore $L_{d+1}(v_j) > L_{d+1}(v_i)$, in contradiction to v_i and v_j being both maximum points for L_{d+1} in $\partial^- z$. Thus, we have:

$$\tilde{z}|_{v_i} = (z + \sum_j w_j)|_{v_i} = (z + w_i)|_{v_i} = 0$$

Namely, for any i , the i th vertex v_i is not in $\tilde{z}$. Hence, the vertex domain of $\tilde{z}$ is contained in the union of the vertices of z with the vertices of $w_1, w_2, \dots, w_k$, substituting $v_1, v_2, \dots, v_k$. Overall, we get:

$$\begin{aligned} \max \{L_{d+1}(u) : u \in \tilde{z}\} &\leq \max \{L_{d+1}(u) : u \in z \bigcup_i w_i / \bigcup_i v_i\} \\ &\leq \max \{L_{d+1}(u) : u \in z\} - 1 \end{aligned}$$

□

Claim 6.2. *Let z be a finite assignment. Then there is $\tilde{z} \sim_{\mathcal{C}_X} z$ such that*

$$\max \{L_{d+1}(u) : u \in \tilde{z}\} = \max \{L_{d+1}(u) : u \in \partial^- z\}$$

Proof. Let z be a finite assignment, and let k be an integer such that $k \geq 1$ and

$$\max \{L_{d+1}(u) : u \in z\} = \max \{L_{d+1}(u) : u \in \partial^- z\} + k$$

For an integer $l \leq k$, consider the assignments $\tilde{z}_0, \tilde{z}_1, \dots, \tilde{z}_l$ defined as follow:

- The first element, $\tilde{z}_0 = z$.
- If the $i - 1$ th element satisfies

$$(\star) \quad \max \{L_{d+1}(u) : u \in \tilde{z}_{i-1}\} > \max \{L_{d+1}(u) : u \in \partial^- \tilde{z}_{i-1}\}$$

Then define the i th element, $\tilde{z}_i$, to be the result of applying the construction, given in Claim 6.1, on $\tilde{z}_{i-1}$, namely $\partial \tilde{z}_{i+1} = \partial \tilde{z}_i$ and

$$\max \{L_{d+1}(u) : u \in \tilde{z}_i\} \leq \max \{L_{d+1}(u) : u \in \tilde{z}_{i-1}\} - 1$$

If the $i - 1$ th element doesn't satisfies $\star$, then set $l = i - 1$.

- If the k th element was defined, set $l = k$.

Since $\sim_{C_X}$ is a transitive relation, we have that for any $i \in [l]$, $z \sim_{C_X} \tilde{z}_i$. If $l \neq k$, then there exists $\tilde{z}_i$ such that $\tilde{z}_i \sim_{C_X} z$ and $\tilde{z}_i$ satisfies $\star$, namely by setting $\tilde{z} = \tilde{z}_i$, we get the desired. So, it is left to handle the case where $l = k$. In that case, set $\tilde{z} = \tilde{z}_k$. By Claim 6.1:

$$\max \{L_{d+1}(u) : u \in \tilde{z}\} \leq \max \{L_{d+1}(u) : u \in \partial^- z\}$$

Yet, since $\tilde{z} \sim_{C_X} z$, it follows that $\partial \tilde{z} = \partial z$, So:

$$\max \{L_{d+1}(u) : u \in \partial^- z\} = \max \{L_{d+1}(u) : u \in \partial^- \tilde{z}\} \leq \max \{L_{d+1}(u) : u \in z\}$$

Therefore, we have the equality:

$$\max \{L_{d+1}(u) : u \in \tilde{z}\} = \max \{L_{d+1}(u) : u \in \partial^- z\}$$

□

Consider remove the proof, since it's trivial. Ideal decoder proof:

Proof. Notice that since $\mathbf{Size}(z) \leq n - 1$, we have that for any $g \in S$:

$$\max_{v \in z} L_g(v) - \min_{v \in z} L_g(v) \leq \mathbf{Size}(z) \leq n - 1$$

Thus, z doesn't spread on a non-trivial cycle and therefore translates to (a collection of) finite error in the ∞ -extension. Denote by ξ_t the corresponding error on the infinite plane after t sweep-cycles, and let ξ_0 be the corresponding error on the infinite plane prior to the first cycle. Since ξ_0 is finite, we use Claim 4.37 and have that each sweep-cycle decreases the boundary by at least one, namely:

$$\mathbf{Size}(\partial^- \xi_{t+1}) \leq \mathbf{Size}(\partial^- \xi_t) + 1$$

Therefore, after at most $\mathbf{Size}(\partial^- z) \leq \mathbf{Size}(z)$ iterations, the boundary of the error vanishes. In particular, the syndrome vanishes. It remains to prove that during the decoding, no logical codewords were turned on. Denote by $\tilde{\xi}_t$ the correction that was applied at cycle t . From Claim 4.36, it follows immediately that for any generator $g \in S$, it holds that:

$$\max \{L_g(v) : v \in \xi_0 \bigcup_t \tilde{\xi}_t\} \leq \max \{L_g(v) : v \in \xi_0\}$$

And since the sweep-rule defines that vertices flips only bits which they are in their positive local view, we also have:

$$\max \{L_{d+1}(v) : v \in \xi_0 \bigcup_t \tilde{\xi}_t\} \leq \max \{L_{d+1}(v) : v \in \xi_0\}$$

Thus, the support of all the qubits that were touched during the decoding, combined with the support of the initial error, satisfy:

$$\begin{aligned} \mathbf{Size}(\{v : v \in \xi_0 \bigcup_t \tilde{\xi}_t\}) &\leq \mathbf{Size}(\{v : v \in \xi_0\}) \\ &\leq \mathbf{Size}(z) \leq n - 1 \end{aligned}$$

□